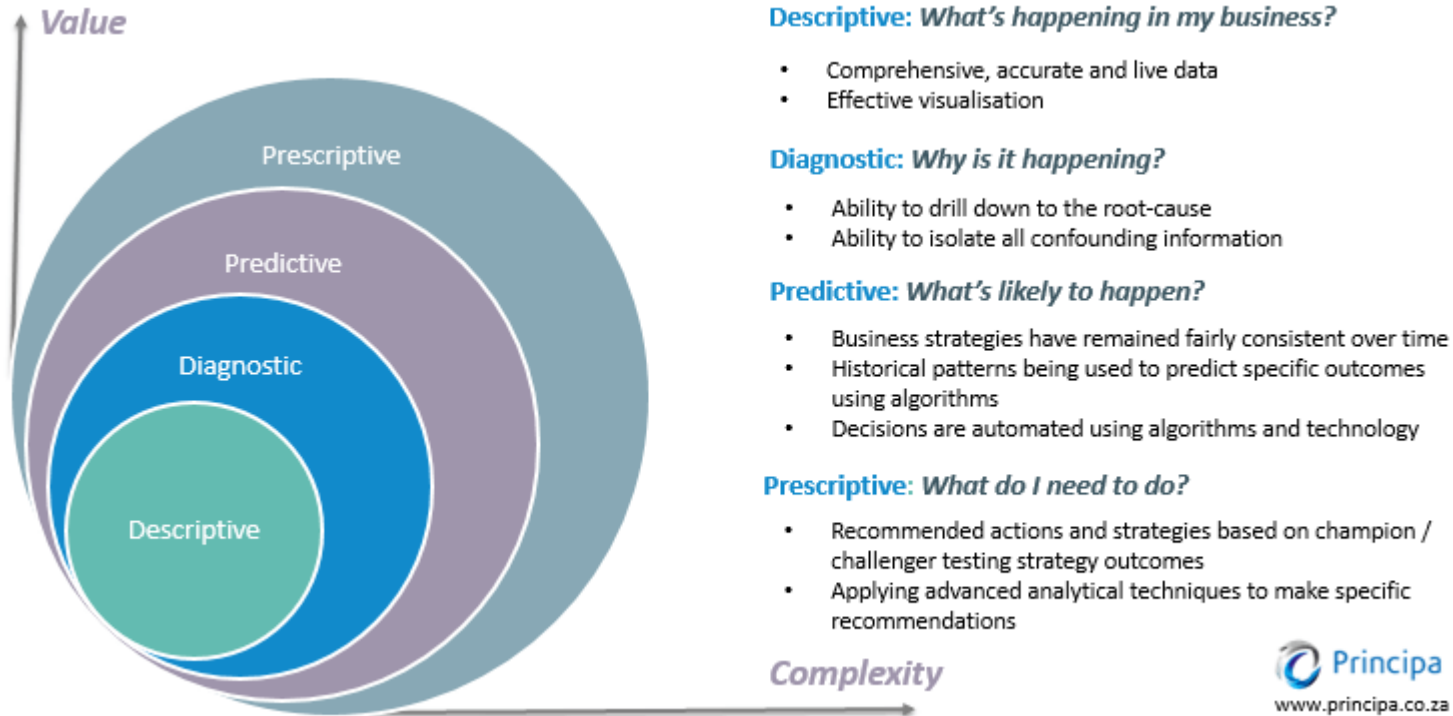


MACHINE LEARNING IS/IS2A

MARIE-ÉLÉONORE KESSACI

Extraction de connaissance



- Les techniques descriptives
= recherche de patterns
 - **Mettre en évidence des informations présentes** mais cachées par le volume de données
 - Réduire, résumer et synthétiser les données
 - Il n'y a pas de variables à expliquer
- Les techniques prédictives
= modélisation
 - **Extrapoler de nouvelles informations** à partir des informations présentes (c'est le cas du scoring)
 - Expliquer les données
 - Il y a une variable à expliquer

Description des données

- Intérêt
 - Favoriser la connaissance et la compréhension des données
- Informations sur le jeu de données
 - Nombre d'observations/individus (lignes)
 - Nombre d'attributs/caractéristiques (colonnes)
 - Nombre de fichiers (images, texte...)
 - Type de variables...
- Statistiques descriptives
 - Pour une variable donnée : la répartition de ses valeurs (tri, histogramme, moyenne, minimum, maximum, etc.).
 - Pour deux ou trois variables données : liens entre les répartitions des valeurs des variables, corrélations
 - Ex : répartition des votes par âge ou par tranche d'âge – corrélation entre âge et nb de votants

Description des données

CLUSTERING

- Le clustering (ou classification non supervisée ou segmentation) consiste à créer des classes/groupes (ie. des sous-ensembles) de données similaires entre elles et différentes des données d'une autre classe. Elle permet une vision générale de l'ensemble (de la clientèle, par exemple)
- Intérêt
 - Favoriser, grâce à la métatypologie, la compréhension et la prédiction
 - Fixer des segments qui serviront d'ensemble de départ pour des analyses approfondies
 - Réduire les dimensions, c'est-à-dire le nombre de variables/attributs, quand il y en a trop au départ
- Méthodes
 - Méthodes hiérarchique
 - Algorithmes des K-moyennes
 - Réseaux de Kohonen (= cartes auto-adaptatives)

Description des données

RÈGLES D'ASSOCIATION

- L'association consiste à trouver quelles valeurs des variables vont ensemble. Les règles d'association sont de la forme : si antécédent, alors conséquence. L'association ne fixe pas de variable cible. Toutes les variables peuvent à la fois être prédicteurs et variable cible. On appelle aussi ce type d'analyse une « analyse d'affinité ».
- Intérêt
 - Mieux connaître les comportements
- Méthodes
 - Algorithme Apriori
- Exemples :
 - Analyse du panier de la ménagère (si achat de fraises, alors achat de chantilly).
 - Étudier quelle configuration contractuelle d'un abonné d'une compagnie de téléphone portable conduit plus facilement à un changement d'opérateur.
 - Analyser les différents symptômes qui conduisent à une maladie

Prédiction

ESTIMATION D'UNE VARIABLE CIBLE : REGRESSION VS. CLASSIFICATION

- L'estimation consiste à définir le lien entre un ensemble de prédicteurs et une variable cible. Ce lien est défini à partir de données « complètes », c'est-à-dire dont les valeurs sont connues tant pour les prédicteurs que pour la variable cible. Ensuite, on peut déduire une variable cible inconnue de la connaissance des prédicteurs.
- Intérêt : Permettre l'estimation de valeurs inconnues.

Suivant le type de la variable cible, ce problème d'estimation porte un nom différent

- Régression : estimation d'une variable **numérique** (continue)
- Classification : estimation d'une variable **catégorielle**

Estimation

RÉGRESSION

- Méthodes :
 - Régression linéaire simple, régression multiple, intervalle de confiance, estimation de points
 - Réseaux de neurones
- Exemples :
 - Estimer la pression sanguine à partir de l'âge, le sexe, le poids et le niveau de sodium dans le sang.
 - Déterminer la valeur d'une maison pour sa mise en vente
 - Estimer le taux de clics pour une publicité en fonction des navigations passées

Estimation

CLASSIFICATION

- Méthodes :
 - Graphiques et nuages de points
 - Méthode des k plus proches voisins
 - Arbres de décision
 - Réseaux de neurones
- Exemples :
 - Segmentation par tranche de revenus : élevé, moyen et faible (3 segments). On cherche les caractéristiques qui conduisent à ces segments
 - Déterminer si un mode de remboursement présente un bon ou un mauvais niveau de risque crédit (deux segments)

Aide à la décision

Restitution des résultats descriptifs / prédictifs

- Aider à la compréhension des résultats
- Être adapter au public visé : opérationnels ? data scientist ?
- Outils de **visualisation**
 - Tableaux de bord
 - Graphiques
- **Interprétabilité** des modèles fournis
 - Attention aux modèles qui n'ont pas de sens pour les métiers
- **Explicabilité** des modèles fournis
 - Attention aux modèles dont l'obtention des résultats n'est pas claire

(Principaux) Métiers liés à la donnée pour IS

- Data **Analyst**
 - Spécialiste de l'analyse de donnée
 - S'adresse principalement aux opérationnels
 - Restitutions graphiques (ex : tableaux de bord)
- Data **Scientist**
 - Spécialiste de l'apprentissage automatique
 - Traite les données brutes
 - Utilise la dataviz pour restituer/expliquer ses observations
- Data **Engineer**
 - Spécialiste du stockage et préparation des données
 - Maîtrise les outils du Big Data
 - A l'interface des Data Analyst et Data Scientist

Objectifs du modules

- Connaître les principes fondamentaux de l'apprentissage automatique
- Savoir modéliser les problèmes d'apprentissage automatique
- Avoir une démarche rigoureuse et scientifique
- Savoir utiliser *intelligemment* la bibliothèque Scikit-Learn

Contenu du cours

- Rappeler les notions importantes liées au traitement de la donnée
- Définir l'apprentissage automatique
- Voir ou revoir différentes méthodes d'apprentissage supervisé
- Comprendre le rôle des hyperparamètres et appréhender l'automatisation
- Aborder les réseaux de neurones

Contenu des TP

- Se familiariser avec la bibliothèque Scikit-Learn (en python)
- Utiliser des méthodes de classification supervisée : arbres de décision et méthodes ensemblistes
- Comparer différents protocoles expérimentaux
- S'initier aux méthodes classiques d'hyperparamétrage
- Appréhender l'entraînement des réseaux de neurones

Modalités du cours

- Organisation :
 - 8h de cours
 - 10h de TP
 - 10h de projet encadré
- Evaluation :
 - 1 DS
 - 1 CC
 - 1 projet

SCIENCE DES DONNÉES



Traitement de la donnée

Préparation

Stockage / Pré-traitement



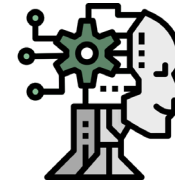
Analyse

Extraction de connaissance



Restitution

Aide à la décision



Disciplines fortement liées

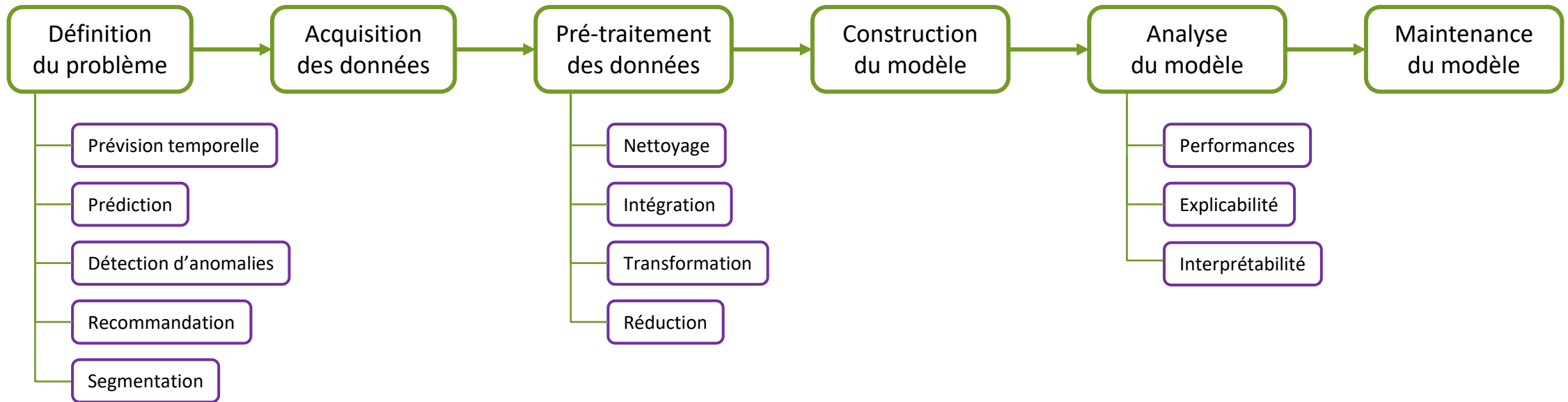
Intelligence Artificielle

Science des données

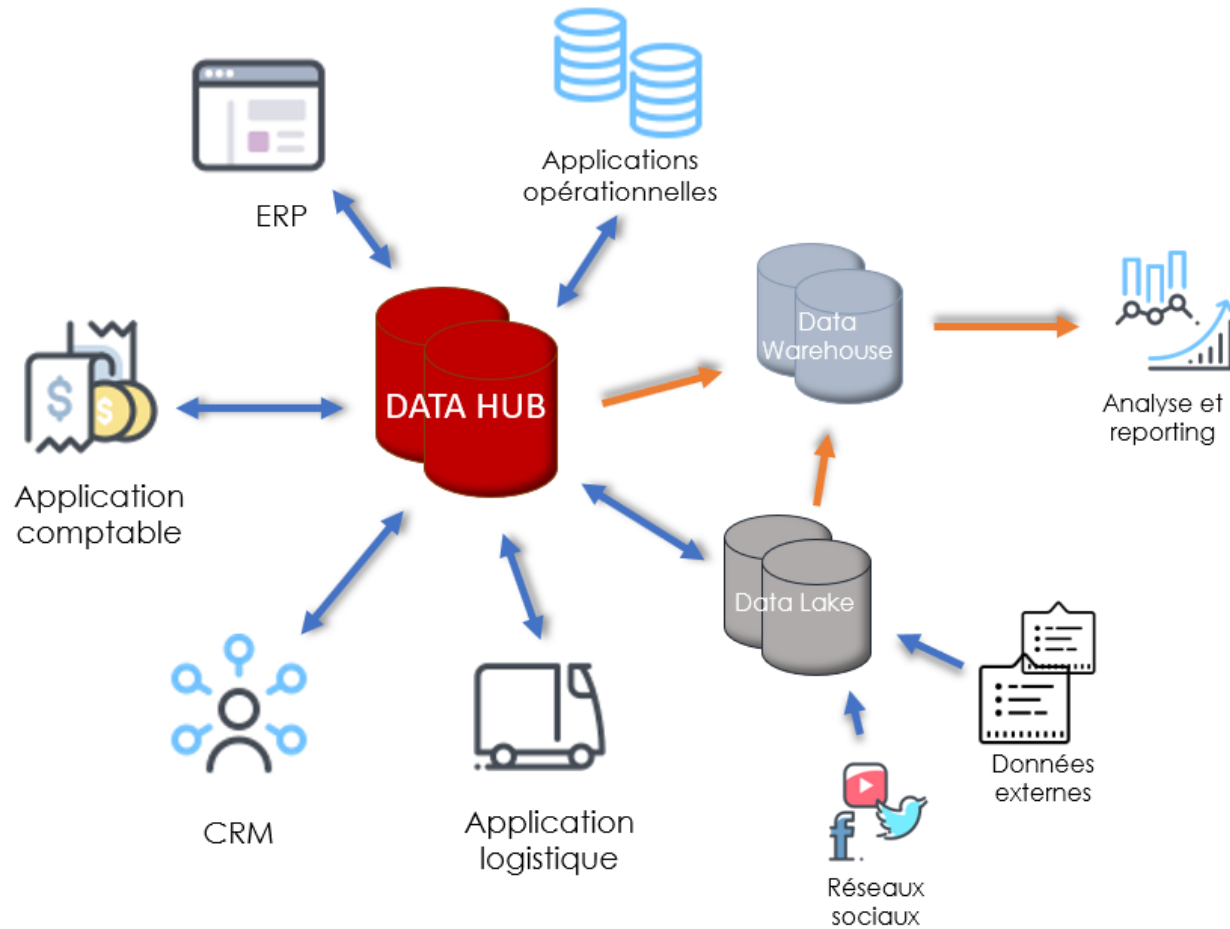
Apprentissage Machine
Machine Learning

Apprentissage Profond
Deep Learning

Etapes d'un projet en SCIENCE DES DONNÉES



Stockage

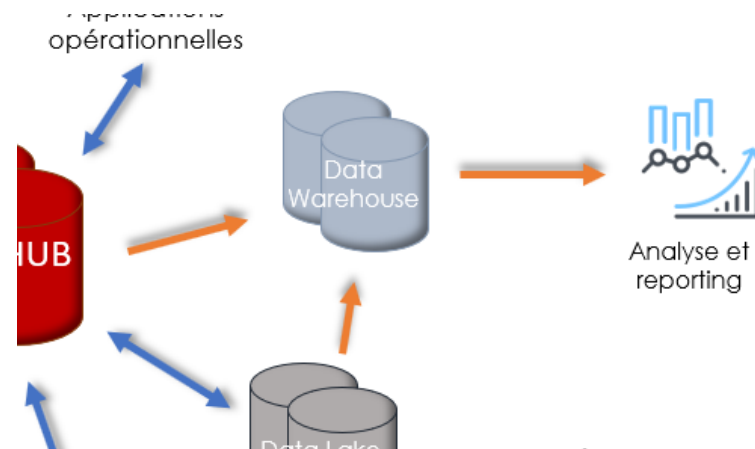


- Data Warehouse = *entrepôt de données*
 - Données agrégées et transformées
- Data Lake = *lac de données*
 - Données brutes
- Data Hub = *référentiel central de données*
 - Données harmonisées et indexées

Stockage

DATA WAREHOUSE

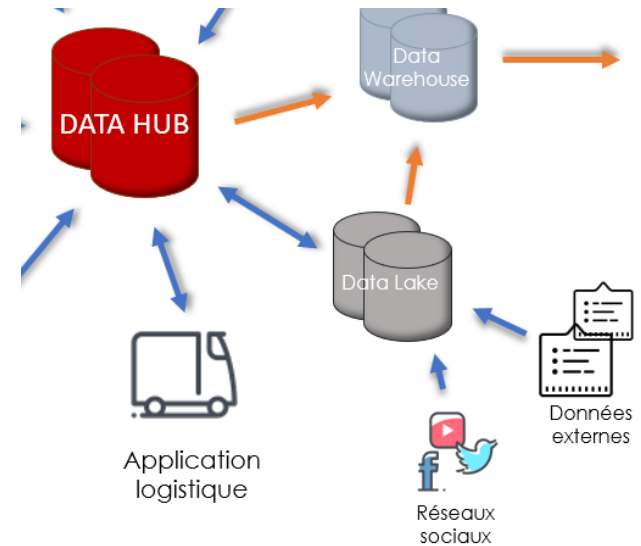
- Bases de données relationnelles
- Processus d'intégration de type ETL (*Extract – Transform – Load*)
 - Données “prêt à l'emploi”
 - Transformées, nettoyées, structurées
- Structure peu (pas) flexible
- A destination des opérationnels
 - Création de datamart (vue métier)
 - Source de la BI (analyse et reporting)



Stockage

DATA LAKE

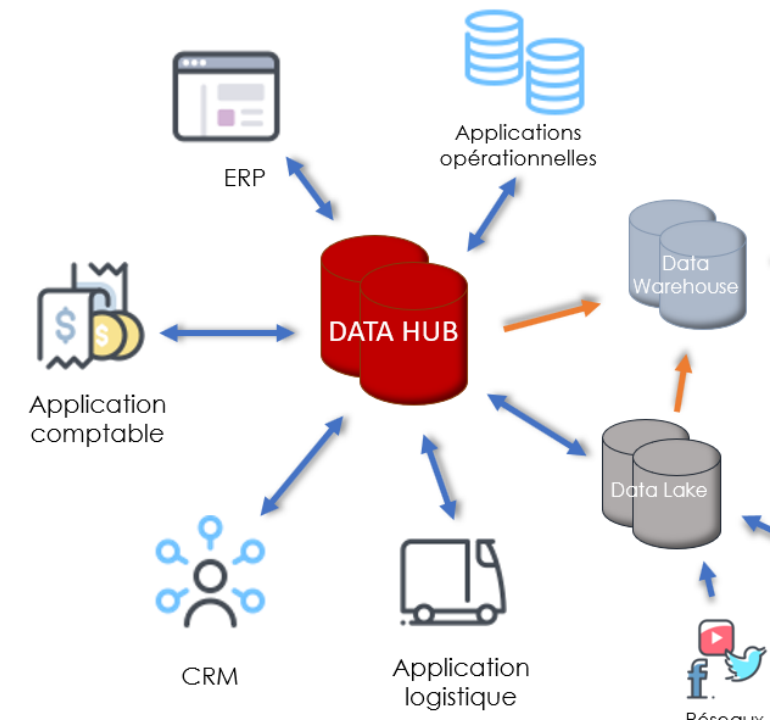
- Données brutes : aucune modélisation ou structuration nécessaire
 - Besoin d'une gouvernance
 - Environnement exploration/développement vs. production
- Centraliser les données en un point unique
- Naissance avec l'émergence du big data (volume, hétérogénéité...)
- Processus d'intégration de type ELT (*Extract – Load – Transform*)
- A destination des data scientists



Stockage

DATA HUB

- Données de l'entreprise + données externes
- Intègre des modèles de contrôle de données et de traçabilité
- Harmonisation des données
- Gestion temps réel
- A destination de tous les acteurs de la donnée



Stockage

	DATA WAREHOUSE	DATA LAKE	DATA HUB
Objectifs	Analyser	Stocker	Partager
Traitement des données	Transformées + Agrégées	(Consolidées)	Harmonisées + indexées
Qualité des données	Triées + nettoyées pour utilisation métier	Brutes	Haute pour gouvernance en temps réel
Utilisateurs	Data Scientists, Data Analysts, métiers	Data Scientists, développeurs	Data Analysts, métiers

Prétraitement des données



- Objectif : Avoir des données exploitables
- Quand ?
 - Cela dépend du stockage : DW, DL, DH, fichiers, web...
- Questions :
 - Que faire si certaines données sont manquantes ?
L'information est-elle non renseignée ou inexistante ?
 - L'adresse n'est pas enregistrée
 - Ce COV n'apparaît pas dans l'air exhalé du patient
 - Toutes les données sont-elles fiables (problème d'inconsistance) ?
 - Un même article appartient à différentes catégories (dans des magasins différents).
 - Le prix d'un même article est très supérieur à la normale dans un magasin donné.
 - Les unités de grandeur sont-elles les mêmes ?
 - Taille de vêtements ? Pointure ?

Etapes de prétraitement

- On parle de *preprocessing* de la donnée
- Nettoyage ou *Data cleaning*
 - Remplir les valeurs manquantes, modifier les données bruitées, identifier et supprimer les valeurs aberrantes (*outliers*), gérer l'inconsistance des données
- Intégration ou *Data integration*
 - Intégrer les données de plusieurs sources
- Transformation ou *Data transformation*
 - Normaliser, agréger, discrétiser les données
- Réduction ou *Data reduction*
 - Supprimer des observations/individus (« lignes ») ou des variables/attributs/caractéristiques (« colonnes »)

Nettoyage des données

DONNÉES MANQUANTES

- Définition
 - Valeurs de variables n'ayant pas été enregistrées
- Définir l'origine des données manquantes
 - Mauvais fonctionnement d'un équipement (appareil de mesure, machines...)
 - Manque de cohérence avec d'autres données et donc effacées en amont
 - Valeur non comprise et donc non enregistrée
 - Certaines données n'étaient pas considérées comme importantes au début de la collecte
 - Format ou contenu de la donnée a été modifiée au cours de la collecte

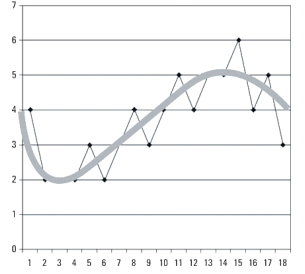
Nettoyage des données

GESTION / TRAITEMENT DES DONNÉES MANQUANTES

- Associer les experts métiers à ce processus
- Bcp d'algorithmes les ignorent => les modèles créés peuvent être incomplets ou imprécis
- Solutions :
 - Ne pas tenir compte des tuples contenant des données manquantes (valeurs nulles)
 - Peu efficaces si données éparses
 - Remplir manuellement les champs non remplis
 - Laborieux ou infaisable
 - Utiliser des valeurs par défaut
 - Utiliser des valeurs connues :
 - Remplacer un salaire manquant par le salaire médian des clients (de la même classe, si problème de classification)
 - Prédire les valeurs manquantes, en le déduisant d'autres paramètres (salaire à partir de l'âge et de la profession) via des techniques d'arbre de décision ou d'inférence bayésienne

Nettoyage des données

GESTION / TRAITEMENT DES DONNÉES BRUITÉES



- Erreur due à un instrument de mesure défectueux, pb de saisie, pb de transmission
- Groupement par classe (data binning) puis lissage par classe
 - Équi-largeur (distance) : n intervalles de même taille
-> méthode simple mais qui peut donner trop d'importance aux outliers
 - Équi-profondeur : n intervalles contenant le même nombre de valeurs
-> méthode qui répartit plutôt équitablement les valeurs dans les classes et qui gère plutôt bien les valeurs biaisées

■ Segmentation / Clustering

- Les valeurs similaires sont placées dans une même classe
- On ne tient pas compte des valeurs isolées

■ Régression linéaire

Exemple de binning

Soient les données : {4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34}

1. Trier les différentes valeurs de l'attribut considéré en 3 ensembles

1. Équi-profondeur : {{4, 8, 9, 15}, {21, 21, 24, 25}, {26, 28, 29, 34}}

2. Équi-largeur : $(34-4)/3=10$: {{4, 8, 9}, {15, 21, 21, 24}, {25, 26, 28, 29, 34}}

2. Remplacer les valeurs initiales par de nouvelles valeurs en fonction du partitionnement réalisé

- par la valeur moyenne des regroupements réalisés {{9, 9, 9, 9}, {23, 23, 23, 23}, {29, 29, 29, 29}}
- par les min et max des regroupements réalisés (lissage par borne)

{{4, 4, 4, 15}, {21, 21, 25, 25}, {26, 26, 26, 34}}

Exemple de régression linéaire

Hypothèse : un attribut Y dépend linéairement d'un attribut X

ex: Années d'expérience X et salaire Y

- Trouver les coefficients a et b tels que $Y = aX + b$
- Remplacer les valeurs de Y par celles prédites

Intégration des données

Fusionner des données de différentes sources dans une base de donnée cohérente

- Multiplicité des sources de données
 - Systèmes de mesure différents : Température, Taille, Pointure, Distance...
 - Doublons
 - Informations identiques stockées différemment : profession, adresse
 - Problème de terminologies : couleur
- Solutions :
 - Utiliser des référentiels connus : les CSP, des adresses conformes aux normes postales
 - Uniformiser les mesures, les catégories
 - Eliminer les données redondantes : exactement pareil, à partir d'un certain seuil



Prune



Violet

Transformation des données

- Normalisation
 - Permettre la comparaison des variables avec des valeurs très différentes
- Discrétisation
 - Discrétiser les variables à valeurs continues
- Construction de nouvelles variables
 - Créer de nouvelles variables à partir des attributs déjà présents
ex : variation du profit sur plusieurs années consécutives
- Mappage dans un espace vectoriel
 - Transformer des informations en attributs quantitatifs
ex : fréquence d'apparition d'un évènement

Transformation des données

STANDARDISATION / NORMALISATION DES DONNÉES = DATA SCALING

- Permettre de réduire la complexité des modèles et d'égaliser le poids de chaque dimension
- Préalable à l'application de certains algorithmes de clustering (k-means) ou pour faire une ACP. Aucun impact sur d'autres (arbres de décisions)
- Standardisation transforme les données pour suivre une loi normale centrée réduite $\mathcal{N}(0,1)$
 - Facilite la convergence lors d'une descente de gradient
 - Meilleure gestion des *outliers*
- Normalisation Max-min transforme les données dans l'intervalle $[0;1]$ et permet de s'affranchir des problèmes d'échelle
 - Utile pour les *classifiers* basés sur la distance: k-NN, artificial NN
 - Mais prise en compte des *outliers* trop importante

Standardisation (Normalisation Z-score)	Normalization Max-Min
$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation}(x)}$	$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)}$

Transformation des données

DISCRETISATION

- Il existe 3 types d'attributs
 - Nominal (= catégoriel) : ensembles de valeurs non ordonnés
 - Ordinal : valeurs dans un ensemble ordonné
 - Continu (=numérique) : ensemble des nombres réels
- Intérêt
 - Réduire le nombre de valeurs d'un attribut à valeur réelle en partitionnant les valeurs dans des intervalles
 - Les label des intervalles remplacent les valeurs brutes
- Motivation pour discrétiser les données
 - Certains algorithmes n'acceptent que des attributs catégoriels
 - Peut aider à la compréhension des données (ex: pattern)
- Méthodes
 - Binning
 - Analyse par clusters
 - Discrétisation basée sur l'entropie

Réduction des données

- Motivation

- Améliore la performance des méthodes : le temps d'exécution des algorithmes est souvent (très) linéaire en fonction du nombre d'observations et du nombre de variables
- Améliore la qualité du modèle : suppression des données parasites pour obtenir des modèles plus performants + réduction de l'over-fitting
- Améliore l'interprétabilité du modèle : réduction de la complexité du modèle en utilisant moins de variables

- Moyen

- Réduire le nombre de variables ou d'observations

- Méthodes

- Sélection d'attributs (*feature selection*) : sélection du sous-ensemble d'attributs les plus pertinents
- Analyse en composantes principales (ACP) : projection dans un nouvel espace vectoriel de dimension réduite

APPRENTISSAGE AUTOMATIQUE



Contexte

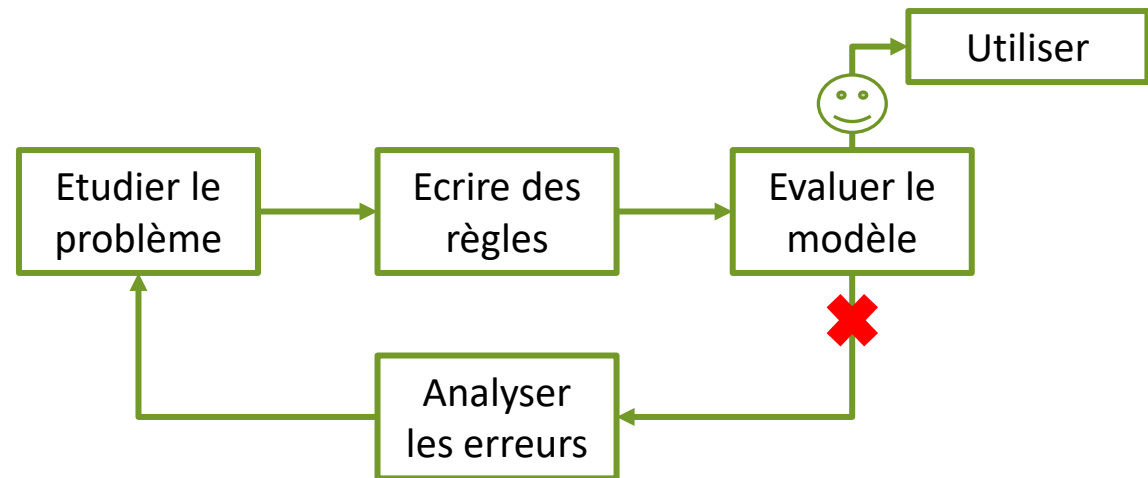
- Machine learning is a **science of the artificial**. The field's main objects of study are artifacts, specifically algorithms that improve their performance with experience [Langley, 1996]
- Machine learning is the study of **computer algorithms** that **improve automatically through experience** [Mitchell, 1997]
- Machine learning is programming computers to **optimize** a performance criterion using **example data or past experience** [Alpaydin, 2004]

Intérêt de l'AA -- Exemple

DÉTECTION AUTOMATIQUE DE SPAMS

Approche Traditionnelle

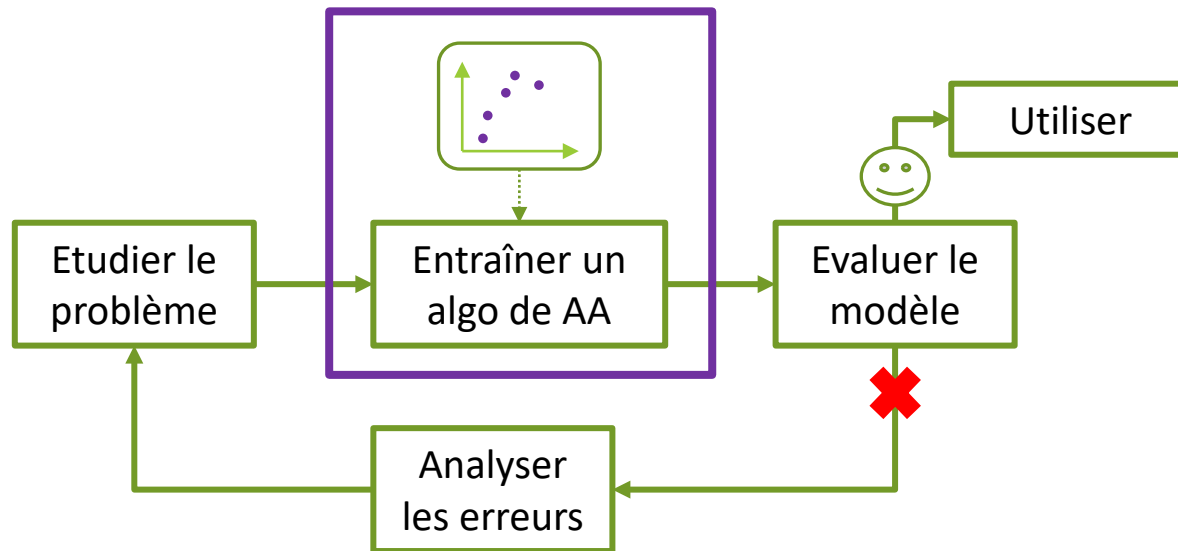
- Etape 1 : Examiner plusieurs messages spam pour trouver des mots/phrases qui permettent de détecter si un message est un spam
- Etape 2 : Ecrire un algorithme de détection des motifs (*pattern*) caractéristiques des spams
- Etape 3 : Evaluer la performance de cet algorithme à bien étiqueter les spams et recommencer à partir de l'étape 1 tant que la performance souhaitée n'est pas atteinte...



Intérêt de l'AA -- Exemple

DÉTECTION AUTOMATIQUE DE SPAMS

Approche avec AA

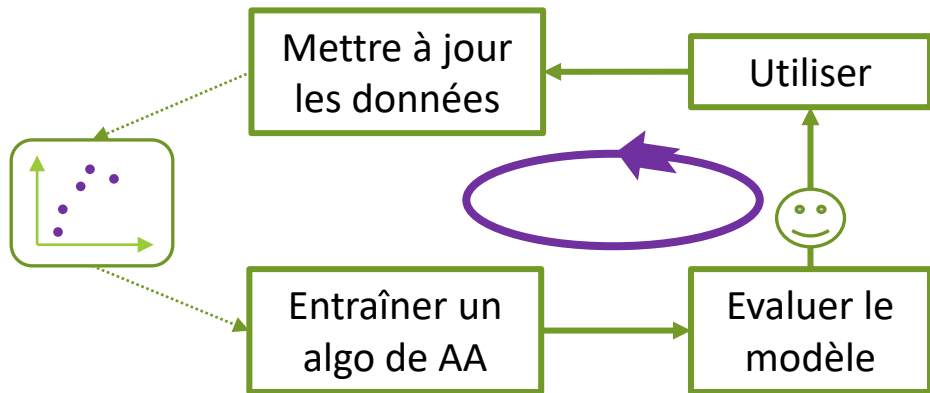


- L'algorithme de AA **apprend automatiquement** quels (groupes de) mots sont de bons prédicteurs de spams en détectant des **motifs étrangement fréquents**.
- Le programme dans son ensemble est **plus court**, plus facilement **maintenable** et normalement plus **exact**.

Intérêt de l'AA -- Exemple

DÉTECTION AUTOMATIQUE DE SPAMS

Approche avec AA dynamique



- L'utilisation du modèle permet d'observer de nouveaux spams
- Le programme met à jour les données
- Le modèle **s'adapte « en temps réel »** à des nouvelles techniques de spam

Apprentissage automatique

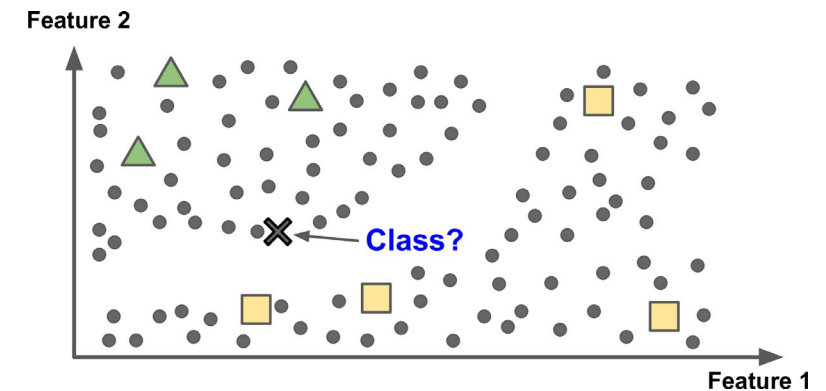
Les approches peuvent être catégorisées (de manière non exclusive) selon que :

- elles soient entraînées ou non en utilisant une ressource humaine
 - **Apprentissage supervisé, non-supervisé, semi-supervisé ou par renforcement**
- elles soient capables ou non d'apprendre à la volée de manière incrémentale
 - **Apprentissage online vs. par lots**
- elles fonctionnent en comparant les nouvelles données avec des données connues ou bien en détectant des nouveaux motifs pour construire un nouveau model prédictif
 - **Apprentissage par instance ou modèle**

Exemple d'un détecteur automatique de spams basé sur un réseau de neurones qui apprend à la volée en utilisant des données d'exemples mises à jour régulièrement
= Apprentissage supervisé, online par modèle

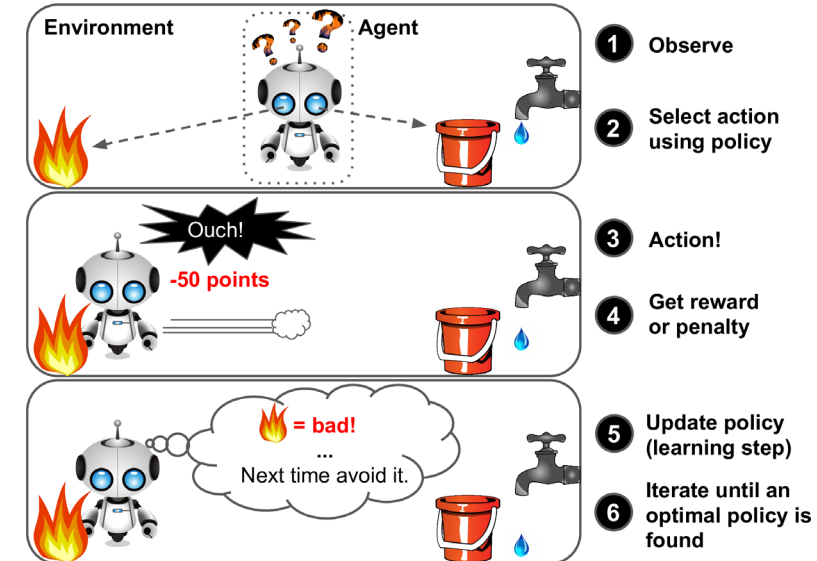
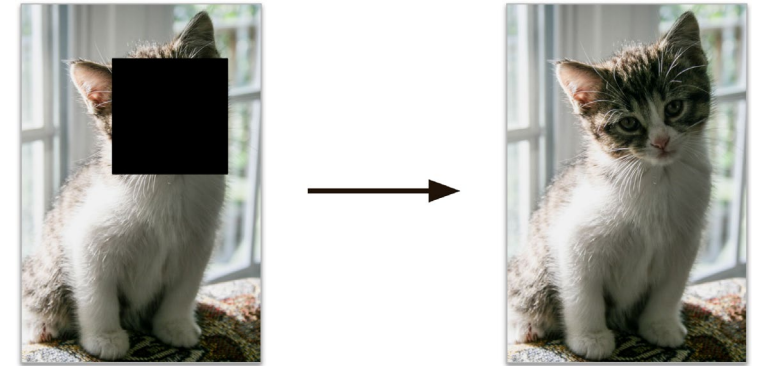
Apprentissage Supervisé/Non-supervisé

- **Apprentissage supervisé** (*supervised learning*)
 - Correspond aux problèmes de classification ou de régression
 - L'ensemble d'apprentissage inclus la solution pour chaque observation (label en classification, valeur en régression)
- **Apprentissage non-supervisé** (*unsupervised learning*)
 - Correspond aux problèmes de segmentation, détection d'anomalie/nouveauté, réduction de dimensions, règles d'association
 - L'ensemble d'apprentissage n'inclus pas de connaissance a priori. Le système doit apprendre sans enseignant.
- **Apprentissage semi-supervisé** (*semisupervised learning*)
 - Seule une faible partie des données d'apprentissage sont étiquetées
 - Combinaison des 2 techniques précédentes d'apprentissage
 - Inclus l'apprentissage actif (*active learning*)



Apprentissage Supervisé/Non-supervisé

- **Apprentissage auto-supervisé** (*self-supervised learning*)
 - Méthode intermédiaire entre de l'apprentissage supervisé et non-supervisé
 - Basé sur les réseaux de neurones
 - Consiste à masquer une partie des données d'apprentissage, puis entraîner le modèle à prédire et identifier ces données cachées. Le modèle génère ainsi lui-même l'étiquetage des données.
- **Apprentissage par transfert** (*transfer learning*)
 - Un modèle conçu pour une tâche est réutilisé pour une tâche différente
- **Apprentissage par renforcement** (*reinforcement learning*)
 - Très différent des précédents types d'apprentissage
 - Le système d'apprentissage est considéré comme un agent qui observe son environnement, sélectionne et exécute des actions et observe le gain/sanction en retour



Apprentissage Online vs. Par lots

- **Apprentissage par lots** (Batch learning)

- Le modèle est entraîné a priori en utilisant toutes les données disponibles.
- Aussi appelé apprentissage *offline*
- La mise à jour des données entraîne un nouvel entraînement du modèle.
- Nécessite un temps de calcul très importants quand le volume de données est trop important

- **Apprentissage online**

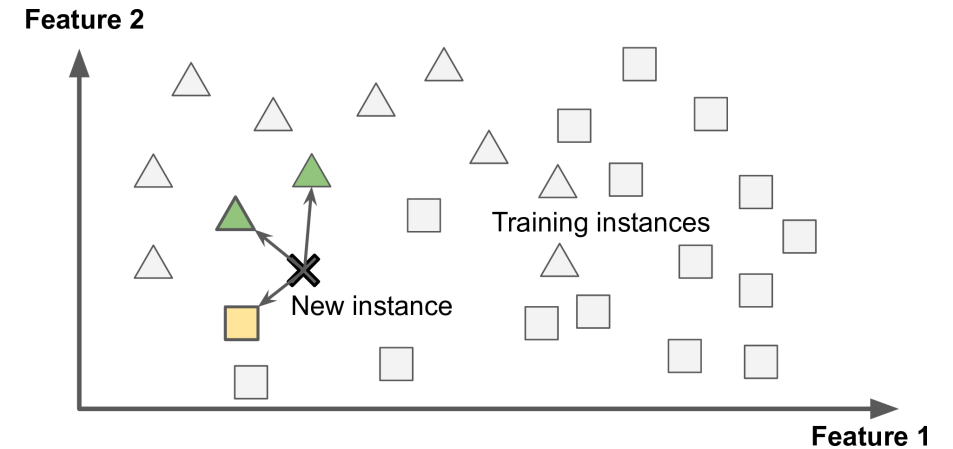
- Entraînement du système se fait de manière incrémentale par l'ajout (ou la mise à jour) de nouvelles données.
- Ajout de données soit séquentiel, soit individuel, soit par lots
- Adaptés aux systèmes qui reçoivent un flot continu de données
- Dépend du *learning rate*
- Vigilance autour de la qualité des données reçues continuellement

Apprentissage par Instance / Modèle

Concerne la généralisation du modèle construit.

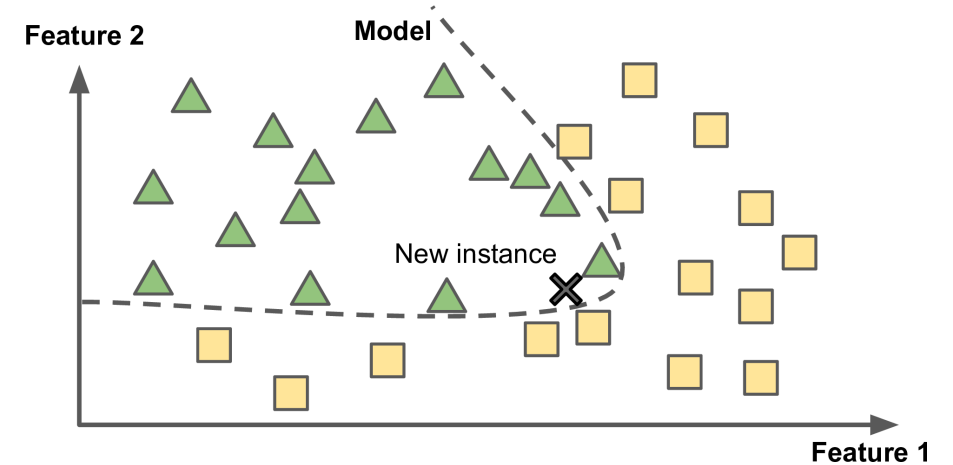
- **Apprentissage par instance** (*instance-based learning*)

- Utilise une mesure de similarité pour catégoriser de nouvelles observations



- **Apprentissage par modèle** (*model-based learning*)

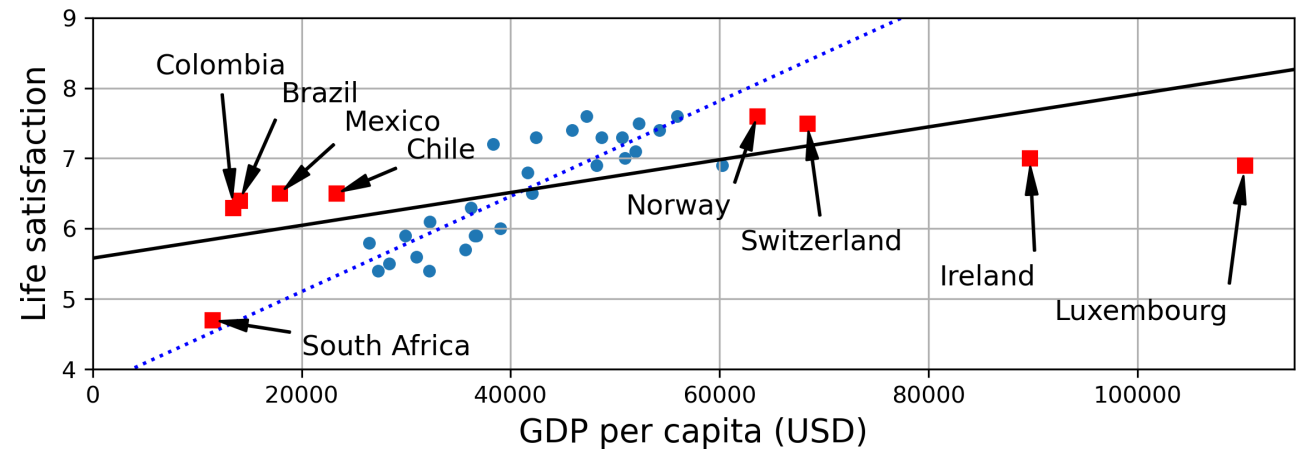
- Construit un modèle pour *prédire* la catégorie des nouvelles observations



Principaux défis de l'AA

PROBLÈMES LIÉS AUX DONNÉES D'ENTRAÎNEMENT

- Quantité insuffisante
- Non représentativité
 - *Bias sampling*
 - Pas assez de données -> modèle peu robuste
 - Trop de données -> modèle peu représentatif
- Mauvaises qualités
 - erreurs de mesures, données manquantes...
- Attributs non pertinents

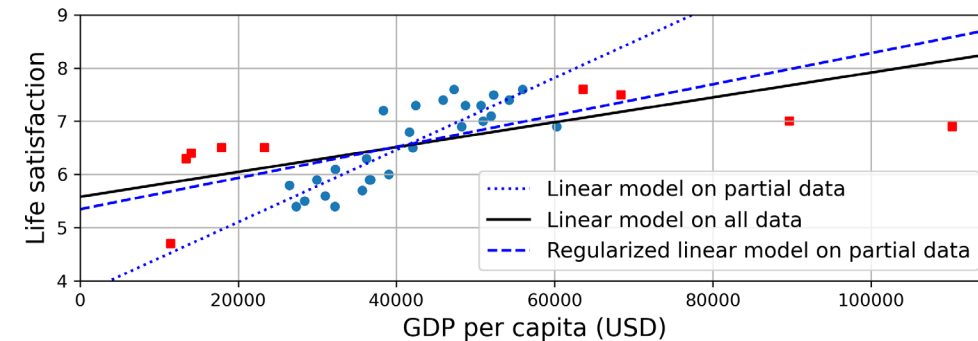
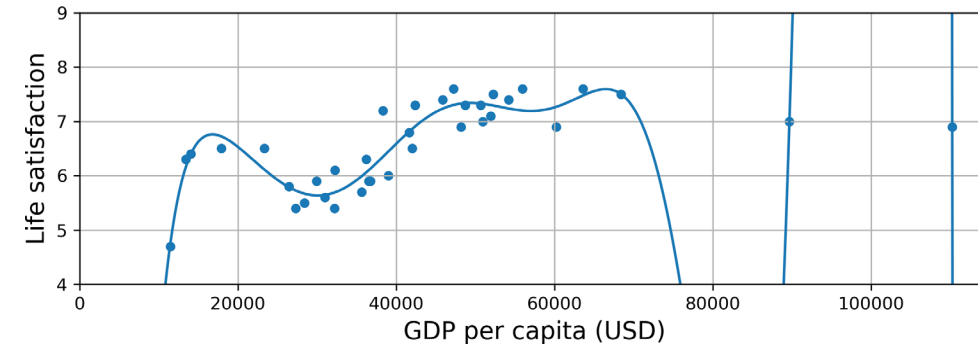


Principaux défis de l'AA

PROBLÈMES LIÉS À LA CONSTRUCTION DU MODÈLE

Sur-ajustement (*overfitting*) des données d'entraînement

- Le modèle est excellent pour prédire des données connues mais est très mauvais dans la prédiction de données inconnues : problème de généralisation du modèle
- Se produit plutôt pour les méthodes non linéaires ou non paramétriques
- Moyens pour éviter le sur-ajustement
 - Simplifier le modèle (ex : sélectionner moins de paramètres, réduire le nb d'attributs utilisés, donner des contraintes)
 - Ajouter plus de données d'entraînement
 - Réduire le bruit dans les données (supprimer des erreurs, des valeurs aberrantes)



Principaux défis de l'AA

PROBLÈMES LIÉS À LA CONSTRUCTION DU MODÈLE

Sous-ajustement (*underfitting*) des données d'entraînement

- Le modèle construit est trop simple pour apprendre la structure sous-jacente des données
 - Exemple : utiliser un modèle linéaire quand les données ne le sont pas
- Moyens pour éviter le sous-ajustement
 - Sélectionner un modèle plus puissant avec plus de paramètres
 - Alimenter le système avec des données de meilleure qualité
 - Réduire les contraintes sur le modèle
 - Donner plus de temps à la construction du modèle

APPRENTISSAGE SUPERVISÉ



Apprentissage supervisé

- Tâche de l'apprentissage automatique
- Objectif :
 - Proposer un **modèle de prédiction à partir d'exemples étiquetés/annotés/labellisés**
- Distinction entre les problèmes de **régression** et de **classification**
- Régression
 - Recherche à estimer une valeur dans un ensemble continu de réels
 - Modèle est appelé régresseur (=regressor)
- Classification
 - Recherche à attribuer une étiquette (un label) à chaque nouvelle entrée dans un ensemble fini de valeur
 - Modèle est appelé classifieur (=classifier)

Construction d'un modèle d'AS

- Identifier le problème d'apprentissage à traiter
 - Classification ? Régression ?
- Besoins
 - Jeu de données d'**entraînement** (=apprentissage) + jeu de données de **test** (=validation)
 - Méthodes de construction d'un modèle : algorithmique , statistique, algébrique
 - Définir et mesurer la performance d'une méthode ? d'un modèle ?
 - Comparer plusieurs méthodes/modèles pour sélectionner le *meilleur*

Jeux de données

- 1er cas :
 - Découpage du jeu de données : $\{\text{Entraînement}\}^K + \{\text{Test}\}^M$
 - Ex : $K=2/3$; $M=1/3$



- 2e cas :
 - Validation croisée à k blocs (*k-fold cross-validation*)



Jeux de données

Points de vigilance

- Données triées / ordonnées
 - Mélanger aléatoirement le jeu de données (par lignes)
- Déséquilibre des classes
 - Conserver le même équilibre dans les sous-ensembles d'entraînement et de test (voir *stratified k-fold cross-validation*)
 - Ré-équilibrer le jeu de données
 - Undersampling : retirer aléatoirement des observations de la classe majoritaire
 - Oversampling : dupliquer aléatoirement des observations de la classe minoritaire -> attention à l'overfitting
 - Générer des instances via des algorithmes spécialisés (ex: SMOTE)
 - Pondérer les classes lors de la mesure de la performance

Méthodes

- De nombreuses méthodes sont envisageables et **il n'existe pas une méthode meilleure** que toutes les autres sur tous les jeux de données
 - Nécessité d'en tester plusieurs !!
- La majorité des méthodes sont paramétrables afin de les rendre plus performantes sur la construction du modèle.
Ces paramètres sont appelés des **hyper-paramètres**.
 - Nécessité de tester plusieurs combinaisons mais peut être fastidieux...
 - Utiliser des méthodes de paramétrage automatique (ex: random search, grid search)

Mesurer la performance d'une régression

- **MSE** – Mean Square Error
 - **RMSE** – Root Mean Square Error
 - **R²** – R-squared
 - **MAE** – Mean Absolute Error
 - **MSPE** – Mean Square Percentage Error
 - **MAPE** – Mean Absolute Percentage Error
-
- MSE, RMSE et R² : même information
 - MAE : robuste aux outliers
 - MSPE, MAPE : pas de valeurs à zéro

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

$$R^2 = 1 - \frac{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}{\frac{1}{N} \sum_{i=1}^N (y_i - \bar{y})^2}$$

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

$$MSPE = \frac{1}{N} \sum_{i=1}^N \left(\frac{y_i - \hat{y}_i}{y_i} \right)^2$$

$$MAPE = \frac{1}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Mesurer la performance d'une classification

- Utiliser la matrice de confusion

		Predicted label	
		Positive (PP)	Negative (PN)
True label	Positive (P)	TP	FN
	Negative (N)	FP	TN

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Sensitivity, recall = \frac{TP}{TP + FN}$$

$$\longrightarrow F_1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

$$Specificity = \frac{TN}{TN + FP}$$

Mesurer la performance d'une classification

■ Accuracy

- Mesure pertinente si le dataset est assez équilibré et si l'intérêt de prédiction est le même pour les deux classes
- Intuitive mais peut demander à être exploiter avec d'autres

■ Precision

- Critère important si on souhaite minimiser les faux positifs.
- Ex : lors de la construction d'un classifieur de spams. On souhaite voir le moins de spam possible mais être sûr de ne pas manquer de courriers importants. Ici, on cherche à maximiser la précision

■ Sensitivity / Recall

- Critère important quand on souhaite minimiser le risque de manquer des cas positifs (ie. prédire des faux négatifs). Il s'agit généralement de cas où l'absence d'un cas positif a un coût beaucoup plus important que la classification erronée d'un élément comme positif
- Ex : dans le milieu médical on préférera ne pas manquer un cas que d'identifier un faux malade => avoir un test positif si la maladie est présente

Mesurer la performance d'une classification

- **F₁ score**

- Combine la précision et la sensibilité
- Fonctionne dans les cas où le dataset est déséquilibré

- **Specificity**

- Critère quand on souhaite vérifier que les observations de l'*autre* classe sont bien prédites
- N'est pas utilisée seule
- Ex : dans le milieu medical, cela revient à vérifier que tous les non-malades ont bien été prédits ainsi => avoir un test négatif chez les non-malades

Exemple

- Détection de cancer
 - Dataset de 100 patients répartis en 57 patients atteints de cancer et 43 patients sains

		Predicted label	
		Cancer	No Cancer
True label	Cancer	45	12
	No Cancer	18	25

Accuracy = 0,7

Precision = 0,714

Sensitivity = 0,789

Specificity = 0,581

$F_1 = 0,75$

Exemple

- Détection de paiements frauduleux sur le web
 - 10 paiements parmi les 1000 effectués correspondent à des fraudes
 - Déséquilibre important entre les classes

		Predicted label	
		Fraud	No Fraud
True label	Fraud	0	10
	No Fraud	0	990

Accuracy = 0.99

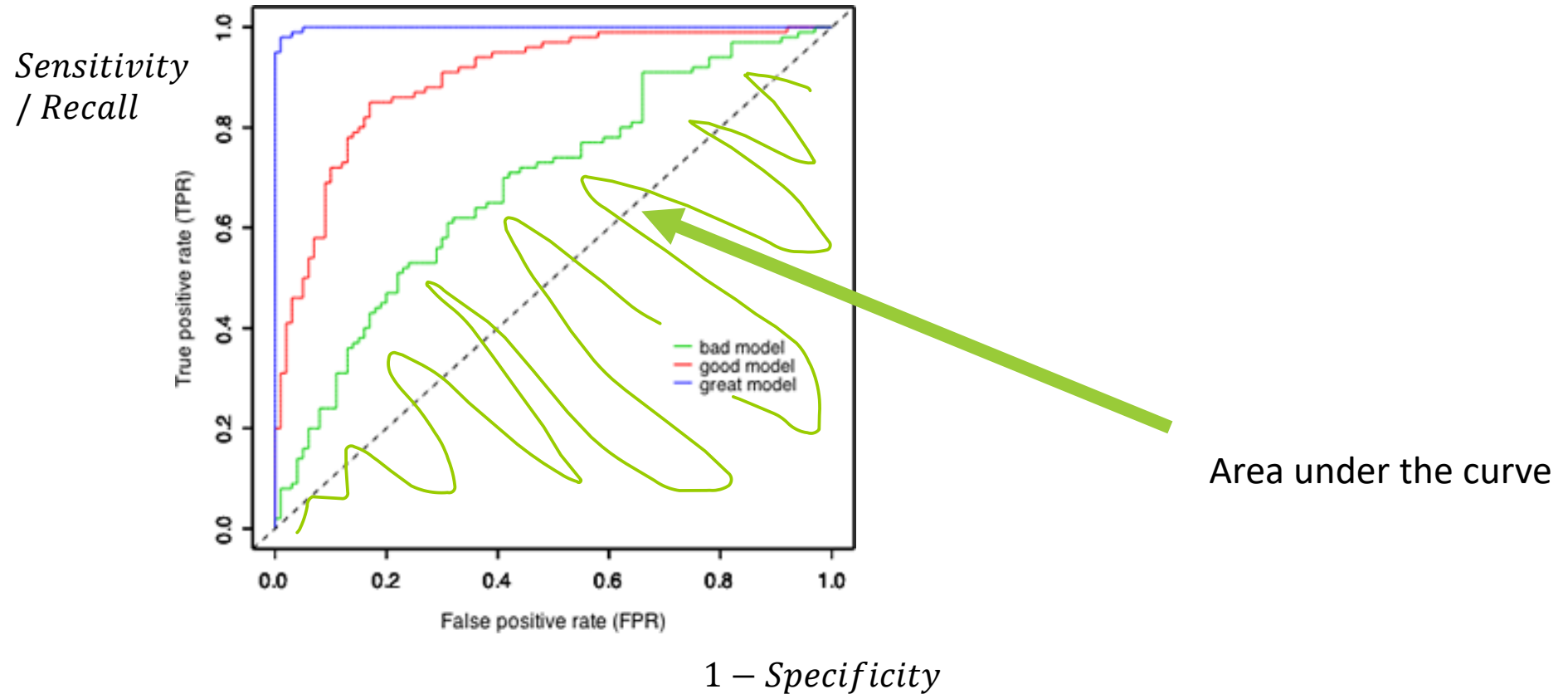
Precision = nan

Sensitivity = 0

Specificity = 1

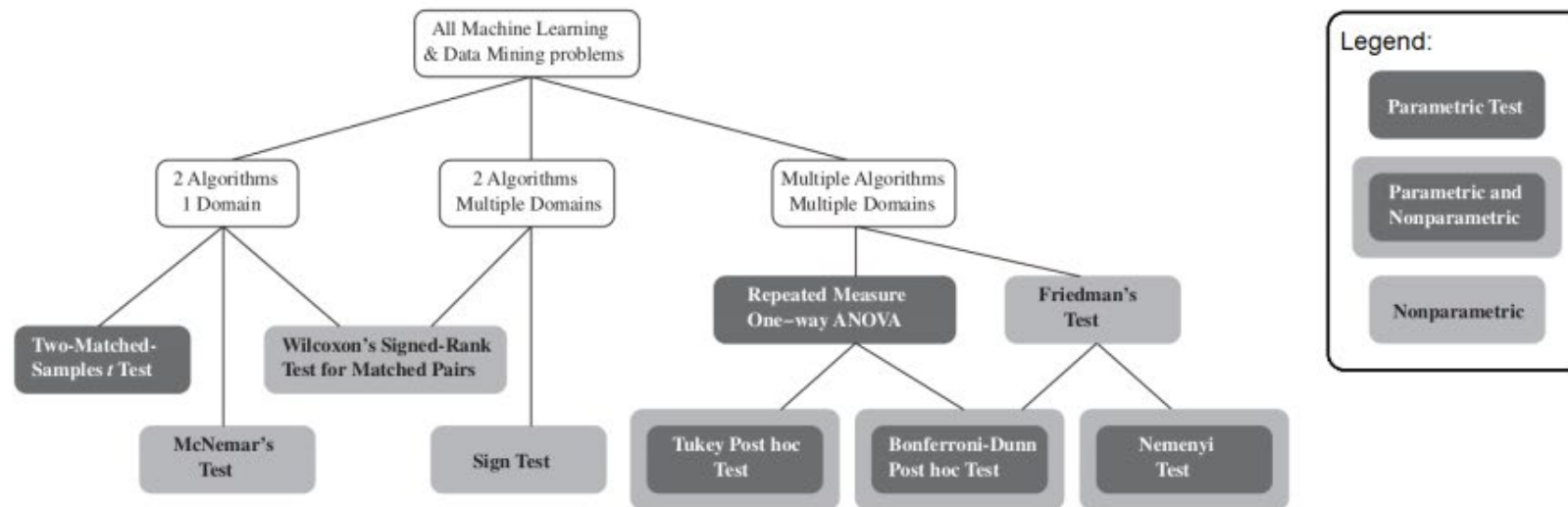
Mesurer la performance d'une classification

- Utiliser la matrice de confusion et la courbe ROC



Comparaison de méthodes

- Choisir la méthode ayant la meilleure performance...
- ... ou plutôt utiliser un test statistique



- Cas avec un compromis entre 2 ou plusieurs performances
 - Approche Pareto
 - Approche agrégation/pondération

CLASSIFICATION

Classification

A partir d'une variable observée X et d'une variable cible Y catégorielle

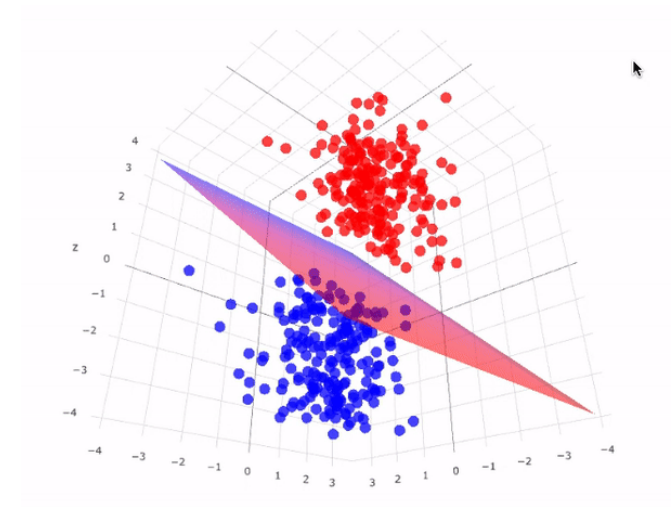
- Approches discriminantes
 - Modélisation de la probabilité conditionnelle $P(Y|X = x)$ + détermination d'un seuil de décision
 - A pour but de maximiser la qualité de la classification sur un jeu de test
 - Exemples :
 - Régression logistique, Perceptron, SVM...
 - Arbres de décision
- Approches génératives
 - Modélisation de la distribution jointe $P(X, Y)$
 - Exemples :
 - Classification naïves de Bayes
 - Analyse linéaires discriminantes
- Autres approches
 - K plus proches voisins (K-NN) basé sur une distance entre les observations
 - Méthodes ensemblistes basées sur un mélange de modèles
 - Bagging (ex : Random forest), Boosting (ex : AdaBoost)

Classifieur linéaire

$$y = f(\vec{w} \cdot \vec{x}) = f\left(\sum_j w_j x_j\right)$$

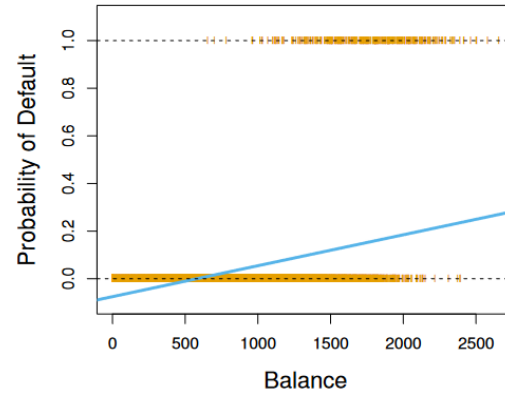
- $\vec{x}$ est un vecteur d'observation à valeurs réelles
- $\vec{w}$ est un vecteur des poids
- f est une fonction qui transforme le produit scalaire dans l'espace désiré

- Si classification binaire alors
 - Séparation d'un espace de grande dimension par un hyperplan
- Utile quand
 - Classes séparables linéairement
 - Dataset creux
 - Nombre de features très grand

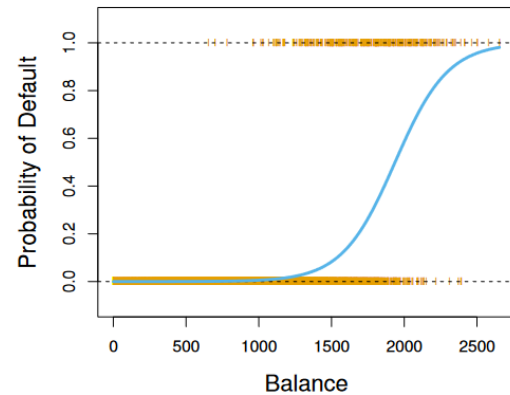


Régression logistique

- 1 variable, 2 classes



Régression linéaire



Régression logistique

$$p(X) = \Pr(Y = 1|X)$$

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$



$$\log \left(\frac{p(X)}{1 - p(X)} \right) = \beta_0 + \beta_1 X$$

Régression logistique

- Plusieurs variables, 2 classes

$$p(X) = \frac{e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}} \quad \longrightarrow \quad \log \left(\frac{p(X)}{1 - p(X)} \right) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$$

- Plusieurs variables, plusieurs classes

Avec la fonction Sigmoidé

$$\begin{aligned} \Pr(Y = k | X = x) &= \frac{e^{\beta_{k0} + \beta_{k1} x_1 + \dots + \beta_{kp} x_p}}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_{l1} x_1 + \dots + \beta_{lp} x_p}} \\ \Pr(Y = K | X = x) &= \frac{1}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_{l1} x_1 + \dots + \beta_{lp} x_p}} \end{aligned} \quad \longrightarrow \quad \log \left(\frac{\Pr(Y = k | X = x)}{\Pr(Y = K | X = x)} \right) = \beta_{k0} + \beta_{k1} x_1 + \dots + \beta_{kp} x_p$$

Avec la fonction Softmax (= fonction exponentielle normalisée)

$$\Pr(Y = k | X = x) = \frac{e^{\beta_{k0} + \beta_{k1} x_1 + \dots + \beta_{kp} x_p}}{\sum_{l=1}^K e^{\beta_{l0} + \beta_{l1} x_1 + \dots + \beta_{lp} x_p}} \quad \longrightarrow \quad \log \left(\frac{\Pr(Y = k | X = x)}{\Pr(Y = k' | X = x)} \right) = (\beta_{k0} - \beta_{k'0}) + (\beta_{k1} - \beta_{k'1}) x_1 + \dots + (\beta_{kp} - \beta_{k'p}) x_p.$$

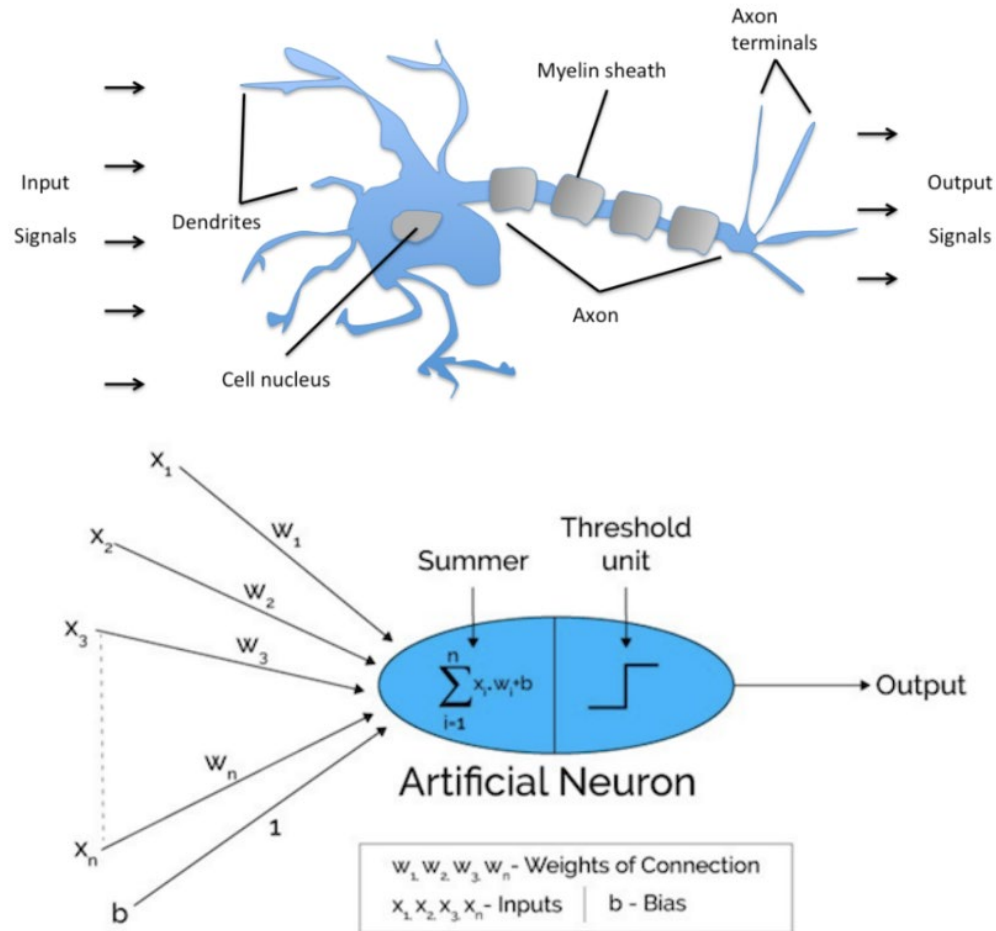
Approches génératives

- Hypothèse : l'existence d'une caractéristique pour une classe, est indépendante de l'existence d'autres caractéristiques

$$\Pr(Y = k | X = x) = \frac{\pi_k f_k(x)}{\sum_{l=1}^K \pi_l f_l(x)}$$

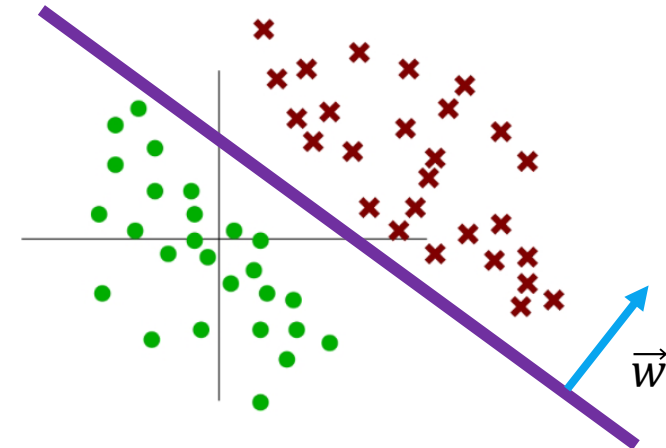
- π_k est la probabilité a priori d'appartenir à la classe k
 - Correspond à la proportion d'éléments qui appartiennent à la classe k
- f_k est la fonction de densité de X pour une observation qui appartient à la classe k
 - 3 approches :
 - Analyse linéaire discriminante
 - Analyse quadratique discriminante
 - Méthode naïves de Bayes

Perceptron

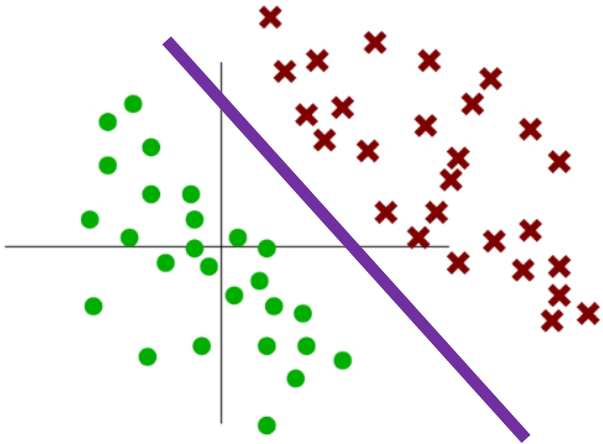
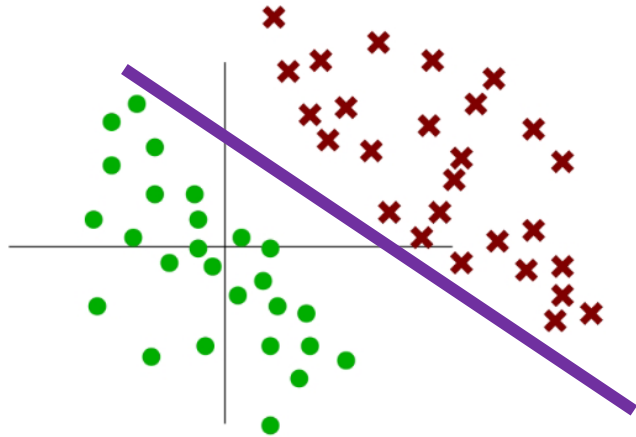
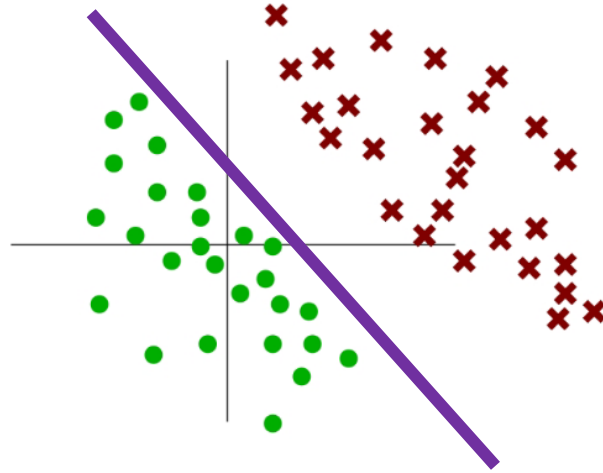
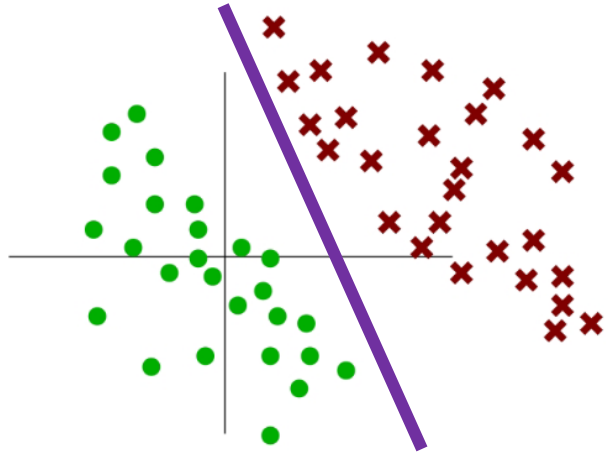


remi.eyraud @ univ aix-marseille

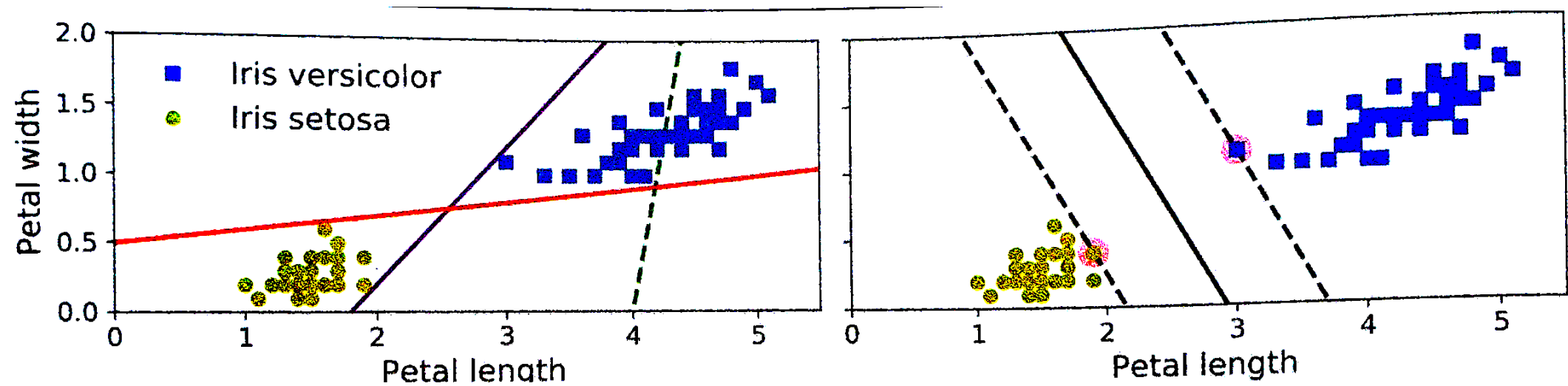
- Recherche un hyperplan pour séparer les classes ie. Trouver le vecteur $\vec{w}$ orthogonal à l'hyperplan
- Observations en dimension d
-> Espace vectoriel en dimension $d+1$
- Basé sur le produit scalaire
- Hypothèse : classes linéairement séparables...



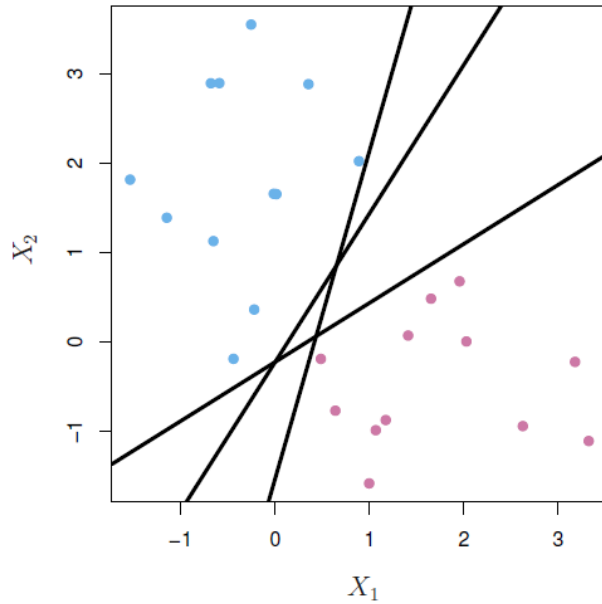
SVM : Support Vector Machine



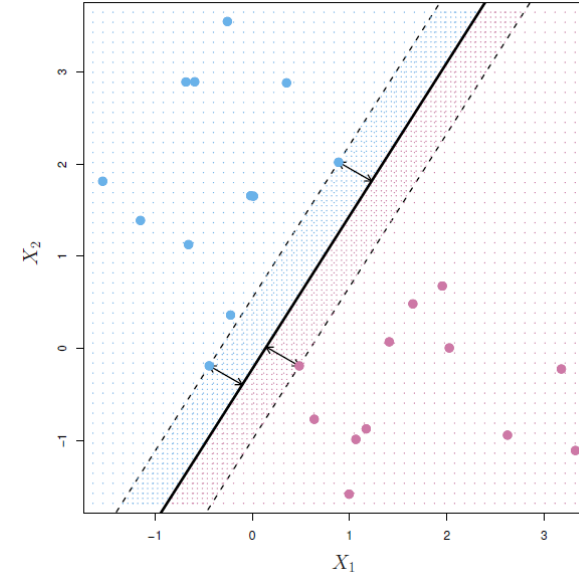
SVM sur la base *iris*



SVM : Support à Vaste Marge



→
Maximiser la marge M



$$\underset{\beta_0, \beta_1, \dots, \beta_p}{\text{maximize}} \quad M$$

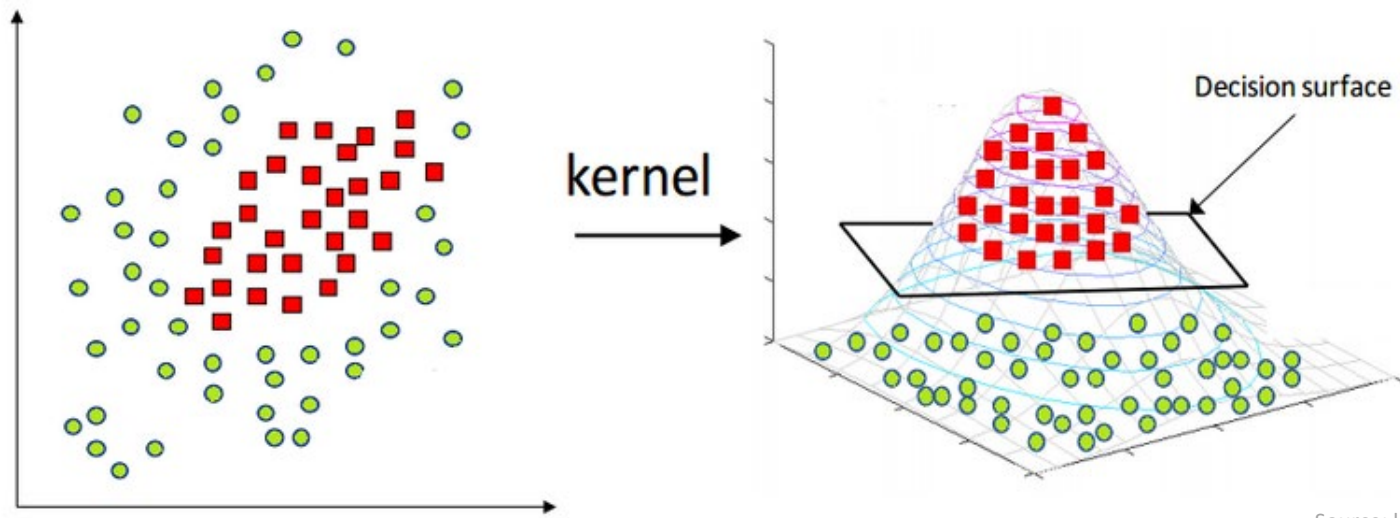
$$\text{subject to} \quad \sum_{j=1}^p \beta_j^2 = 1,$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq M$$

for all $i = 1, \dots, N$.

SVM

- Que se passe-t-il quand la séparation linéaire des classes n'est pas possible ?
- Softmargin = marges douces
 - Accepter une tolérance de mauvais classement selon l'hyper-parameter C
- Kernel tricks = astuce du noyau
 - Transformer le produit scalaire en une fonction noyau

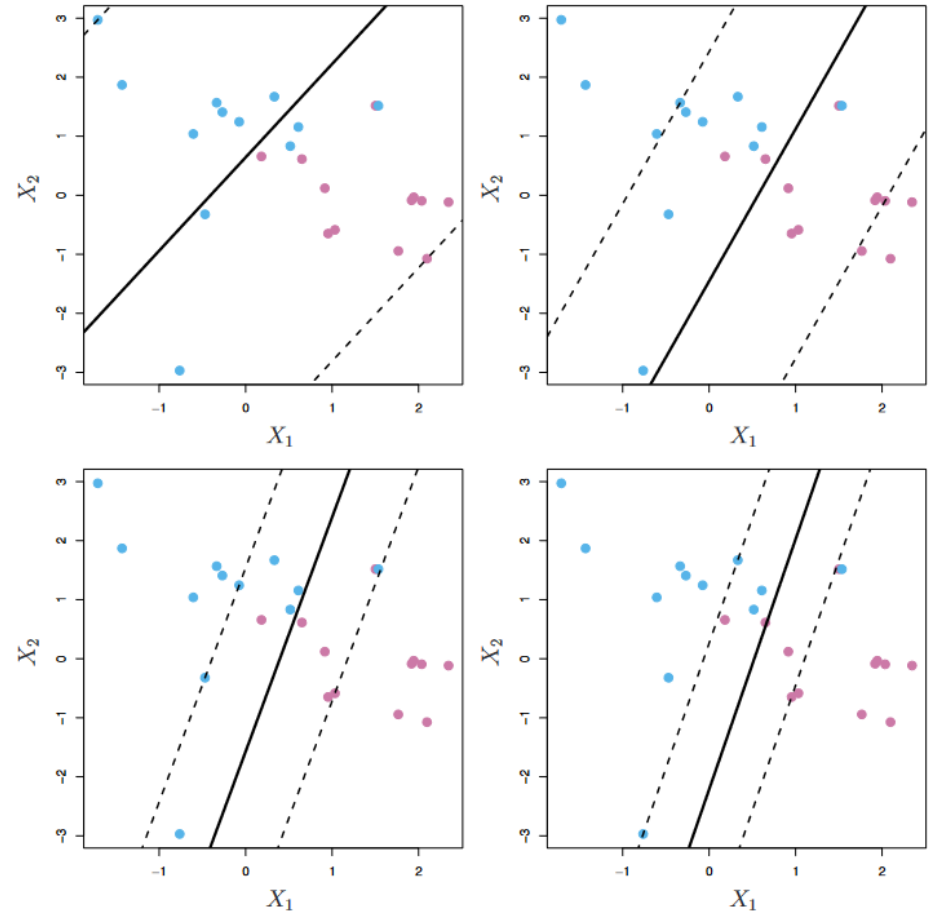


Source: kdnuggets

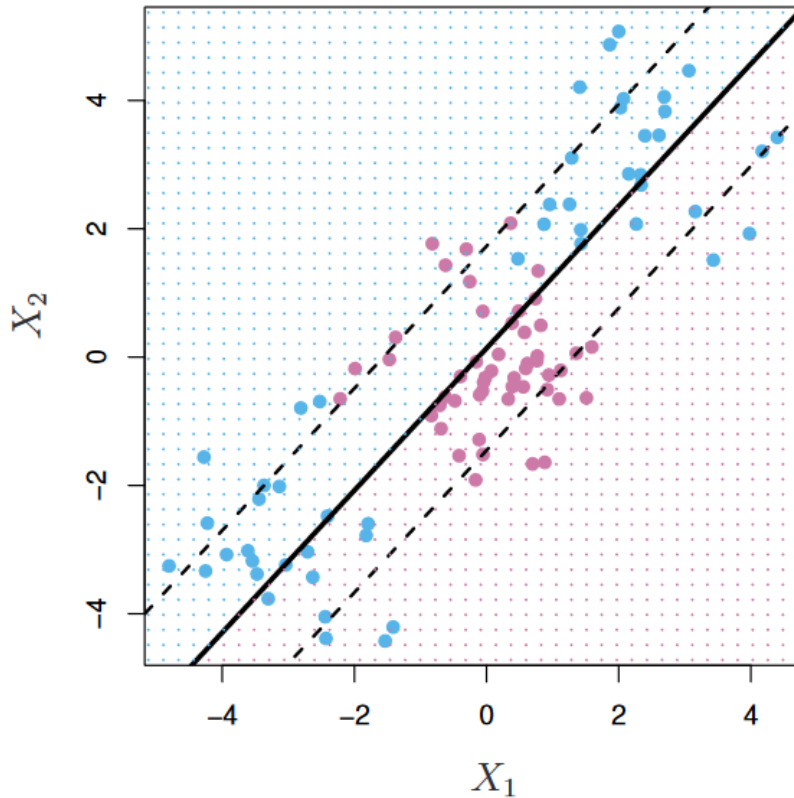
SVM avec marges douces

$$\begin{aligned} & \underset{\beta_0, \beta_1, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n}{\text{maximize}} && M \quad \text{subject to} \quad \sum_{j=1}^p \beta_j^2 = 1, \\ & y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M(1 - \epsilon_i), \\ & \epsilon_i \geq 0, \quad \sum_{i=1}^n \epsilon_i \leq C, \end{aligned}$$

hyperparamètre



SVM et astuce du noyau



$$\begin{aligned}
 & \underset{\beta_0, \beta_{11}, \beta_{12}, \dots, \beta_{p1}, \beta_{p2}, \epsilon_1, \dots, \epsilon_n, M}{\text{maximize}} && M \\
 & \text{subject to } y_i \left(\beta_0 + \sum_{j=1}^p \beta_{j1} x_{ij} + \sum_{j=1}^p \beta_{j2} x_{ij}^2 \right) \geq M(1 - \epsilon_i) \\
 & \sum_{i=1}^n \epsilon_i \leq C, \quad \epsilon_i \geq 0, \quad \sum_{j=1}^p \sum_{k=1}^2 \beta_{jk}^2 = 1.
 \end{aligned}$$

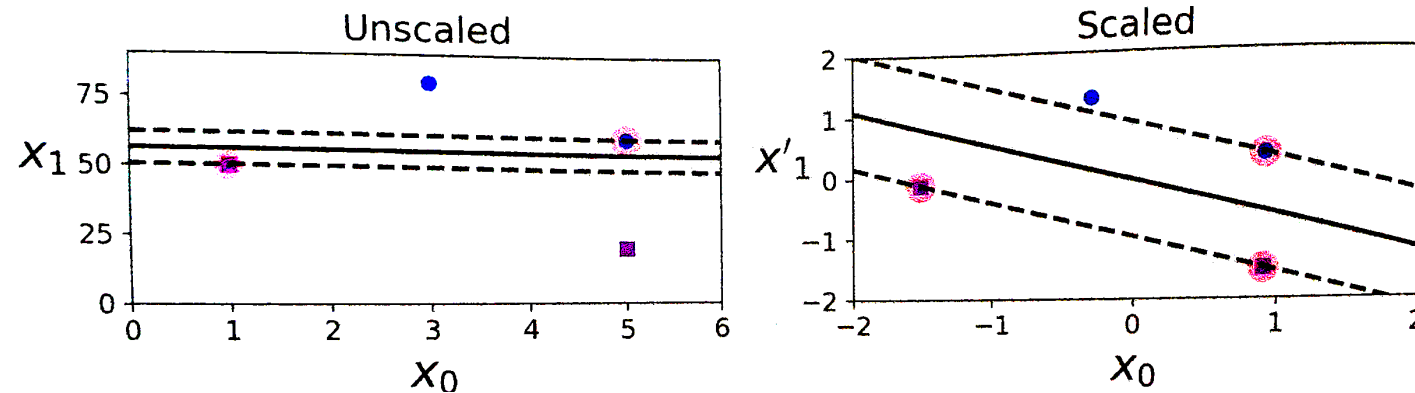
$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \hat{\alpha}_i \langle x, x_i \rangle$$



$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \hat{\alpha}_i K(x, x_i)$$

SVM

- Performants sur des jeux de données de dimension raisonnable
- Séparation linéaire tolérante au bruit (marges douces)
- Adaptés à l'astuce du noyau (SVM originaux = SVM linéaire = SVM à noyau)
- Quelques hyper-paramètres à régler
- Il existe des fonctions noyaux bien connues, mais (presque) pas de recette pour choisir la bonne...
- Ne pas oublier de normaliser les données

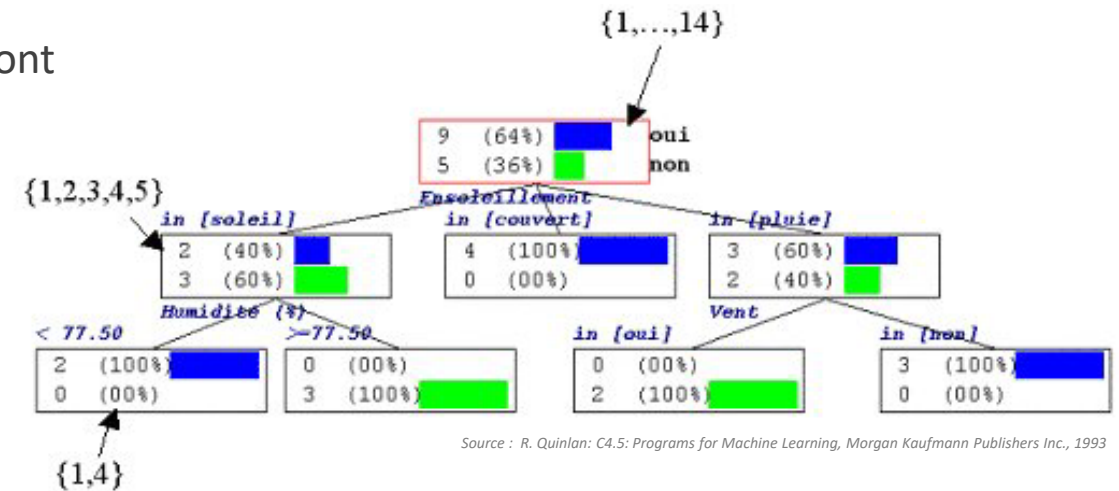


Arbres de décision

Arbre de règles portant sur des tests effectués sur les attributs des données

Structure générique de l'algorithme

- Noeud terminal ?
 - Lorsque (presque) tous les exemples de S en ce nœud sont dans la même classe
 - Lorsqu'il n'y a plus d'attributs à tester à ce niveau
- Quelle classe à un nœud terminal ?
 - Classe majoritaire
 - Classe la plus représentée, si égalité
- Sélection d'un test ? Choix de l'attribut qui fait le mieux progresser la discrimination des données de S : gain en information
 - Indice de Gini (CART)
 - Critère d'entropie (C4.5)



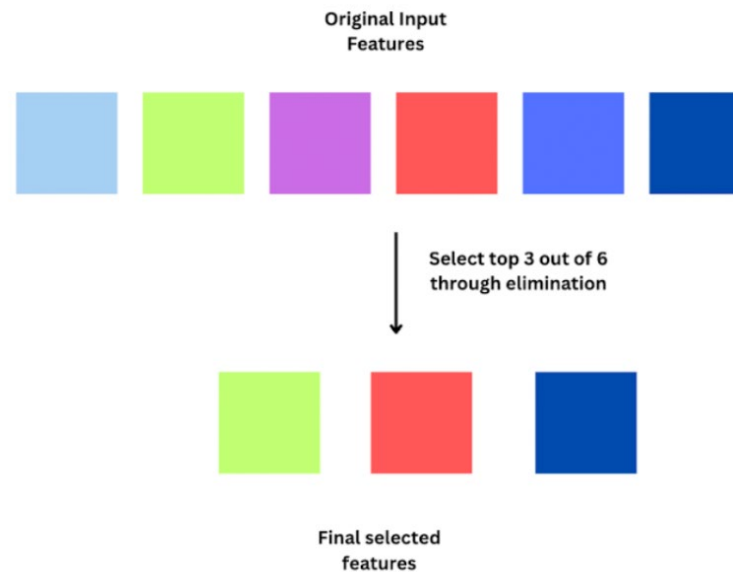
Source : R. Quinlan: C4.5: Programs for Machine Learning, Morgan Kaufmann Publishers Inc., 1993

Réduction des données en classification

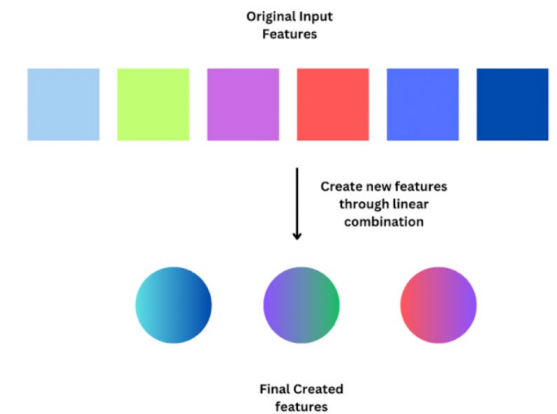
SÉLECTION D'ATTRIBUTS/VARIABLES

- Objectif
 - Sélectionner le sous-ensemble d'attributs le plus pertinent
- Problème
 - 2^d sous-ensembles possible
ex : pour 20 attributs, il y a plus d'un million de combinaisons possible
- Méthodes
 - Filter (par filtrage)
 - Wrapper (par emballage)
 - Embedded (intégré)

Feature Selection



Feature Extraction



Réduction des données en classification

MÉTHODES FILTER POUR LA SELECTION D'ATTRIBUTS



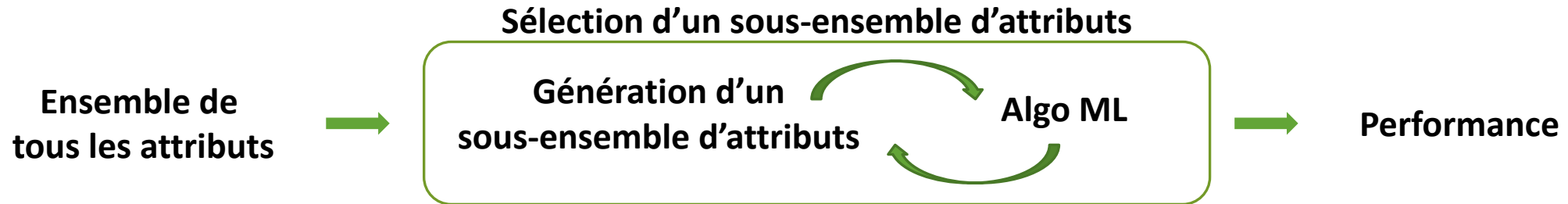
- La sélection des attributs est **indépendante** de la méthode de ML
- Les attributs sont choisis en fonction de leur corrélation avec l'attribut cible
- Plusieurs tests/méthodes sont à utiliser suivant le type des attributs

Attribut / Cible	Continu	Catégoriel
Continu	Corrélation de Pearson	LDA
Catégoriel	ANOVA	Test Khi-2

- Ces tests/méthodes ne permettent pas d'étudier la (non) multi-colinéarité des attributs qui doit être pourtant vérifiée

Réduction des données en classification

MÉTHODES WRAPPER POUR LA SELECTION D'ATTRIBUTS



- Un sous-ensemble d'attributs est utilisé pour entraîner un algo de ML, puis selon le résultat le sous-ensemble est plus ou moins modifié
- Ces méthodes sont très coûteuses en temps de calcul car
 - nécessitent l'entraînement de l'algo de ML
 - Sont souvent basées sur des heuristiques « glouton »

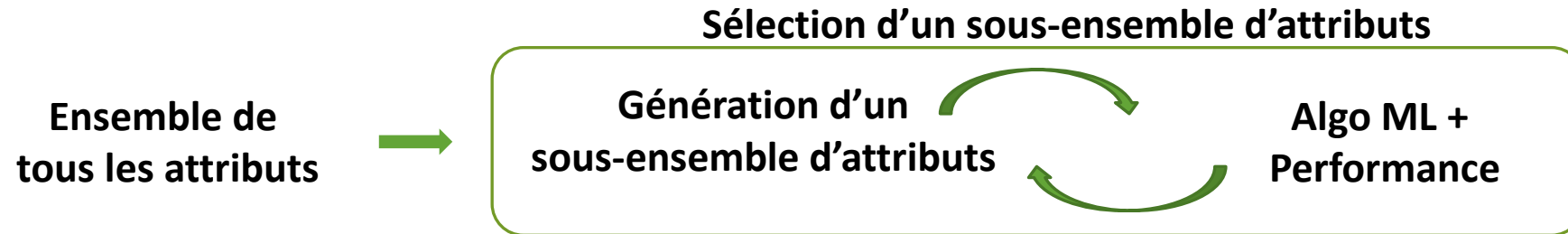
Réduction des données en classification

MÉTHODES WRAPPER POUR LA SELECTION D'ATTRIBUTS

- Méthode **Forward** (approche bottom-up) : débute avec un sous-ensemble d'attributs vide, puis y ajoute un à un l'attribut qui conduit à la meilleure performance, et s'arrête quand aucun attribut ne peut être ajouté sans dégrader la performance
- Méthode **Backward** (approche top-down) : débute avec un l'ensemble complet des attributs , puis supprime un à un l'attribut qui permet d'augmenter le plus la performance, et s'arrête quand on ne peut plus améliorer la performance.
- Méthodes **récurives** : des sous-ensembles sont successivement générés et testés pour former le meilleur sous-ensemble d'attributs qui permet d'avoir la meilleure performance. Ici des méthodes d'optimisation exactes peuvent être utilisées, comme le branch-and-bound mais qui nécessite une structure monotone, et des méthodes d'optimisation heuristiques telles que les métaheuristiques

Réduction des données en classification

MÉTHODES EMBEDDED POUR LA SELECTION D'ATTRIBUTS



- Combinaison des méthodes *filter* et *wrapper*
- Ces méthodes intègrent leurs propres méthodes de construction du sous-ensemble d'attributs
- Méthodes les plus connues pour des données continues :
 - LASSO : régularisation L1
 - RIDGE : régularisation L2

Réduction des données

- Principales différences entre les méthodes de type *filter* et *wrapper*
 - Les *filter* mesurent l'importance des attributs en étudiant la corrélation avec l'attribut cible alors que les *wrapper* mesurent l'intérêt d'utiliser un sous-ensemble d'attributs en analysant la performance d'un modèle entraîné
 - Les *filter* sont plus rapides que les *wrapper* mais le sous-ensemble d'attributs sélectionnés n'est pas toujours le plus pertinent contrairement à celui obtenu par les *wrapper*
 - Les *filter* utilisent des méthodes basées sur les statistiques alors que les *wrapper* reposent sur l'utilisation de la validation croisée.
 - Les algo de ML utilisés sur des sous-ensembles d'attributs sélectionnés par *wrapper* sont souvent moins sujets aux surentraînement contrairement à ceux sélectionnés par *filter*

Méthodes ensemblistes

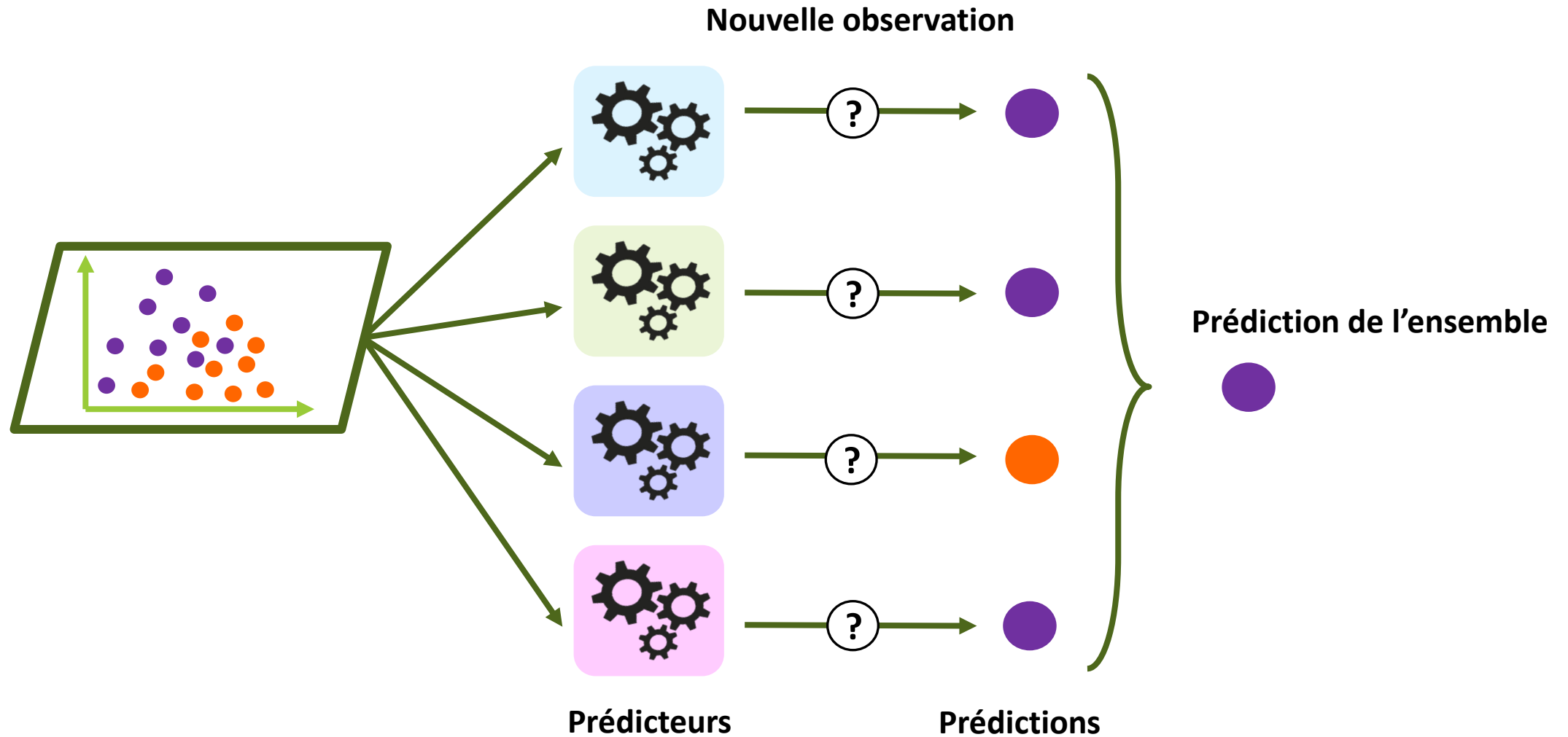
La sagesse de la foule...

- Exploiter la diversité de plusieurs modèles
 - Apprendre plusieurs modèles sur la base du même échantillon ou des variantes, et les combiner pour obtenir un modèle unique final
 - Combinaison (vote pondéré ou moyenne) de prédicteurs *faibles* pour obtenir un prédicteur *fort*
= combiner différents modèles pour augmenter les performances globales
 - Exploration originale de l'espace d'entrée
- No-free-lunch theorem
 - Il n'existe pas d'algorithme d'apprentissage qui fasse mieux que les autres dans tous les cas
- En pratique, on teste...
 - différents algorithmes, hyper-paramètres
 - différents sous-échantillons et espaces de description

Méthodes ensemblistes

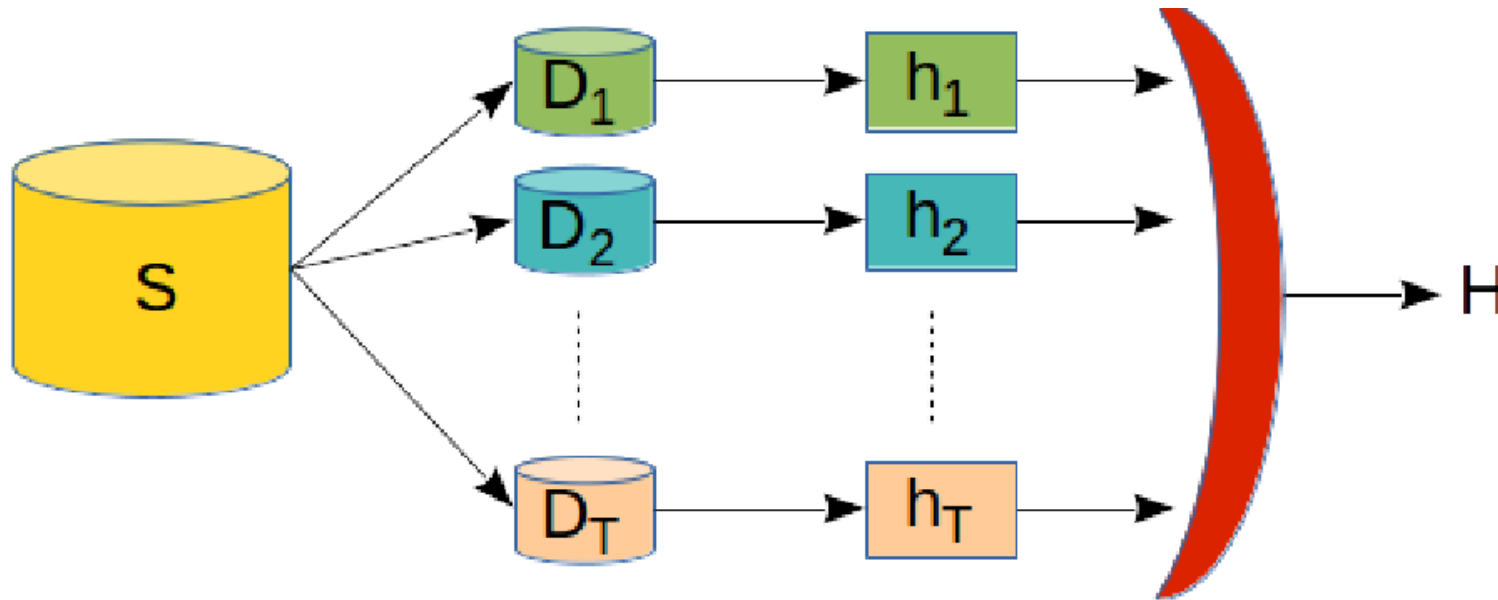
- Système de votes
- Méthodes de Bagging / Pasting
- Méthodes de Boosting
 - ADABOOST
 - Gradient Boosting
- Méthodes de Staking

Système de votes



Bagging

- Combiner des modèles et décorréler les erreurs : chaque modèle est appris sur un sous-échantillon de S
 - Extraire aléatoirement T sous-échantillons
 - Sur chaque échantillon S_t , apprendre une hypothèse h_t pour le prédicteur défini
 - Combiner les hypothèses par vote majoritaire (somme initialement, ou autre dans variantes)



Bagging

- Raisons d'un succès

- Chaque sous-hypothèse est instable (biais fort mais variance faible), mais leur combinaison est stable (erreurs non-correlées)
- Préférable avec hypothèse de base de type arbre de décision, réseau de neurones, etc.
- A éviter avec modèles de base stable

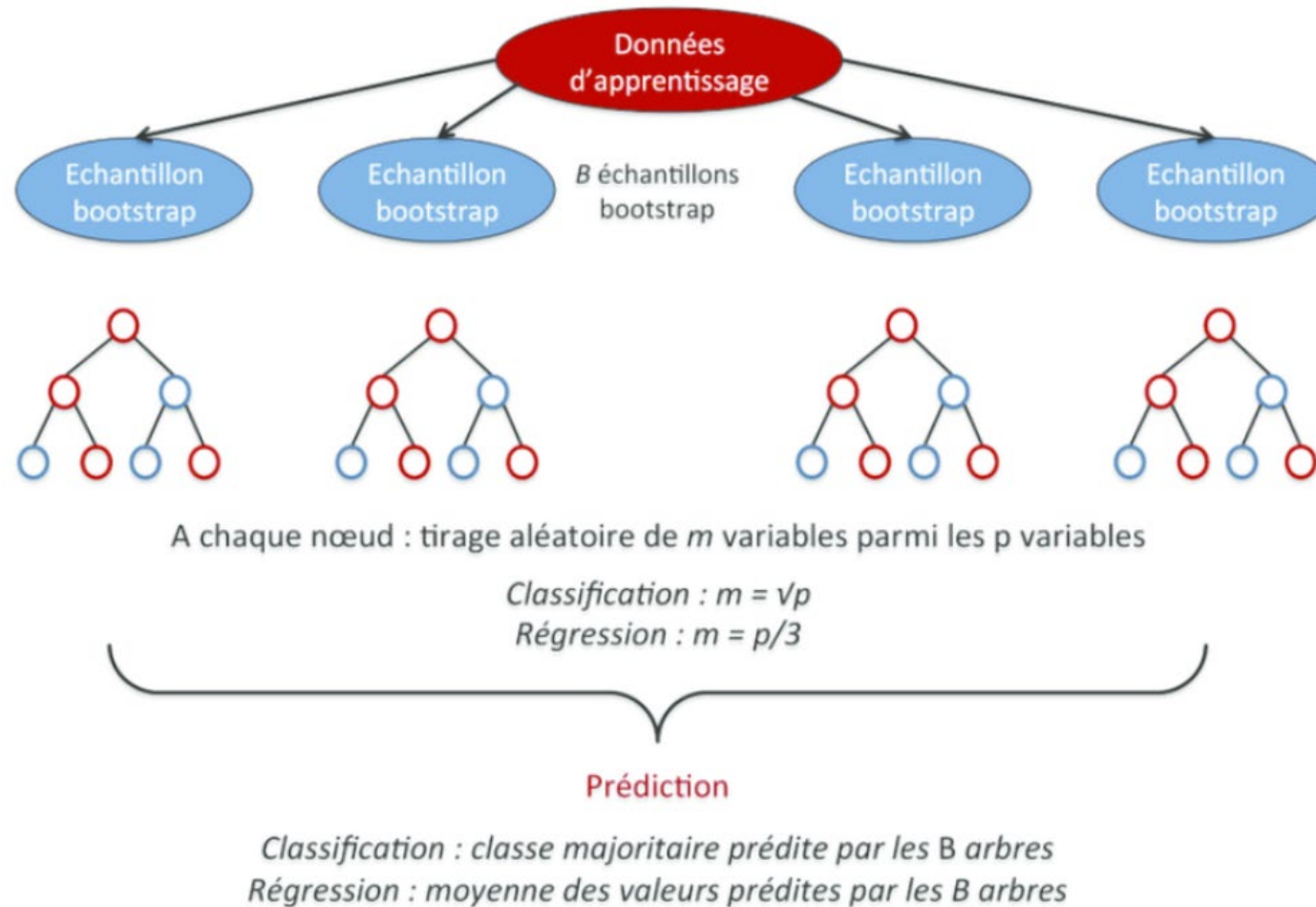
- En pratique

- Le bootstrap est effectuée avec remise
- Sous- échantillon constitué d'environ 60% des exemples de S
- Variantes :
 - Pasting (tirage sans remise)
 - Random Subspaces (tirage de sous-ensemble d'attributs)
 - Random Patches = Bagging+RS

Forêts aléatoires – Random forests

- Principe : décorréler les arbres de l'ensemble pour décorréler leurs erreurs
 - Chaque arbre est appris sur un sous-ensemble de l'échantillon initial S (en pratique, bootstrap 66% avec remise)
 - Lors de l'apprentissage de chaque arbre, chaque test est sélectionné sur la base de k attributs tirés aléatoirement parmi les d de l'espace de description, souvent $k = \sqrt{d}$
 - La prédiction finale est issue du vote des arbres
- En pratique
 - Fonctionne sur CART : mêmes hyper-paramètres + nombre T d'arbres
 - Un estimateur consistant sous certaines contraintes fortes
 - Très populaire, malgré une convergence moins rapide que beaucoup d'autres méthodes
 - Interprétabilité limitée

Forêts aléatoires – Random forests



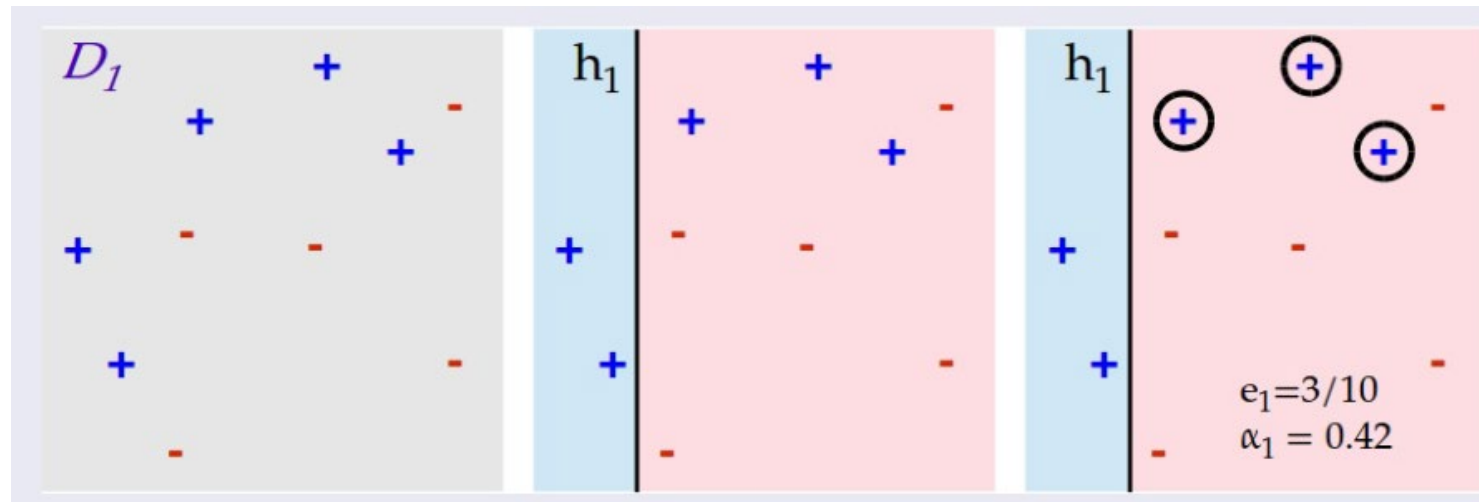
Boosting

- Entraîner des prédicteurs séquentiellement
- Essayer de corriger le prédicteur précédent

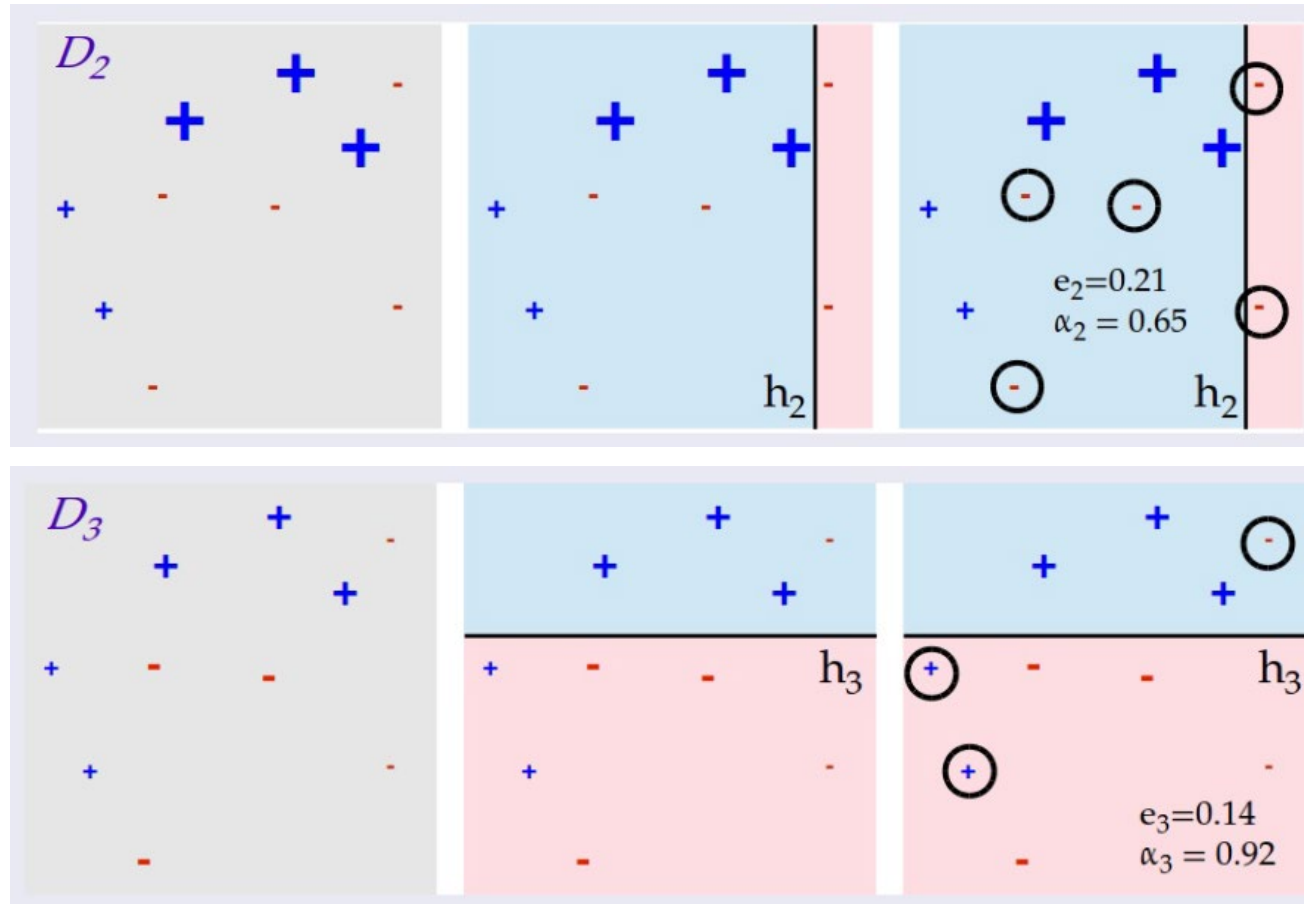
- Méthodes
 - ADA Boost
 - Gradient Boosting

Boosting et AdaBoost

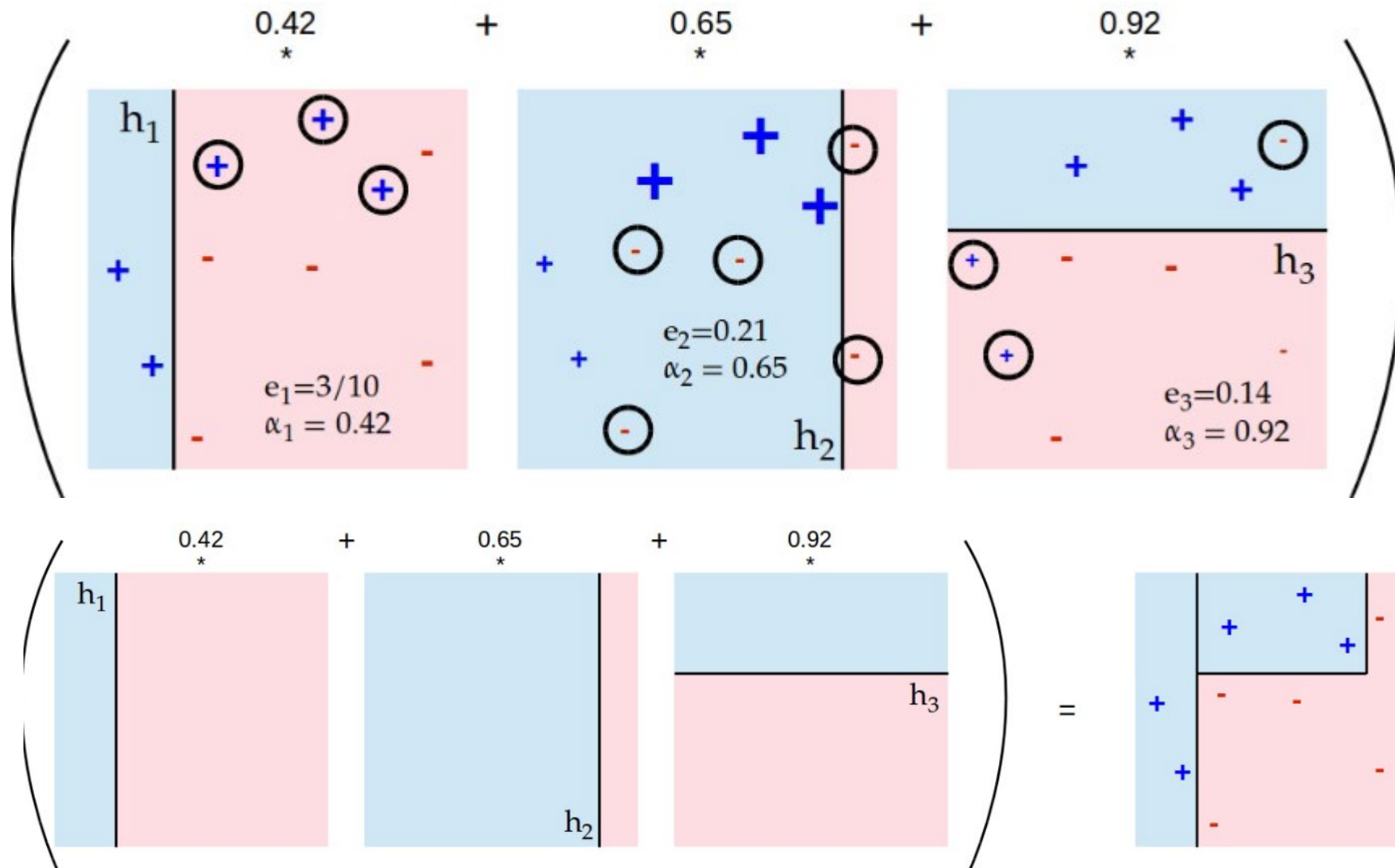
- Apprendre une règle de classification forte à partir d'un vote pondéré de règles faibles globalement (à peine mieux que le hasard), mais attendues expertes localement
- Algorithme itératif avec changement progressif de la distribution de l'échantillon d'apprentissage : inciter l'itération suivante à se focaliser sur les exemples difficiles



Boosting et AdaBoost



Boosting et AdaBoost



Gradient Boosting

- Même principe que ADABOOST
 - Chaque prédicteur est entraîné en corrigeant les erreurs du précédent
- Mais diffère sur la méthode de correction
 - Essaie d'améliorer l'erreur résiduelle observée avec le prédicteurs précédents
- Méthode de boosting très puissante et très performante
- Implémentation Python pour les arbres de décisions : XGBoost

Boosting

- Avantages

- bonnes propriétés de généralisation, rarement de sur-apprentissage, pas d'hyper-paramètre trop compliqué à ajuster, calculable, facile à implémenter, jolies garanties théoriques

- Inconvénients

- mauvaise tolérance au bruit, nécessite pas mal de données, nécessite des apprenants faibles un peu instables

A retenir

- Prédiction finale issue d'une combinaison linéaire de prédictions (locales)
- Méthode itérative, gloutonne
- Chaque classifieur dépend des précédents (pas d'indépendance comme dans bagging)
- Bonnes garanties de généralisation

Stacking

- Stacking = stacked generalization
- Principe : utiliser un modèle entraîné (méta-modèle) pour agréger les réponses des prédicteurs *faibles*
- Plusieurs étapes d'apprentissage sont nécessaires et donc demandent bcp de ressources

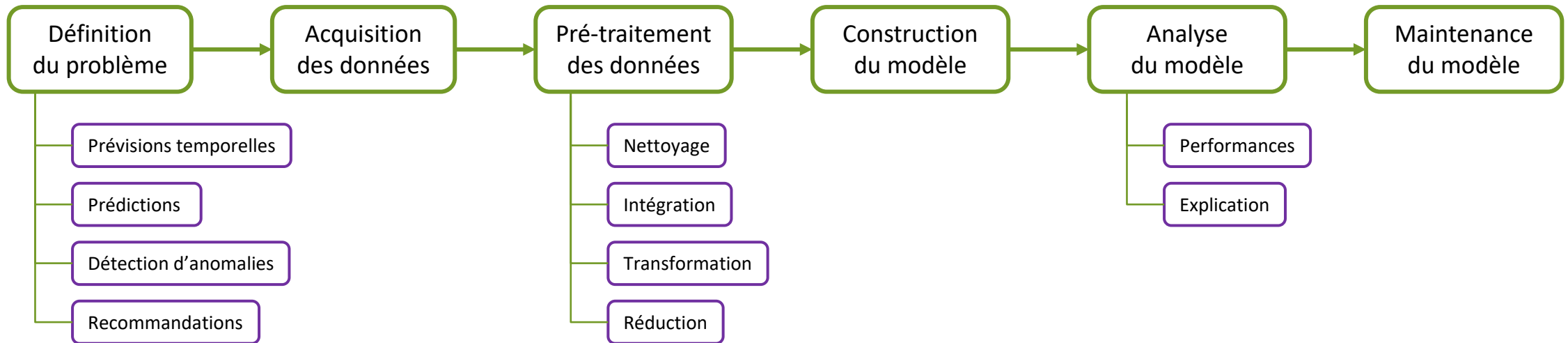
EXPLICABILITÉ -- INTERPRÉTABILITÉ

Interprétabilité et Explicabilité [Gilpin et al., 2018]

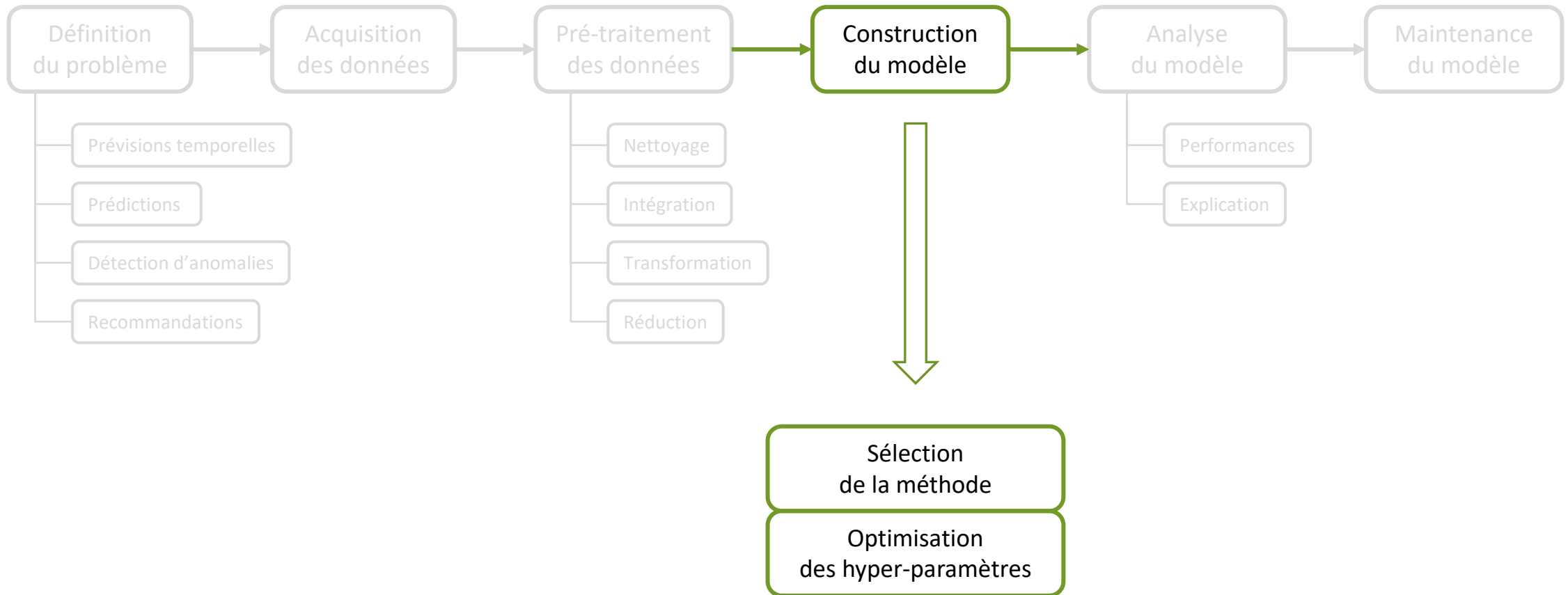
- Besoin de transparence et de confiance dans les algos de ML
- **Interprétabilité** consiste à fournir une information représentant à la fois le raisonnement de l'algorithme de Machine Learning et la représentation interne des données dans un format interprétable par un expert en ML ou des données. Le résultat fourni est fortement lié aux données utilisées et nécessite une connaissance de celles-ci mais aussi du modèle.
- **Explicabilité** consiste à fournir une information dans un format sémantique complet se suffisant à lui-même et accessible à un utilisateur, qu'il soit néophyte ou technophile, et quelle que soit son expertise en matière de ML.
- « Interprétabilité = Comment ? » vs « Explicabilité = Pourquoi ? »
- L'interprétabilité est la première étape à réaliser afin de faire de l'explicabilité.
- Un modèle explicable est donc interprétable
mais l'inverse ne l'est pas automatiquement !

HYPER-PARAMÉTRAGE ET AUTO-ML

Pipeline d'un projet d'AA



Construction du modèle



Sélection de la méthode

- $\mathcal{D}$ dataset split in two parts $\mathcal{D}_{train}$ and $\mathcal{D}_{valid}$
- $\mathcal{A} = \{A^{(1)}, \dots, A^{(n)}\}$ a set of ML algorithms where n is not necessarily finite
- $f_i = A^{(i)}(\mathcal{D}_{train})$ where $i \in \{1, \dots, n\}$, denote the trained model
- $\mathcal{L}(f_i, \mathcal{D}_{valid})$ a loss function evaluated on $\mathcal{D}_{valid}$ for $A^{(i)}$

Model selection problem consist of selecting the algorithm A^* which minimize the loss function $\mathcal{L}$:

$$A^* \in \arg \min_{A^{(i)}} \mathcal{L}(f_i, \mathcal{D}_{valid})$$

Optimisation des hyper-paramètres (HPO)

- Choisir l'ensemble optimal des hyperparamètres d'une méthode d'apprentissage
- Hyperparamètre = paramètre dont la valeur est utilisée pour contrôler l'apprentissage du modèle
 - $\mathcal{D}$ dataset split in two parts $\mathcal{D}_{train}$ and $\mathcal{D}_{valid}$
 - λ the hyperparameters of a given ML algorithm A with a domain Λ
 - $\mathcal{L}(A_{\lambda}(\mathcal{D}_{train}), \mathcal{D}_{valid})$ a loss function for A , which is trained on $\mathcal{D}_{train}$ and evaluated on $\mathcal{D}_{valid}$

The HPO problem is to find λ^* minimizing the loss function $\mathcal{L}$:

$$\lambda^* \in \arg \min_{\lambda \in \Lambda} \mathcal{L}(A_{\lambda}(\mathcal{D}_{train}), \mathcal{D}_{valid})$$

CASH : Combiner sélection et optimisation

C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown, Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms," in Proc. of KDD-2013, 2013, pp. 847-855

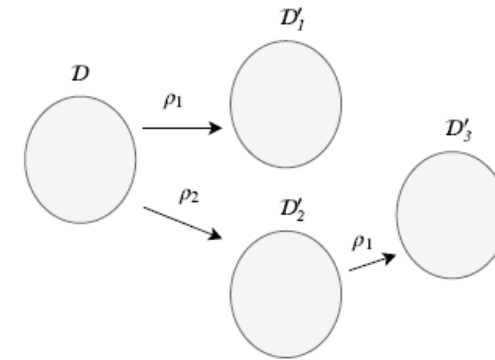
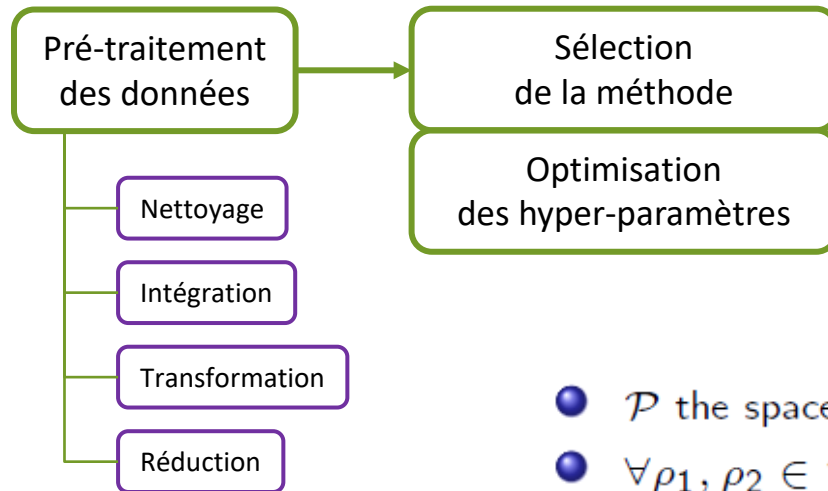
- $\mathcal{D}$ dataset split in two parts $\mathcal{D}_{train}$ and $\mathcal{D}_{valid}$
- Let $\mathcal{A} = \{A^{(1)}, \dots, A^{(n)}\}$ a set of ML algorithms where n is not necessarily finite
- $\Lambda^{(1)}, \dots, \Lambda^{(n)}$ hyperparameter spaces respective to $A^{(1)}, \dots, A^{(n)}$
- $\mathcal{L}(A_{\lambda}^{(i)}(\mathcal{D}_{train}), \mathcal{D}_{valid})$ a loss function for $A^{(i)}$, using $\lambda \in \Lambda^{(i)}$, trained on $\mathcal{D}_{train}$ and evaluated on $\mathcal{D}_{valid}$
 $i \in \{1, \dots, n\}$

The CASH problem is to find a combination A^* associated with λ^* which minimize the loss $\mathcal{L}$:

$$A_{\lambda^*}^* \in \arg \min_{A^{(i)} \in \mathcal{A}, \lambda \in \Lambda^{(i)}} \mathcal{L}(A_{\lambda}^{(i)}(\mathcal{D}_{train}), \mathcal{D}_{valid})$$

CASHAP : CASH + Pré-traitement

L. Parmentier, O. Nicol, L. Jourdan, and M. Kessaci, TPOT-SH: A Faster Optimization Algorithm to Solve the AutoML Problem on Large Datasets," in 2019 IEEE 31st International Conference on Tools with Artificial Intelligence (ICTAI), 2019, pp. 471-478.



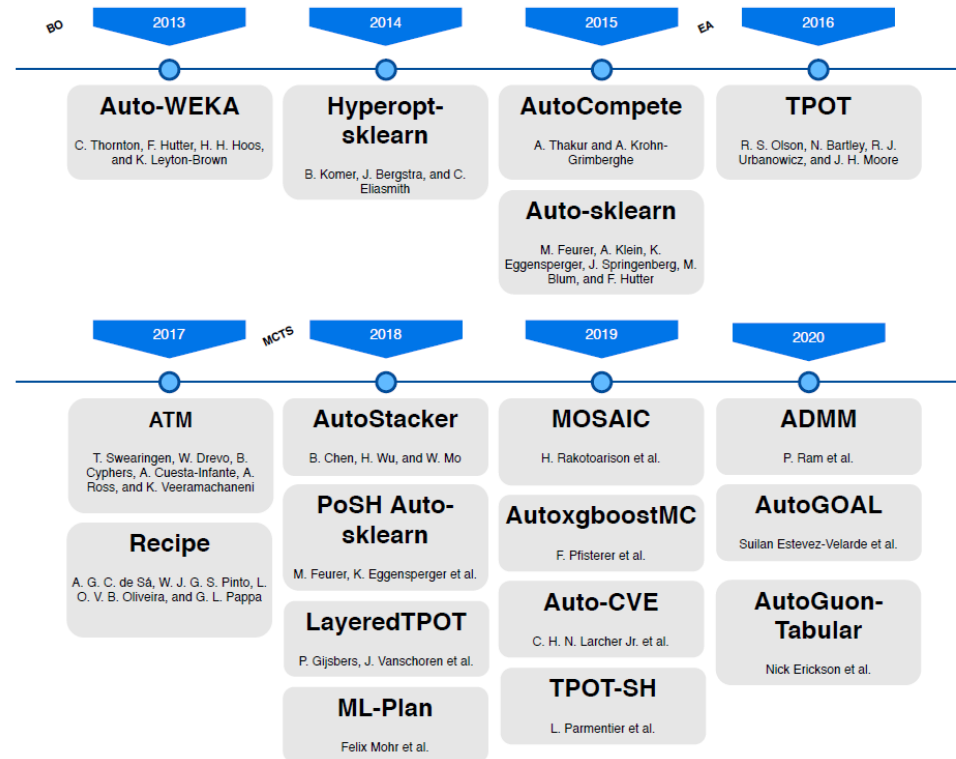
- $\mathcal{P}$ the space containing all different preprocessing methods
- $\forall \rho_1, \rho_2 \in \mathcal{P}, \rho_1 \circ \rho_2 \in \mathcal{P}$
- $\mathcal{A}, \Lambda$ and $\mathcal{L}$ are defined in Equations (1, 3)

The CASHAP problem is to find a combination ρ^* , A^* and λ^* minimizing the loss $\mathcal{L}$:

$$(\rho^*, A_{\lambda^*}^*) \in \arg \min_{\rho \in \mathcal{P}, A^{(i)} \in \mathcal{A}, \lambda \in \Lambda^{(i)}} \mathcal{L}(A_{\lambda}^{(i)}(\rho(\mathcal{D}_{train})), \rho(\mathcal{D}_{valid}))$$

Auto-ML

- Optimiser de manière automatique les composants du pipeline
- Actuellement, les méthodes se concentrent sur le problème CASH



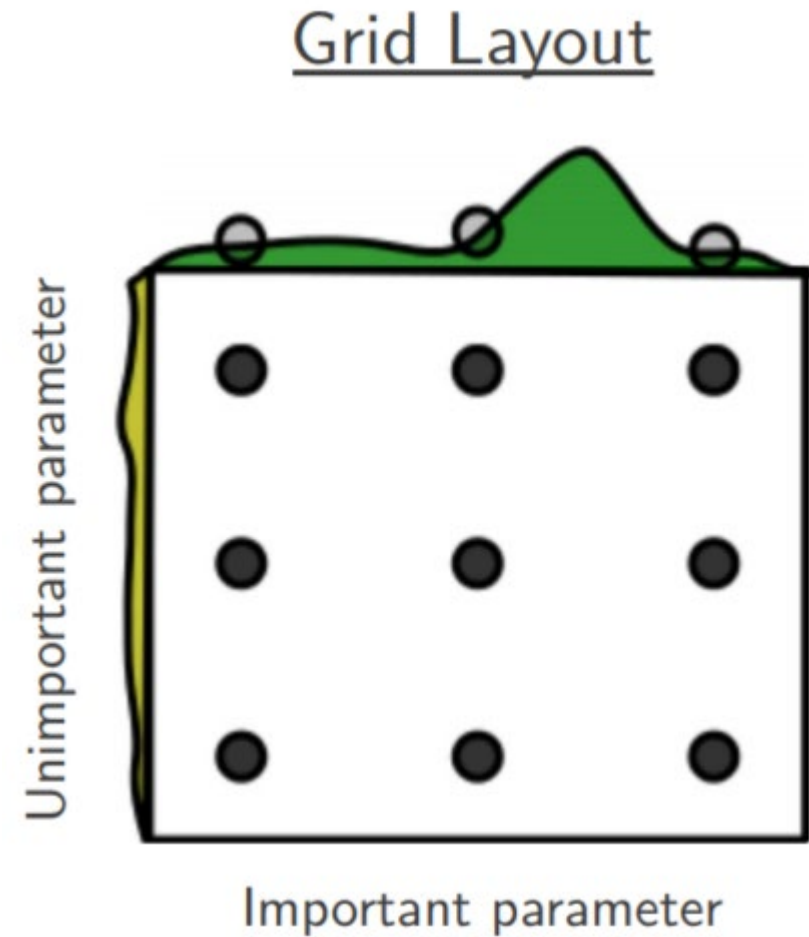
Optimisation des Hyper-paramètres

Approches :

- Grid search
- Random search
- Optimisation bayésienne
- Optimisation basée sur le gradient
- Optimisation par métaheuristiques

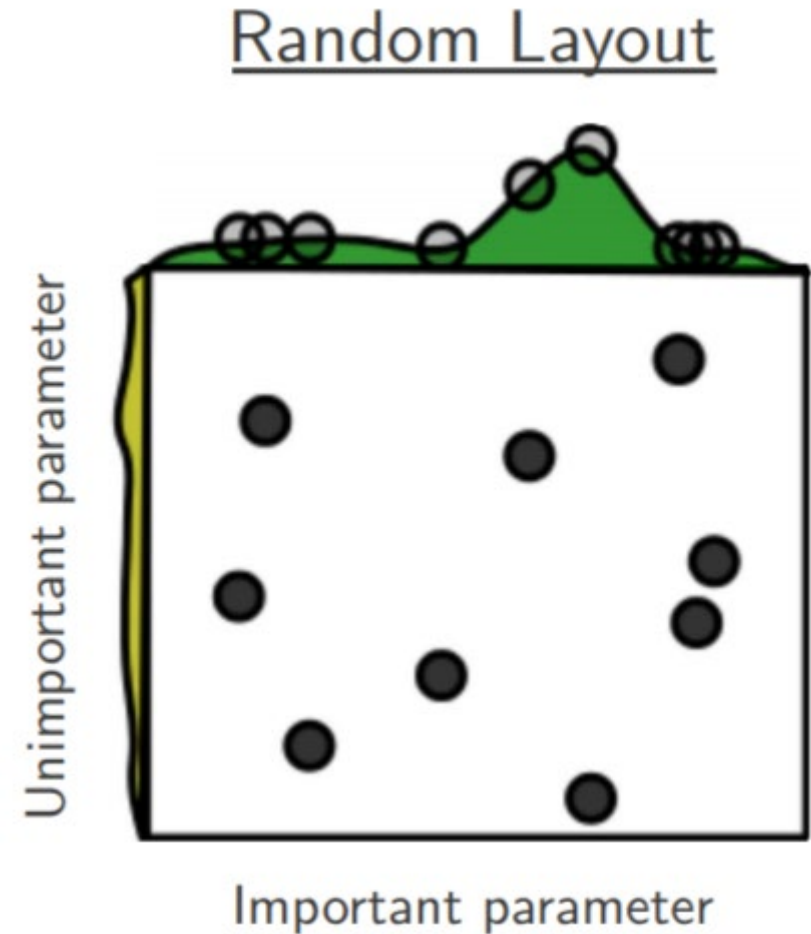
Grid Search

- Recherche exhaustive = test de toutes les combinaisons possibles
 - Discrétisation des valeurs réelles
 - Choix arbitraire des valeurs testées
- Utilisation de la validation-croisée et d'une mesure de performance
- Indépendance des itérations



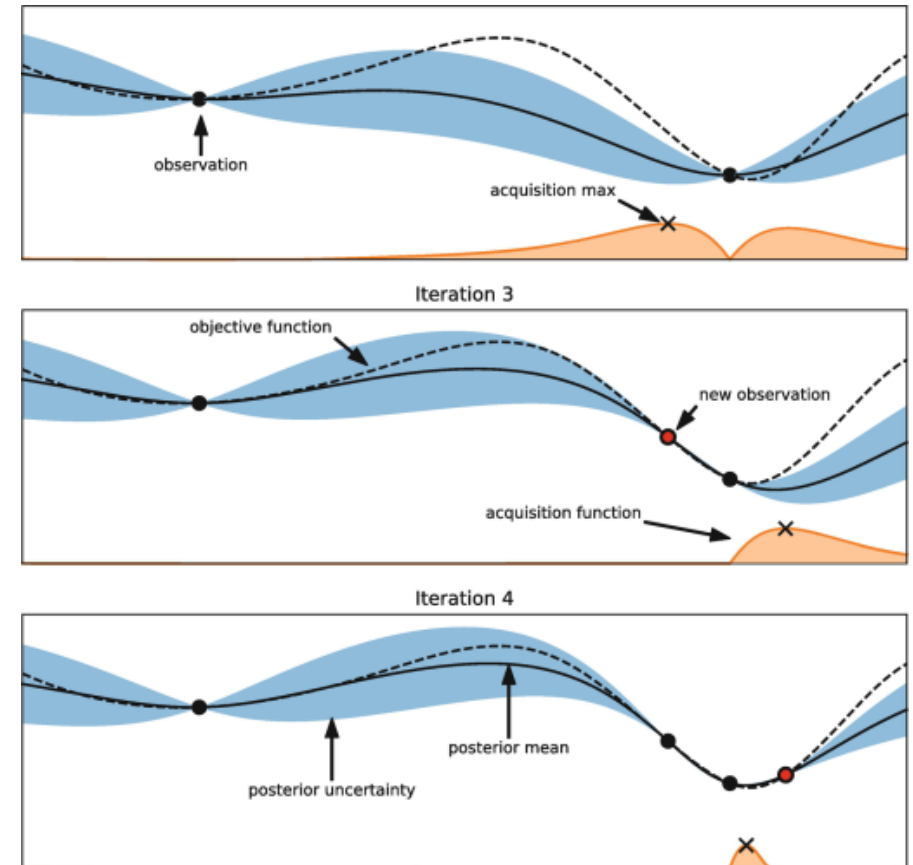
Random Search

- Sélection aléatoire des combinaisons à tester
 - Choix des combinaisons basé sur une loi de probabilité
- Généralisation à tout type d'hyper-paramètres
- Meilleur que Grid Search quand seul un petit nombre d'hyper-paramètres affecte la performance finale du modèle
- Utilisation de la validation-croisée et d'une mesure de performance
- Nécessite de fixer un budget



Optimisation Bayésienne

- Méthode d'optimisation pour des fonctions bruitées et boîte noire
- Construit un modèle probabiliste d'une fonction allant de l'espace des hyper-paramètres aux valeurs de performances du modèle évalué sur un ensemble de données de validation
- A chaque itération, BO essaie de s'approcher de l'ensemble de hyper-paramètres qui optimise la performance
- Bonne balance exploration/exploitation
- Pour un faible budget d'évaluation
 - BO $\gg$ GS, RS
- Inconvénient majeur : pas de parallélisation possible car méthode séquentielle



Source : F. Hutter

Optimisation basée sur le gradient

- Uniquement pour des hyper-paramètres numériques
 - Relaxation continue des hyperparamètres numériques discrets
- Utilisation de la dérivation automatique pour exécuter de manière itérative l'algorithme du gradient
- Facilement scalable : plusieurs millions de valeurs de hyper-paramètres

Optimisation par métaheuristiques

Principalement basée sur les algorithmes évolutionnaires

- Etape 1 : créer une population (aléatoire) de solutions (combinaisons de valeurs de HH)
- Etape 2 : Evaluer les solutions et obtenir une performance (par validation croisée)
- Etape 3 : Trier les solutions par performance
- Etape 4 : Remplacer les moins bonnes solutions avec d'autres solutions issues de croisement/mutation des meilleures solutions
- Continuer tant que les performances ne sont pas satisfaisantes

Successive Halving

- Objectif :

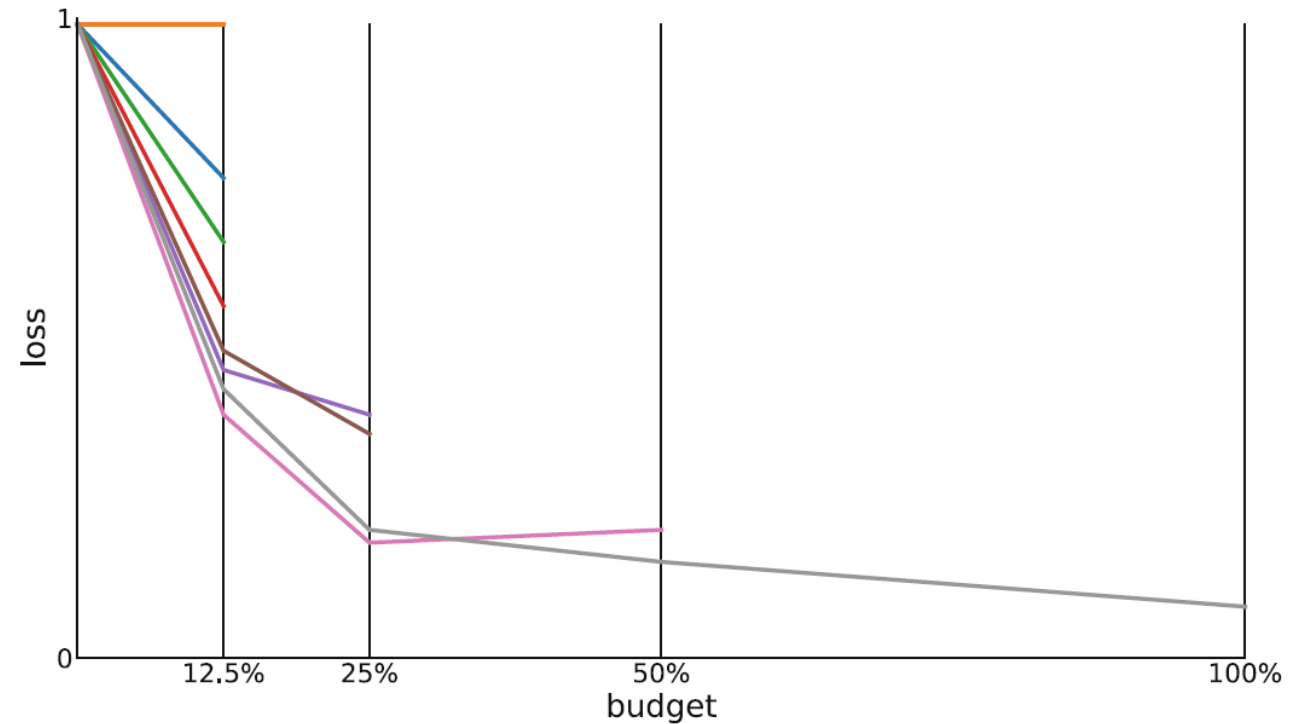
Gagner du temps lors de la phase d'optimisation des hyper-paramètres

- Processus : Entraîner les configurations les plus prometteuses

- Exemple :
par rapport à un budget temps

- Itération 1 =
Tester N configurations sur budget/N
- Itération 2 =
Tester N/2 meilleures configuration sur budget/(N/2)
- ... jusqu'à obtenir une seule configuration

ou par rapport au nombre d'observations utilisées



RÉSEAUX DE NEURONES

Pourquoi considérer des réseaux de neurones ?

- Le cerveau naturel : un modèle très séduisant
 - Capable d'apprentissage
 - Robuste et tolérant aux fautes
 - S'accommode d'informations incomplètes, incertaines, imprécises
 - Massivement parallèle
- Neurones
 - $\approx 10^{11}$ neurones dans le cerveau humain (950 x 103 dans celui de l'abeille)
 - $\approx 10^3$ à 10^4 connexions / neurone
 - Potentiel d'action / période réfractaire / neuro-transmetteurs
 - Signaux excitateurs / inhibiteurs

Vers des réseaux de neurones artificiels

- Attraites

- Calculs en parallèle
- Robustes et tolérants aux fautes (distribué)
- Algorithmes simples
- D'emploi très général

- Défauts

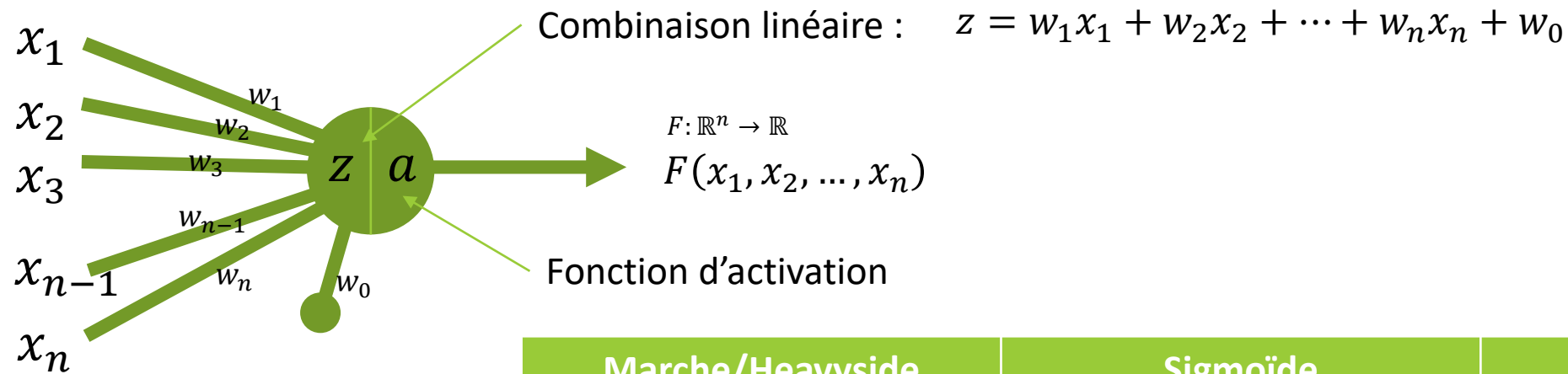
- Opacité des raisonnements...
- ... et de l'hypothèse produite

Historique

1943	W. McCulloch et W. Pitts	Modélisation du neurone biologique en neurone formel
1949	D. Hebb	Règle de Hebb : « <i>des neurones qui s'excitent ensemble se lient entre eux</i> »
1957	F. Rosenblatt	Modèle du Perceptron - Premier neuro-ordinateur
1960	B. Widrow	Modèle ADALINE (ADaptative LINear Element) inspiré du Perceptron
1969	M. Minsky et S. Papert	Mise en avant des limites du Perceptron - Abandon des recherches
1967-1982	S. Grossberg, T. Kohonen, etc.	Poursuite déguisée des recherches
1982	J. J. Hopfield	Modèle de Hopfield - Théorie du fonctionnement et des possibilités des réseaux de neurones
1983		Machine de Boltzmann
1985	Rumelhart et Y. LeCun	Réseaux de neurones multicouches - Rétropropagation du gradient

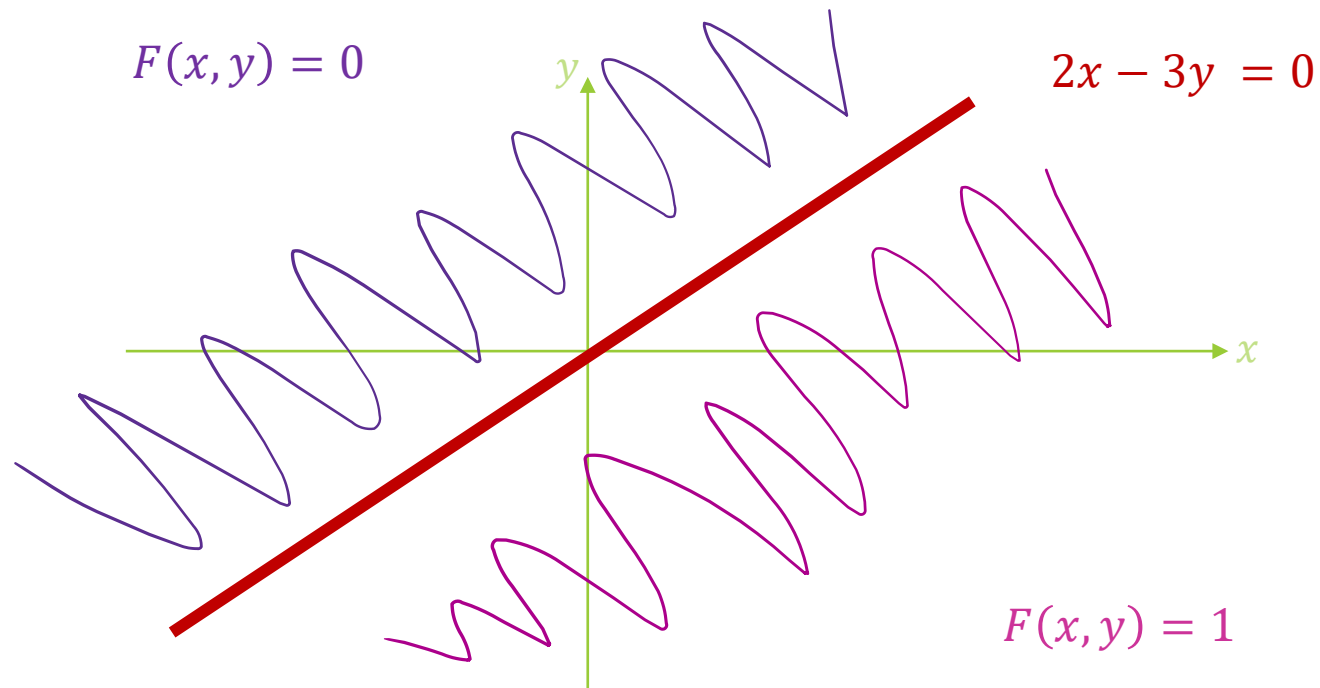
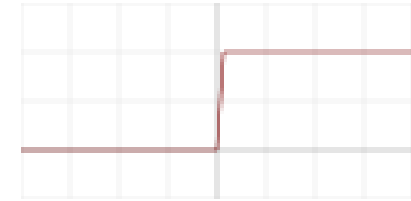
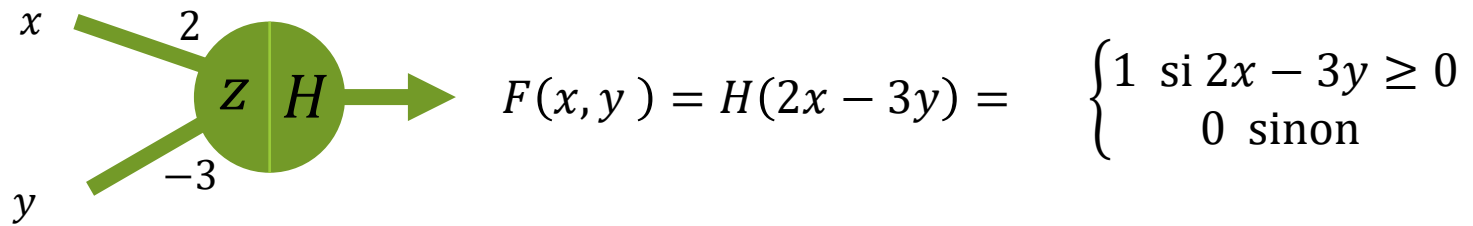
Perceptron

Entrée : $X = (x_1, x_2, \dots, x_n)$

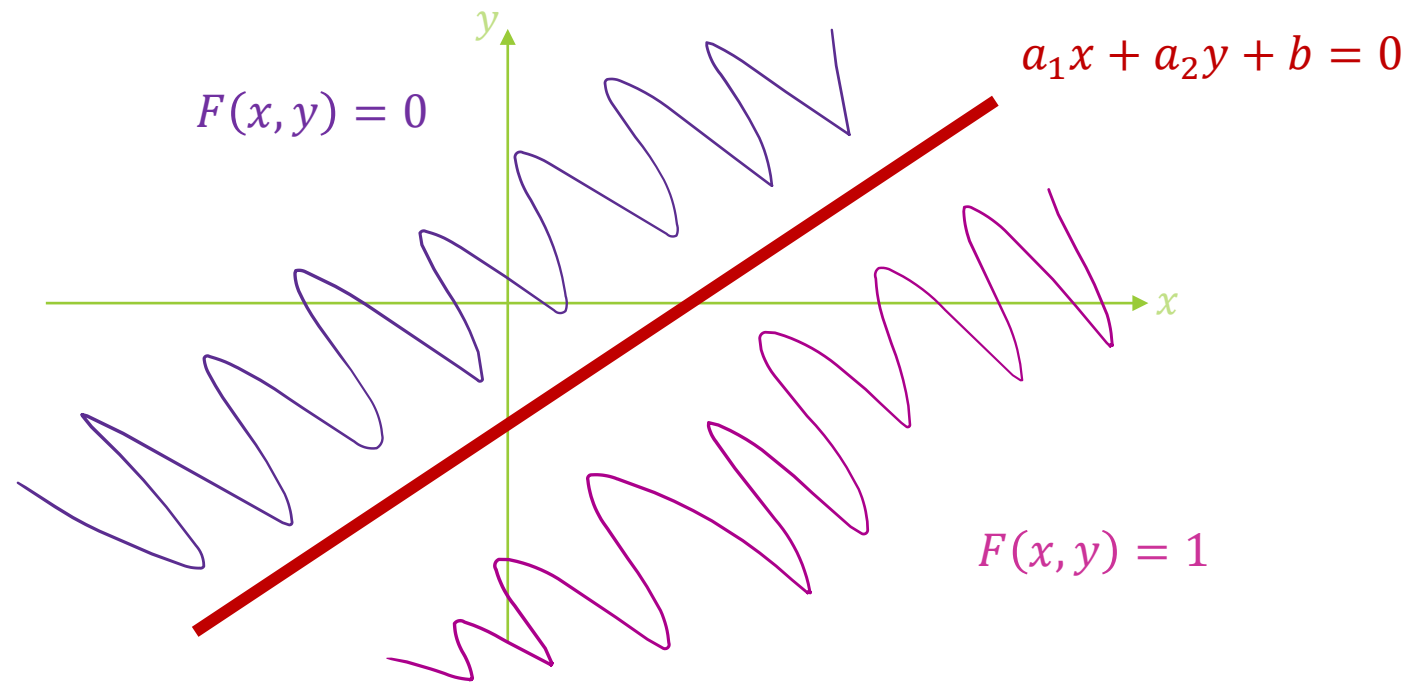
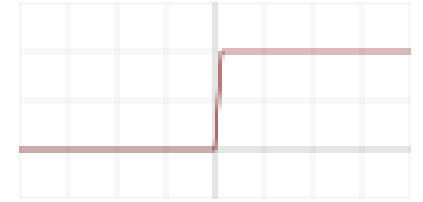
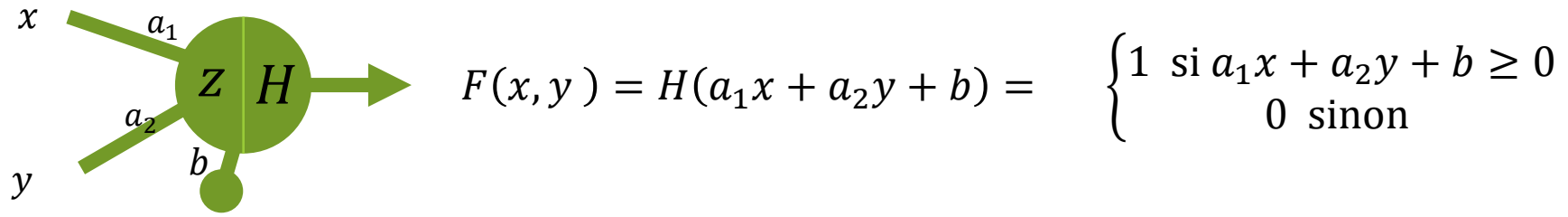


Marche/Heavyside	Sigmoïde	ReLU
$f(x) = \begin{cases} 0 & \text{si } x < 0 \\ 1 & \text{si } x \geq 0 \end{cases}$	$f(x) = \frac{1}{1 + e^{-x}}$	$f(x) = \begin{cases} 0 & \text{si } x < 0 \\ x & \text{si } x \geq 0 \end{cases}$

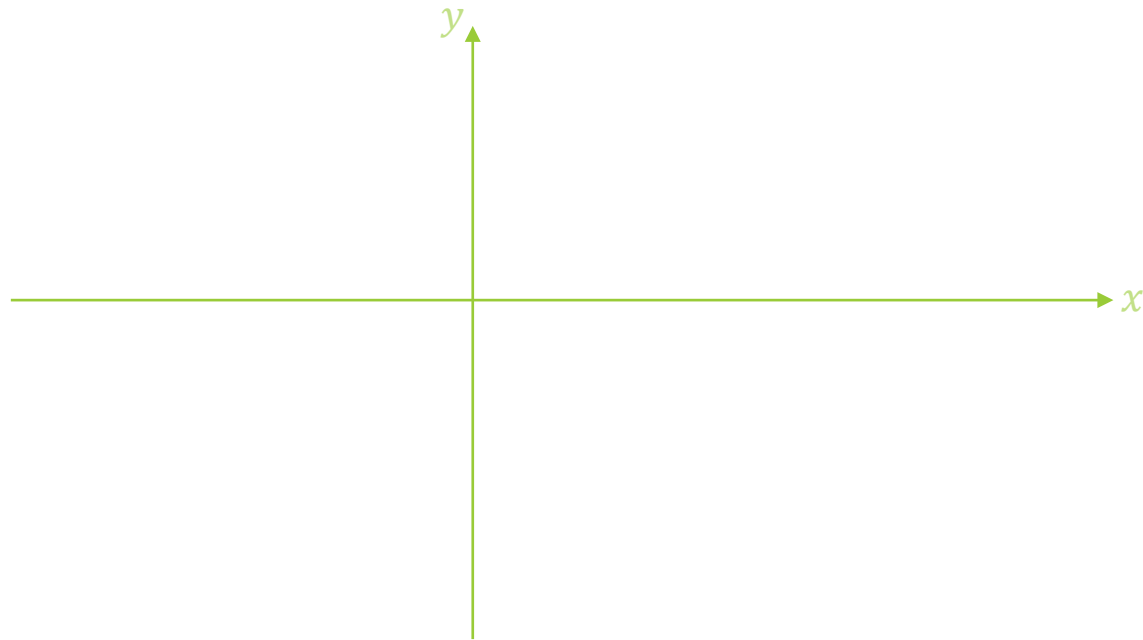
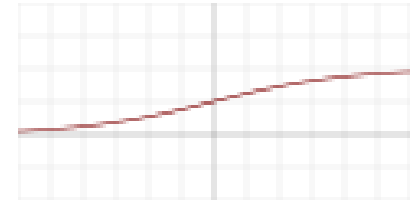
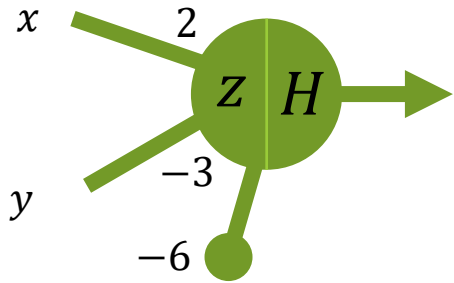
Exemple d'un neurone linéaire (Heavyside)



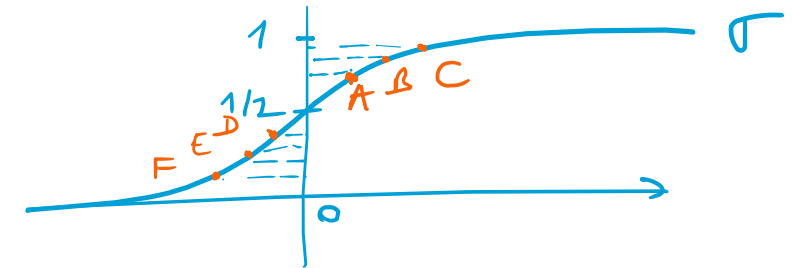
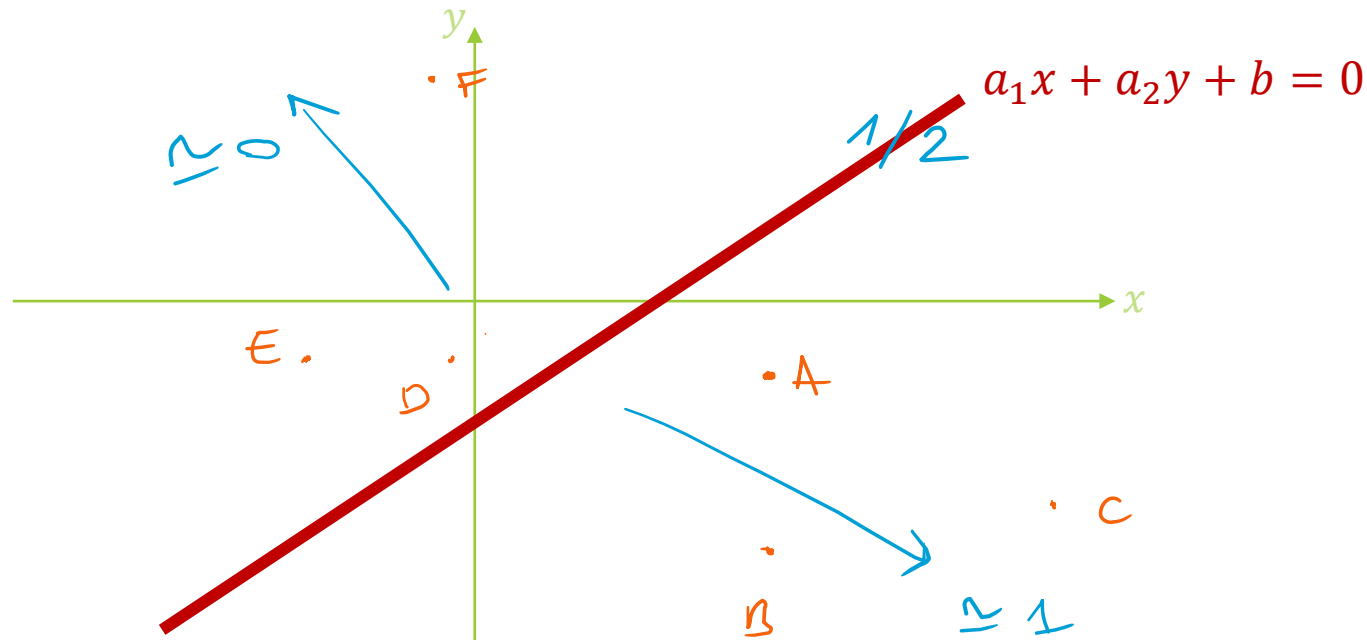
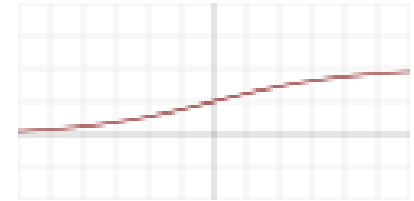
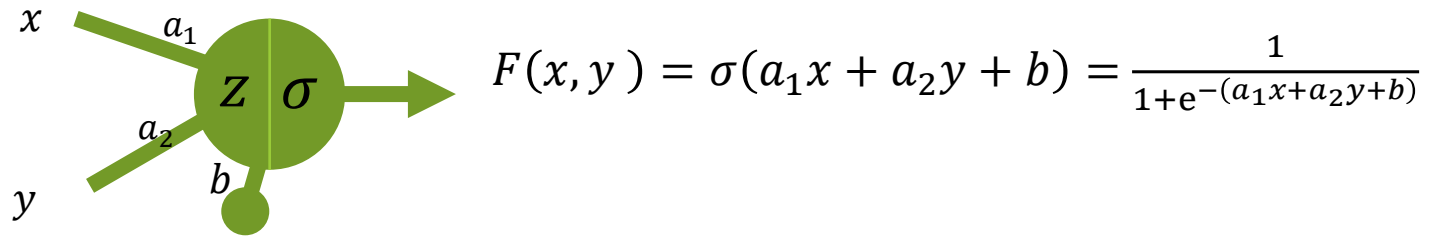
Exemple d'un neurone avec biais



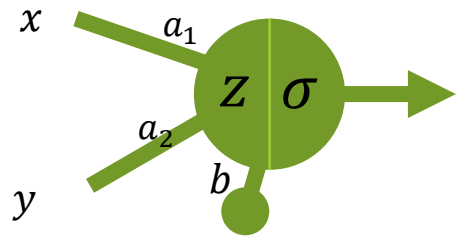
Exemple d'un neurone (Sigmoide)



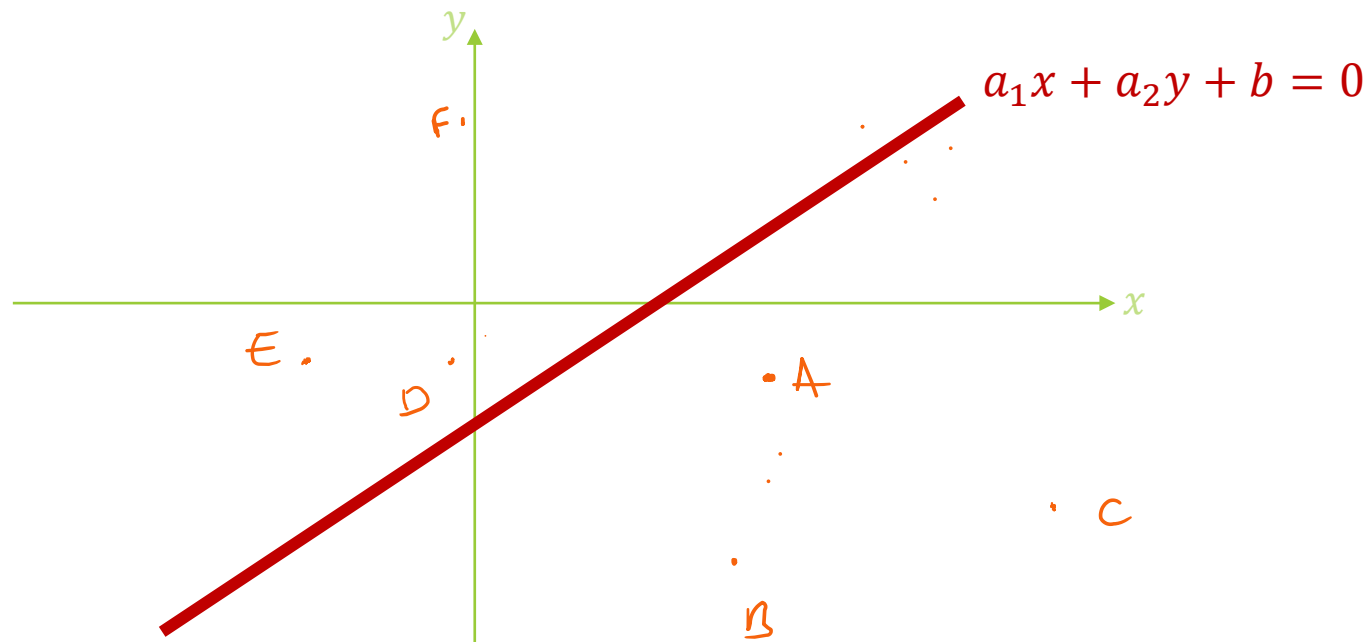
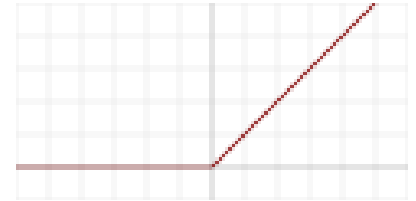
Exemple d'un neurone (Sigmoidé)



Exemple d'un neurone (ReLU)

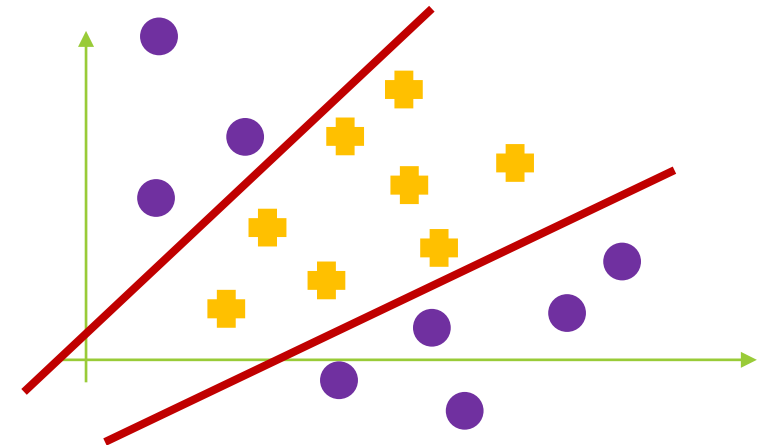
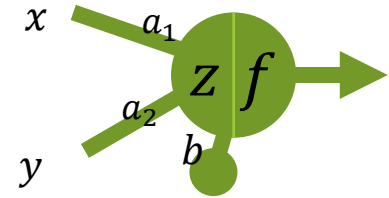


$$F(x, y) = r(a_1x + a_2y + b) = \max(0, a_1x + a_2y + b)$$

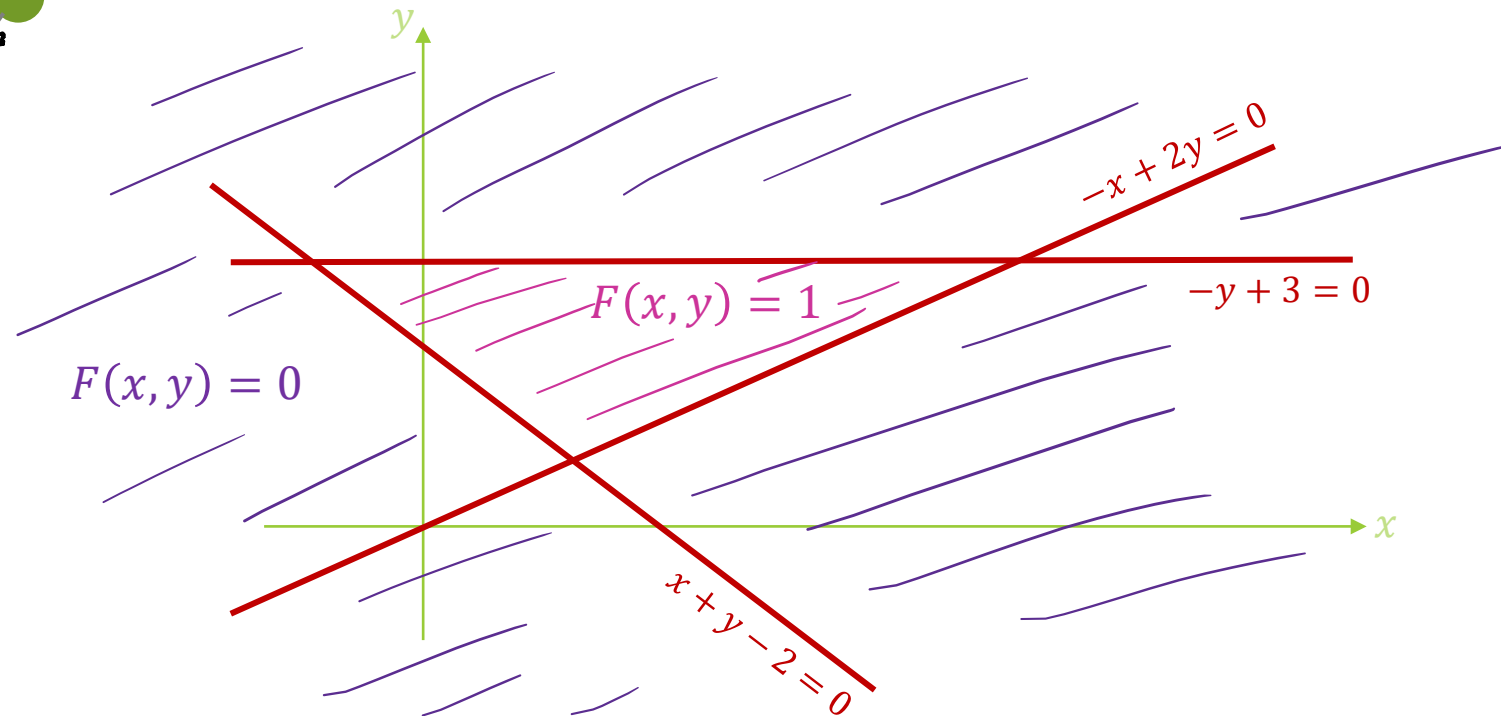
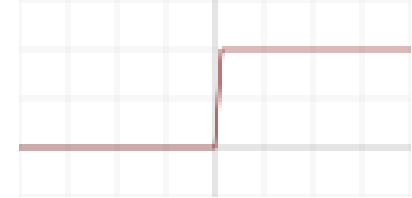
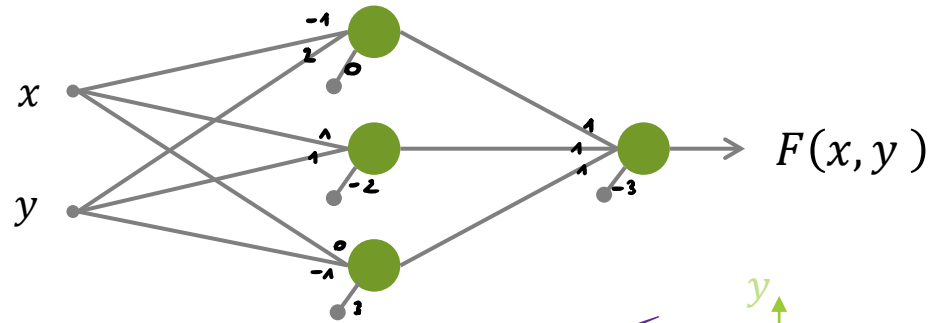


Du Perceptron au Perceptron Multi-Couche

- Perceptron
 - Variables d'entrées (features)
 - 1 neurone qui traite avec une somme pondérée les valeurs en entrée
 - Pour la classification avec $m(>2)$ classes, on a autant de neurones que de classes !
- Limitation du Perceptron :
 - Ne permet de traiter que des données séparables linéairement
- Proposition :
 - Utiliser les neurones successivement et en parallèle

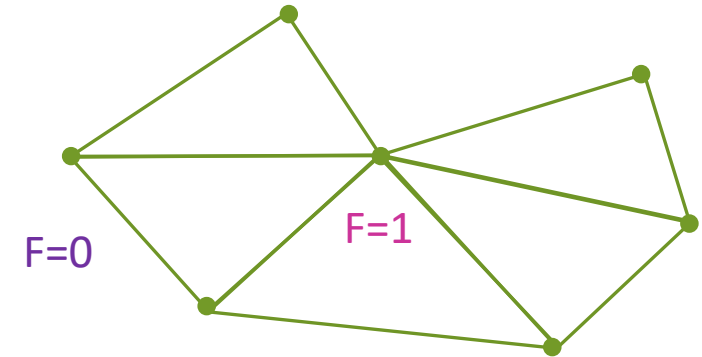


Couche de neurones : Approx. triangle

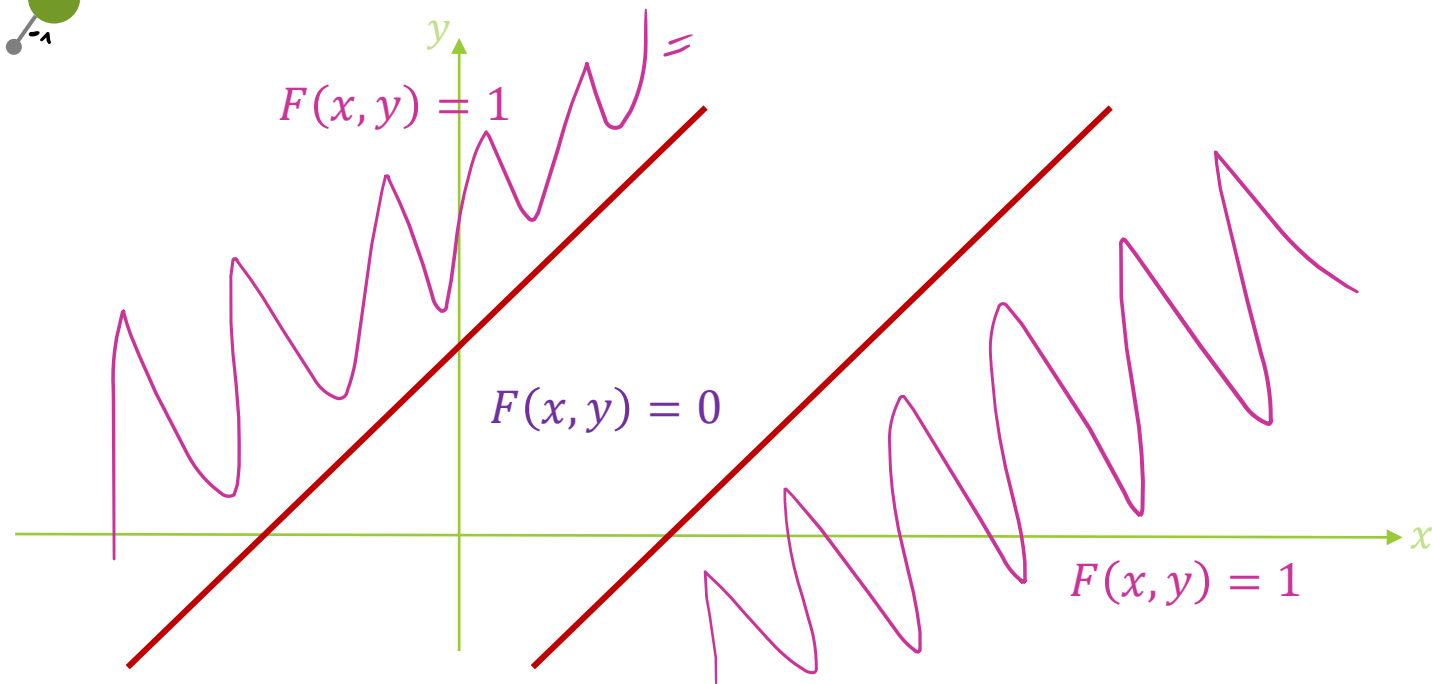
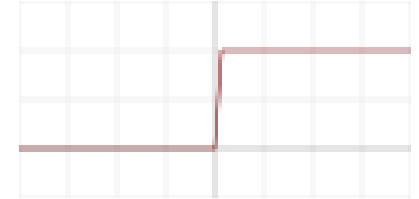
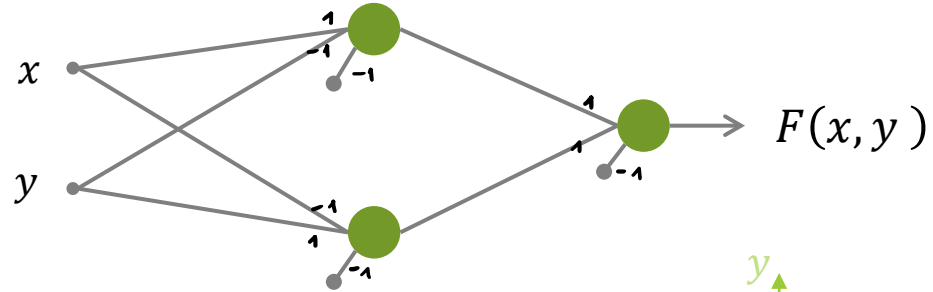


Couche de neurones : Approx. courbe de Jordan

- Toute zone polygonale est réalisable
ie. peut être construite par un réseau de neurones
- Une zone A délimitée par une courbe de Jordan
peut être approchée par un polygone A' réalisable.

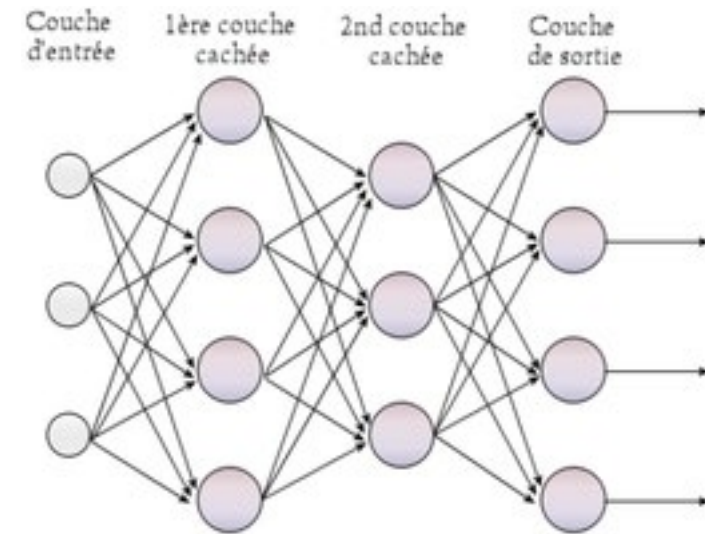


Couche de neurones : zones séparables



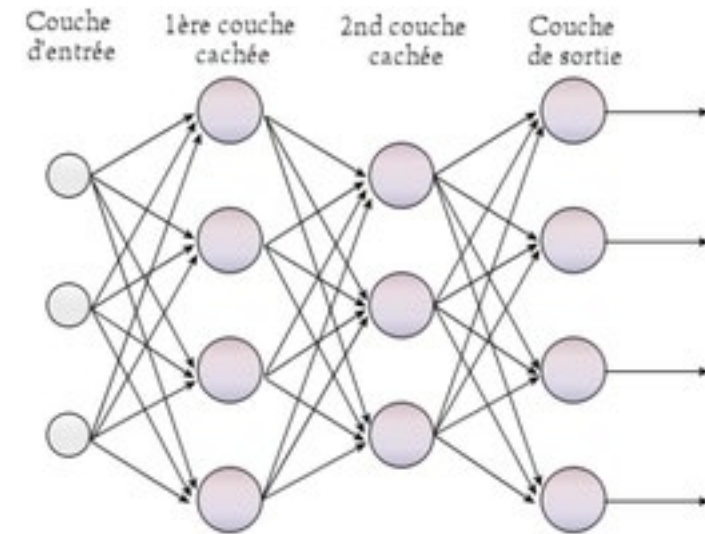
Perceptron multicouche (PMC)

- En anglais : Multi Layer Perceptron
- Un perceptron multicouche est composé de couches de neurones
- Une couche est un ensemble de neurones sans connexion entre eux
- Le Perceptron multicouche dispose
 - d'une couche d'entrée (Input layer)
 - une couche de sortie (Output layer)
 - d'une ou plusieurs couches cachées (Hidden layers)
- Les neurones sont tous connectés en entrée à chacun des neurones de la couche précédente et en sortie à chacun des neurones de la couche suivante



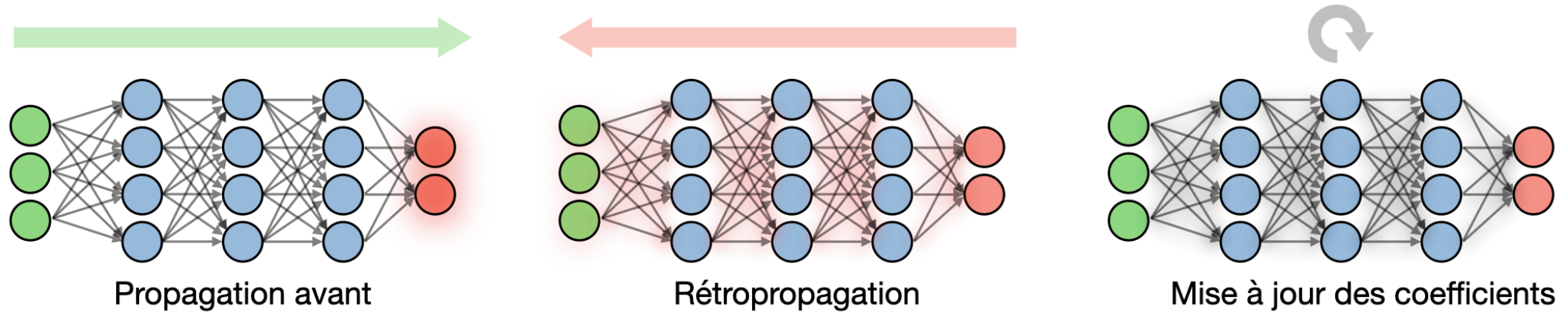
Neurones de sorties

- en régression :
 - couche de sortie = 1 neurone
 - Fonction d'activation : *identité*
- en classification binaire :
 - couche de sortie = 1 neurone
 - Fonction d'activation : *sigmoïde*
- en classification à m classes :
 - couche de sortie = m neurones
 - Fonction d'activation : *softmax*
- en classification à m classes, k étiquettes
 - Couche de sortie = m neurones
 - Fonction d'activation : *sigmoïde*



Attention : les fonctions d'activation indiquées sont celles généralement utilisées pour ce type de problème mais il est nécessaire d'en tester d'autres selon le cas

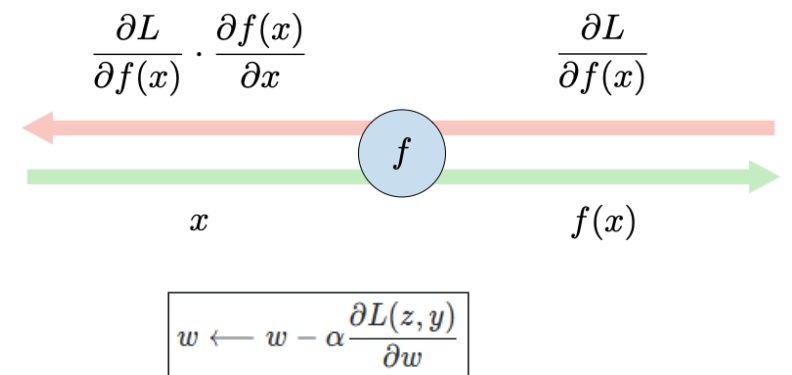
Entraînement d'un PMC



- Initialisation des coefficients de chaque neurone

Puis on exécute les étapes suivantes tant que l'erreur obtenue n'est pas satisfaisante

- Propagation avant
- Calcul de l'erreur
- Propagation arrière
 - Utilisation de la méthode de **rétro-propagation du gradient**
- Mise à jour des coefficients



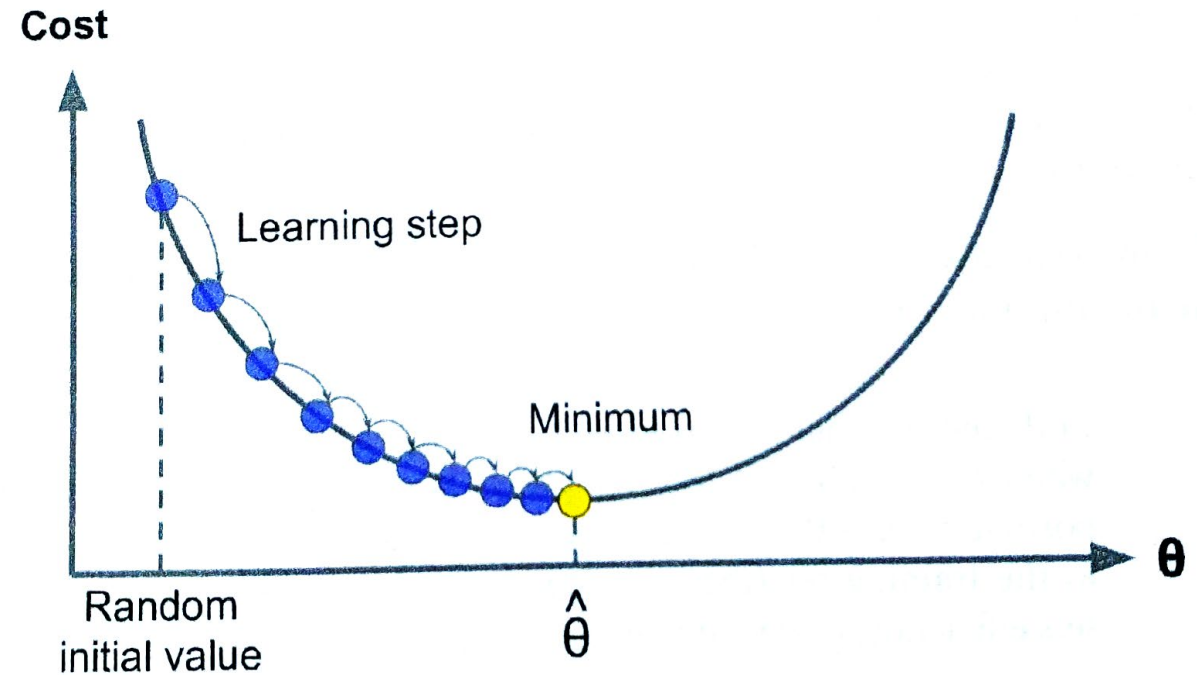
Descente du gradient

- Algorithme d'optimisation numérique capable de trouver les solutions optimales d'une fonction
- Principe : Ajuste itérativement la valeur des variables

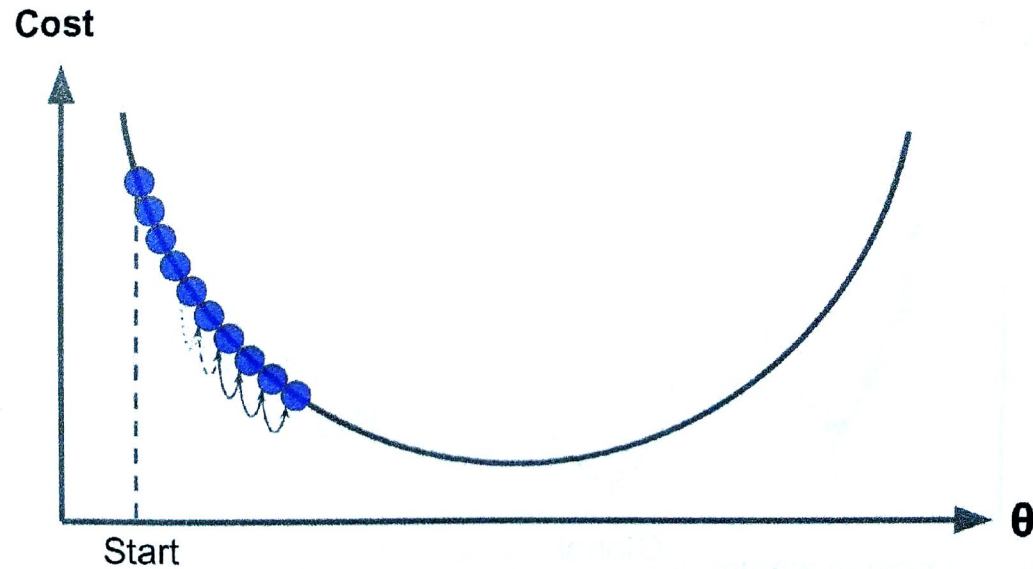
Exemple en régression/classification :

- Fonction = fonction erreur du modèle
- Solution = paramètre θ du modèle

But : Trouver $\hat{\theta}$ qui minimise la fonction d'erreur du modèle construit

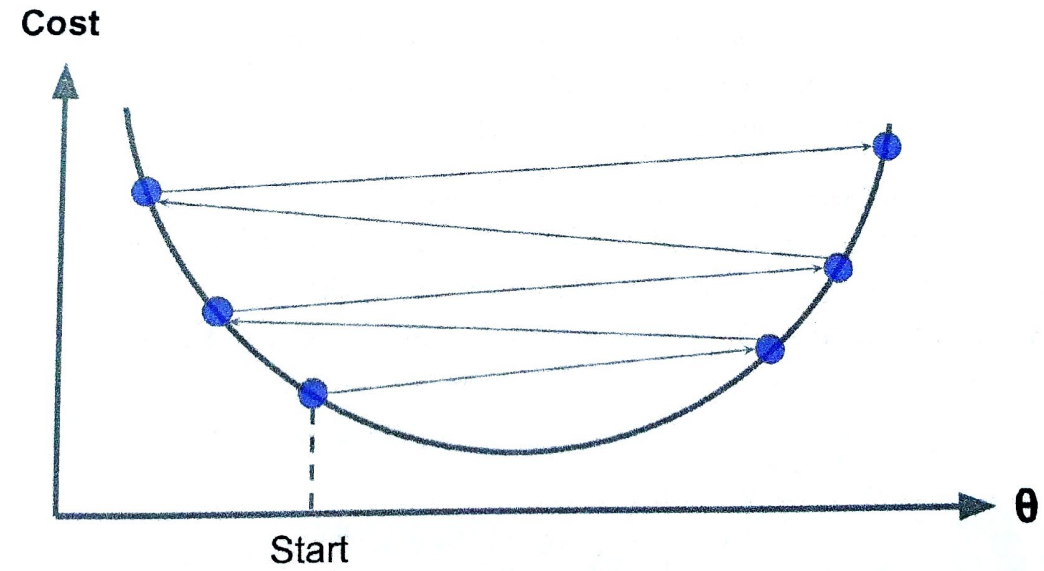


Taux d'apprentissage



Taux d'apprentissage trop petit :

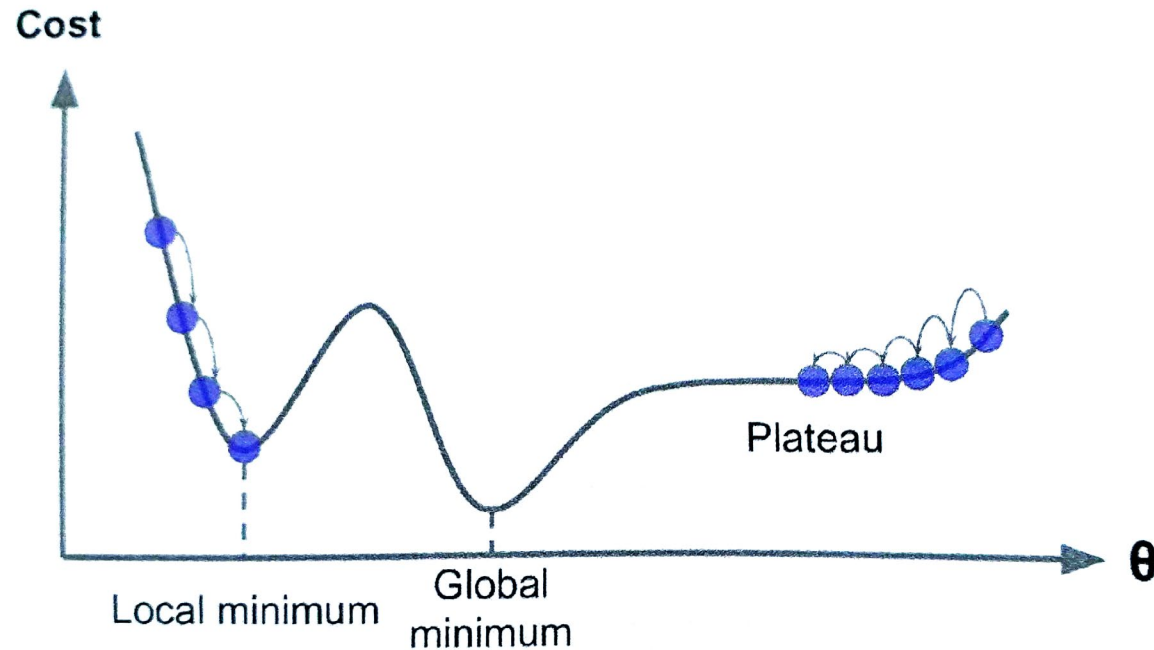
- Très long à converger



Taux d'apprentissage trop grand :

- L'algorithme peut diverger

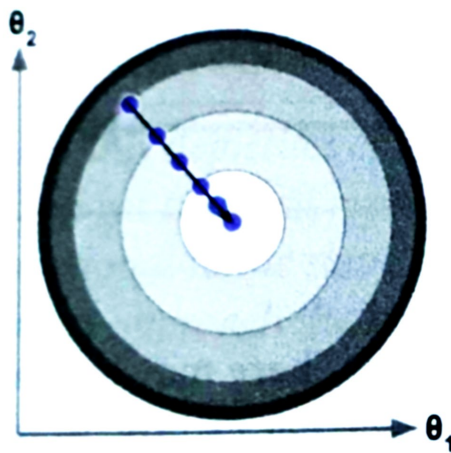
Pièges pour la descente du gradient



- Présence d'optima locaux
- Présence de zones plates

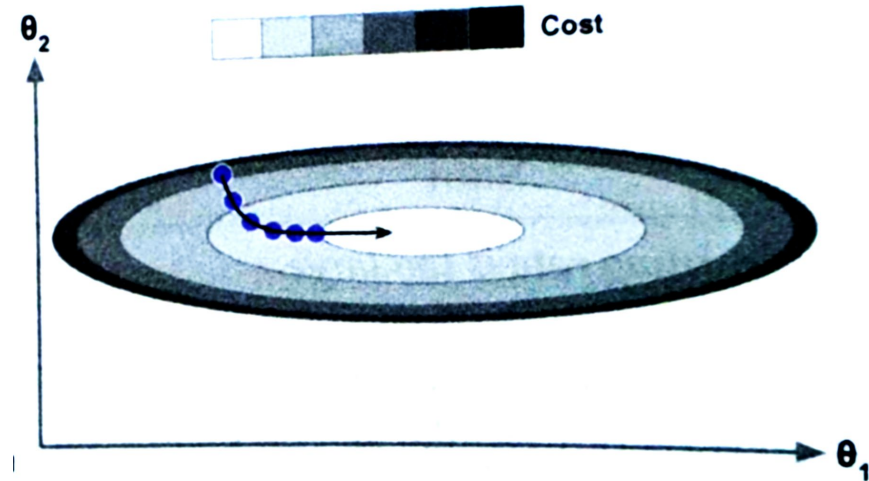
Pièges pour la descente du gradient

- Différentes échelles des paramètres



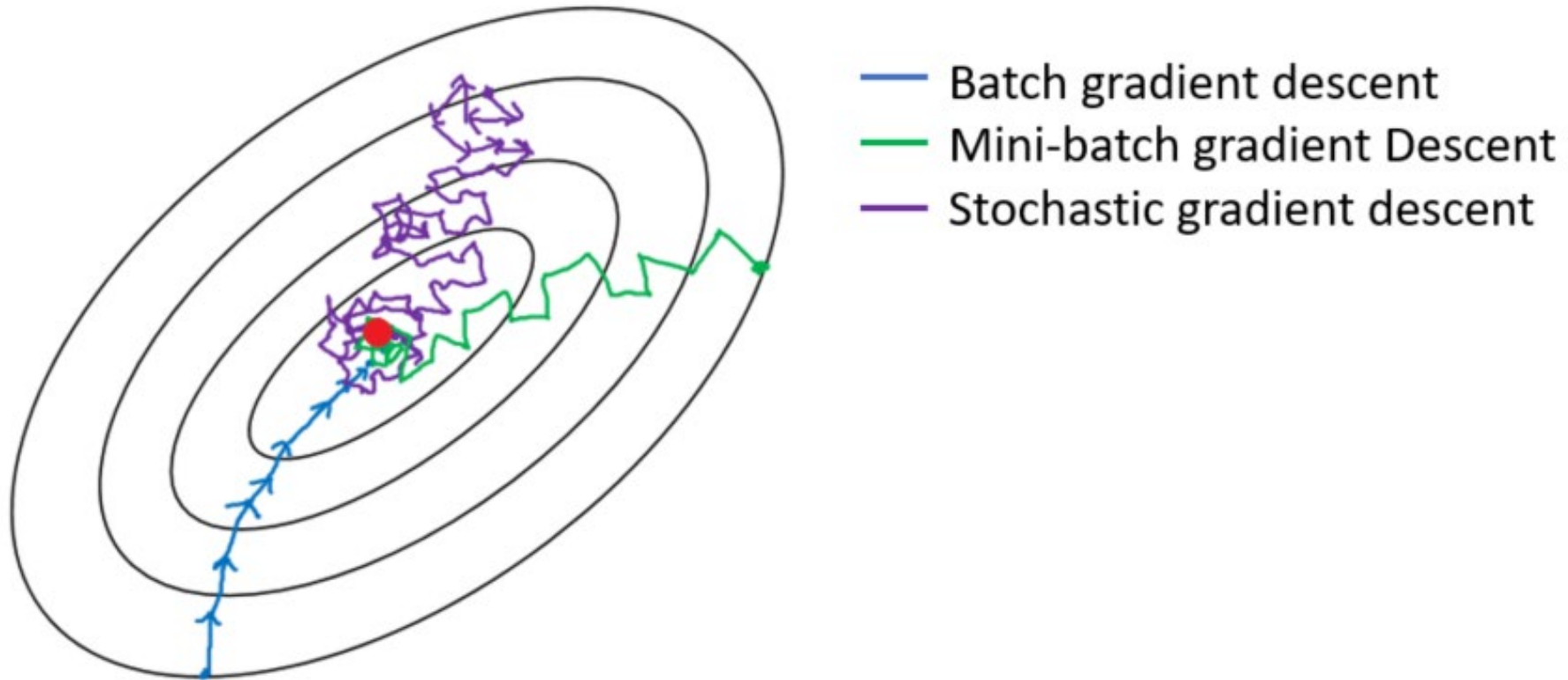
de même ordre de grandeur

Paramètres θ_1 et θ_2
vs.



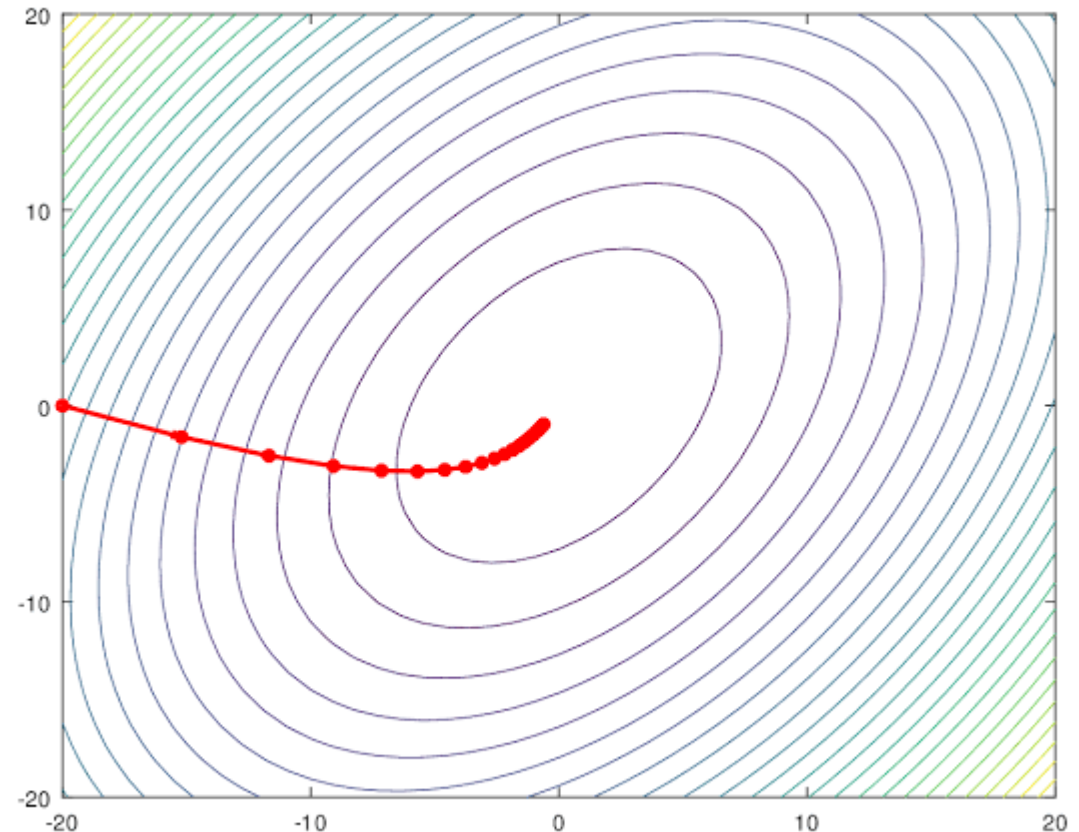
d'ordre de grandeur très différents

Descente de gradient en pratique



Descente de gradient par lot

- A chaque itération,
 - Toutes les observations sont passées au modèle
 - L'erreur est calculée sur toutes les observations
- Avantage
 - Très efficace
- Inconvénients :
 - Très lent
 - Très gourmand en ressources de calcul

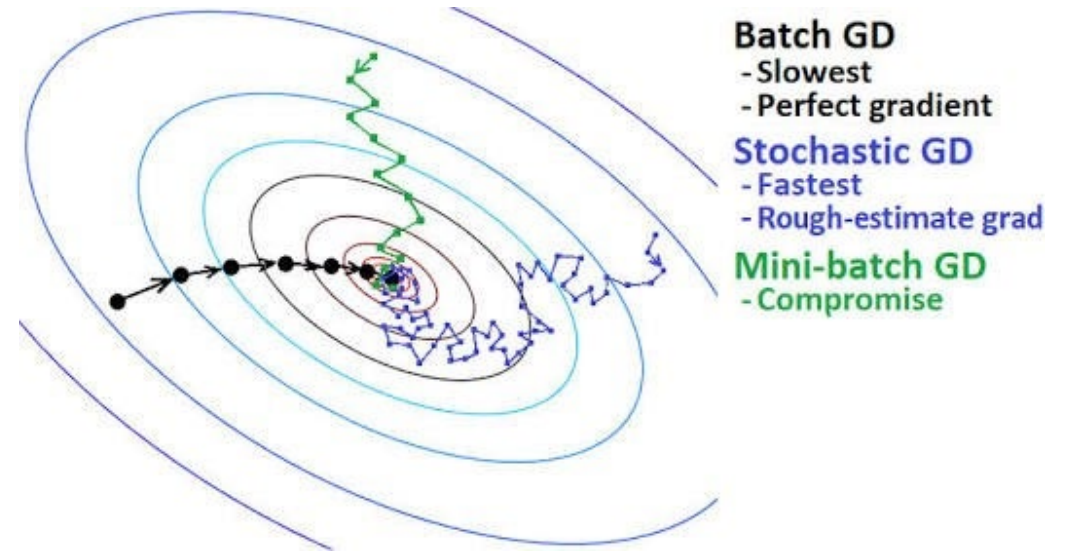


Descente de gradient stochastique

- A chaque itération,
 - Une seule observation est passée au modèle
 - L'erreur du gradient est calculée sur une seule observation
 - Chaque observation est choisie aléatoirement sans remise
- Avantage
 - Chaque itération est très rapide
- Inconvénients :
 - Bcp d'oscillations dans l'espace
 - Plus difficile de contrôler le taux d'apprentissage

Descente de gradient par mini-lots

- A chaque itération,
 - Un *petit* sous-ensemble d'observations est passé au modèle
 - L'erreur du gradient est calculée sur le sous-ensemble d'observations considéré
 - Chaque sous-ensemble d'observations est choisi aléatoirement sans remise
- Avantage
 - Chaque itération est assez rapide
- Inconvénients :
 - Oscillations dans l'espace
 - Plus difficile de contrôler le taux d'apprentissage

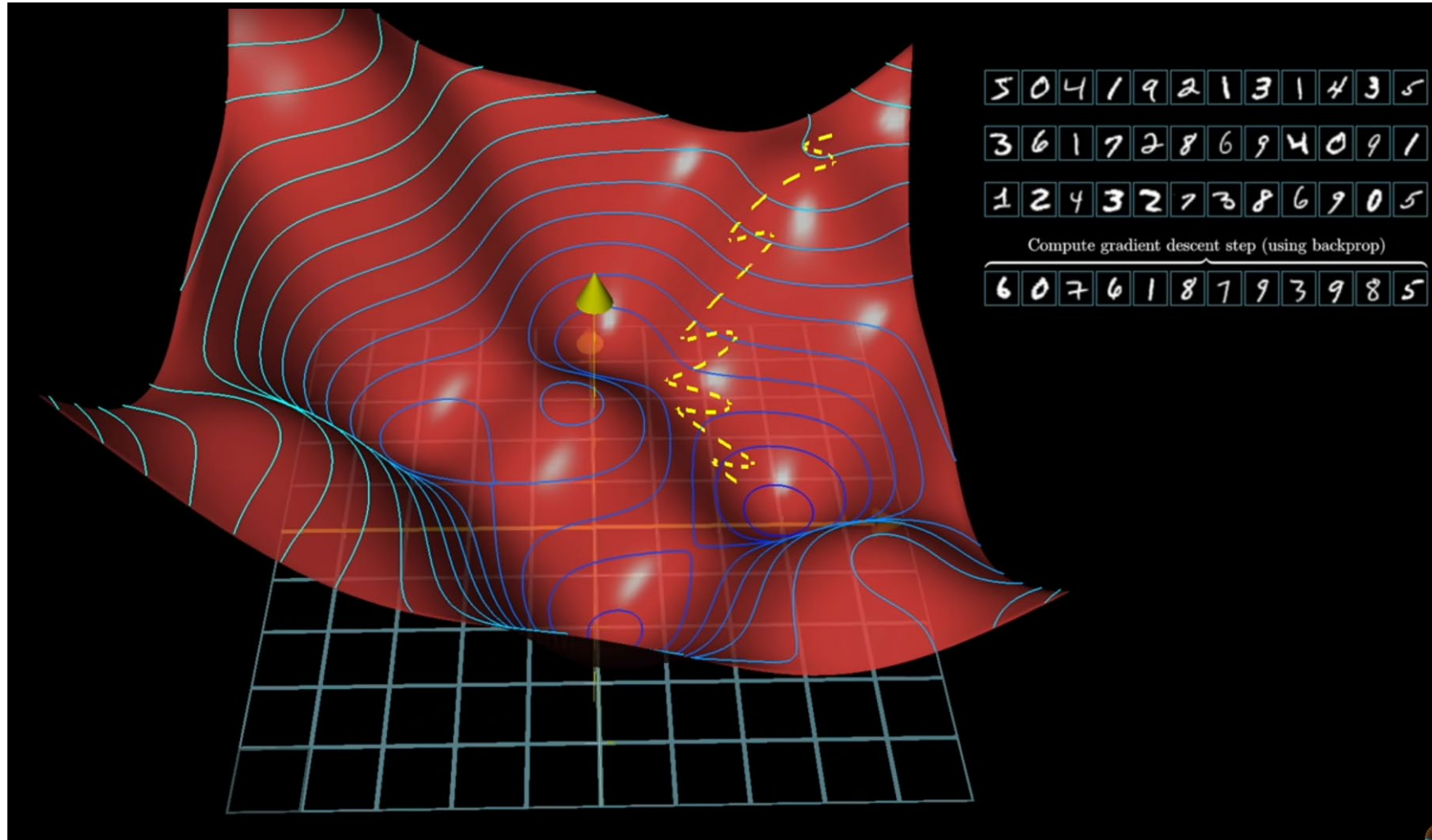


Mini-batch Gradient Descent

“Mini-batches”

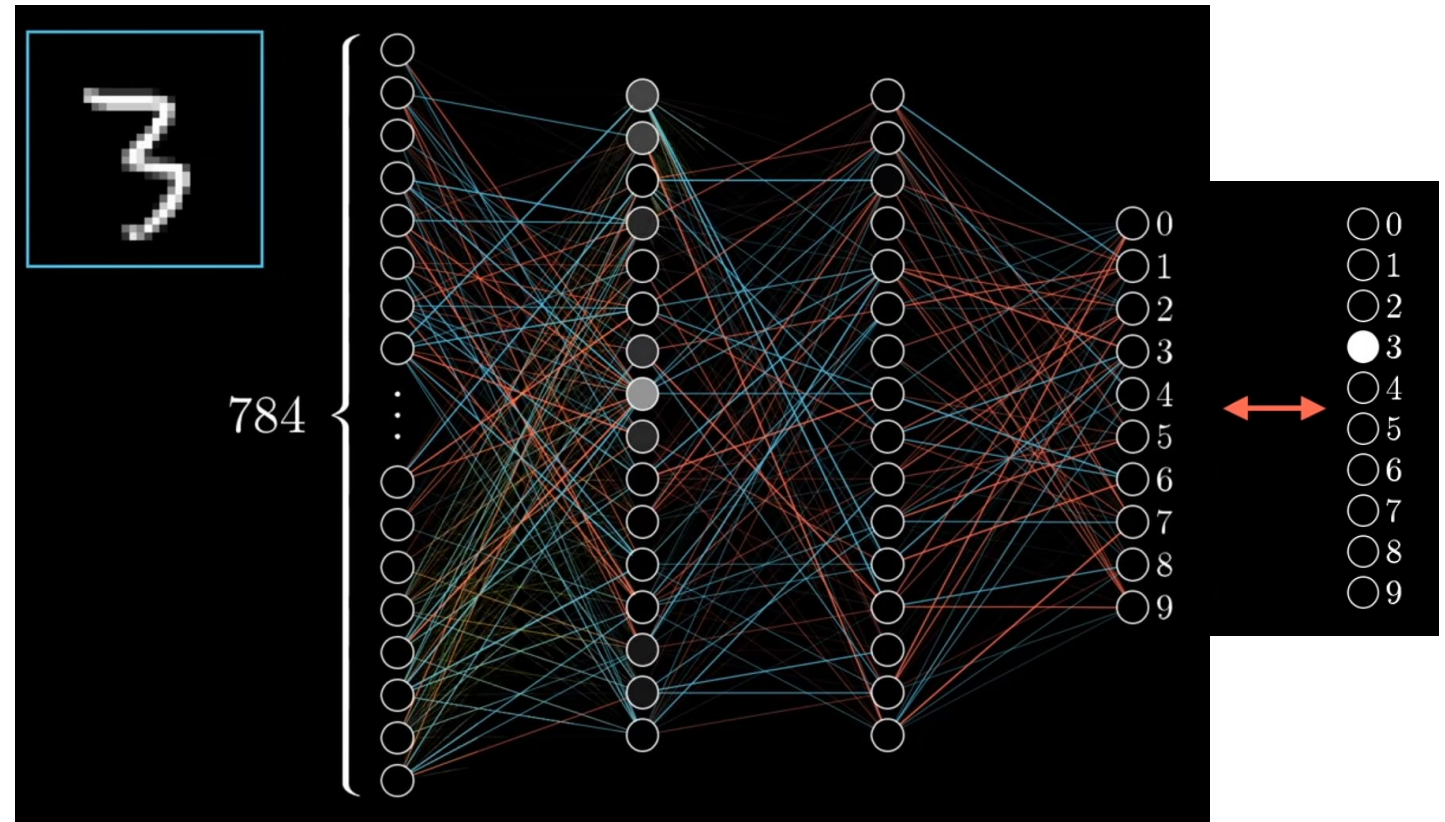


Stochastic Gradient Descent



Entraînement du PMC

- Réseau de neurones à propagation avant (feedforward NN) :
 - Information ne se déplace que dans une seule direction
 - Départ = nœuds d'entrée
 - Passage par les couches cachées
 - Arrivée = nœuds de sortie



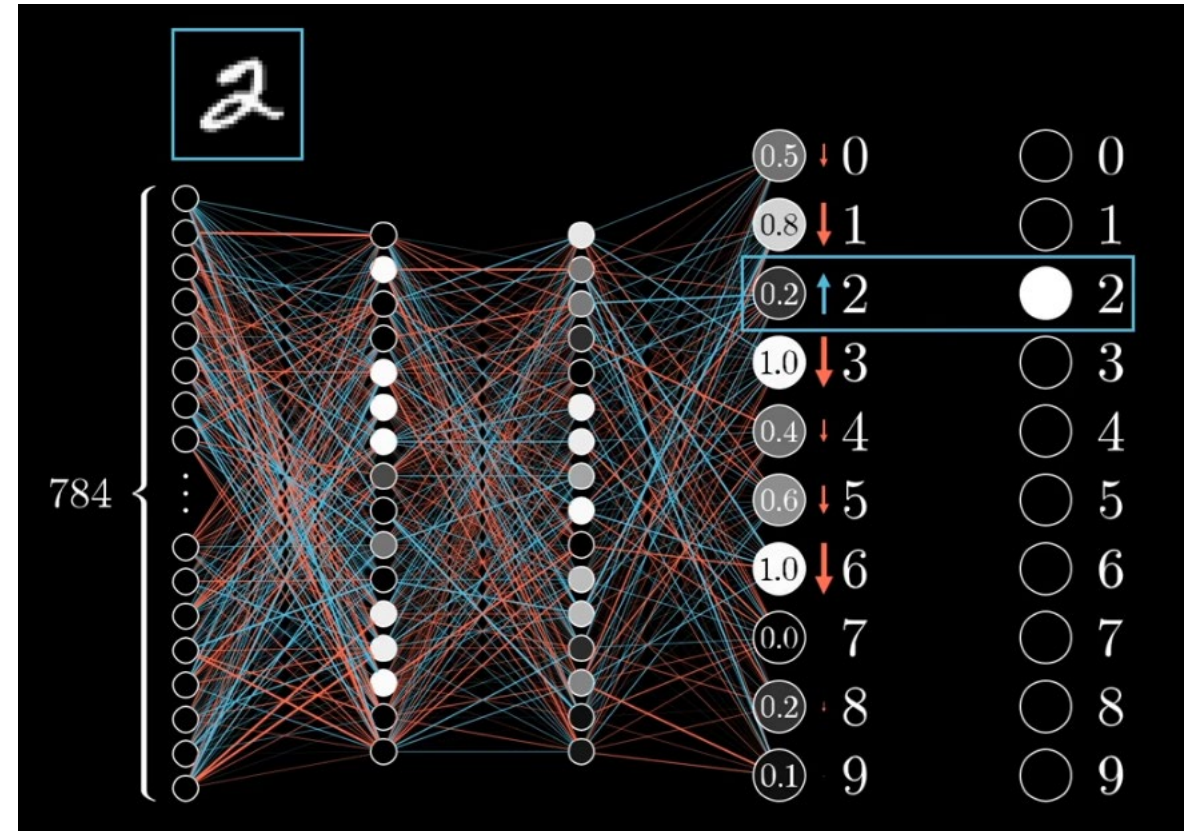
Entraînement du PMC

- Evaluation = Calcul de la fonction de coût
 - Quantifie l'erreur entre la réponse estimée et la bonne réponse sur des données d'entraînement.
 - Moyenne des écarts pour chaque classe
 - Régression : MSE

$$\frac{1}{L} \sum_{i=1}^L (y_i - \hat{y}_i)^2$$

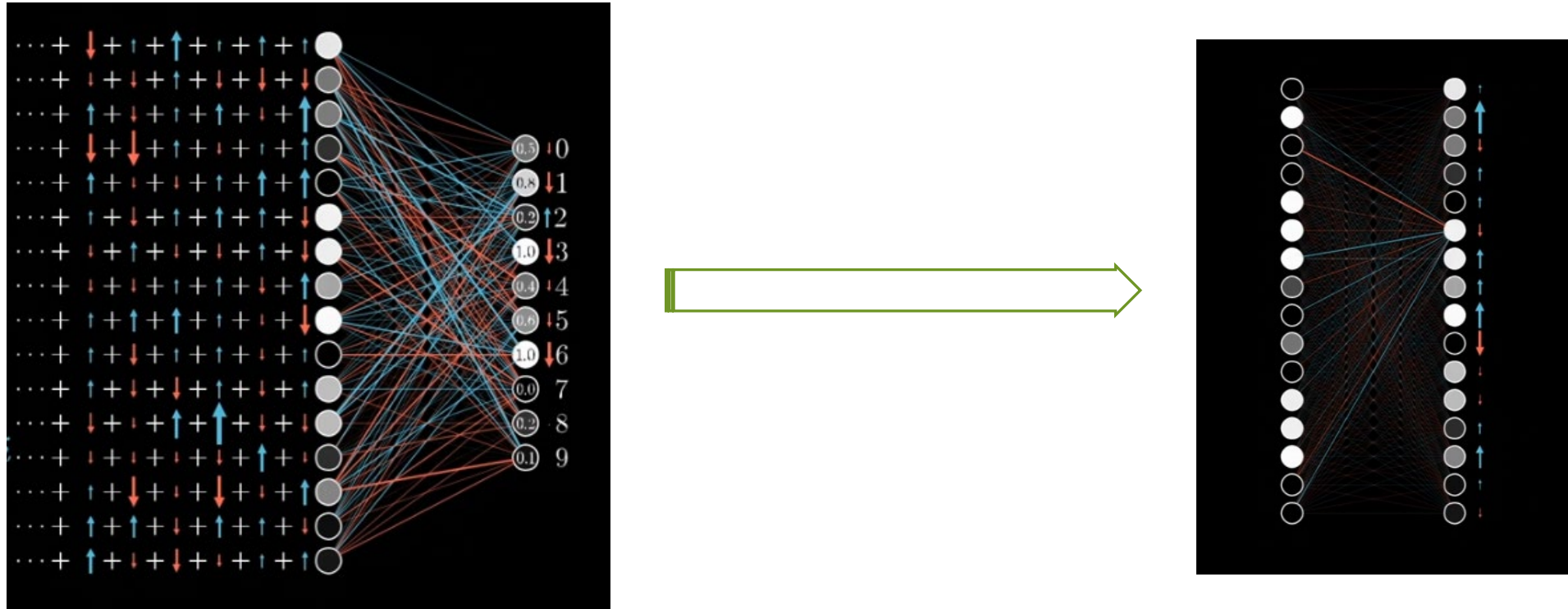
- Classification : LogLoss (Entropy)

$$-\frac{1}{L} \sum_{i=1}^L y_i \log(p(y_i)) + (1 - y_i) \log(1 - p(y_i))$$

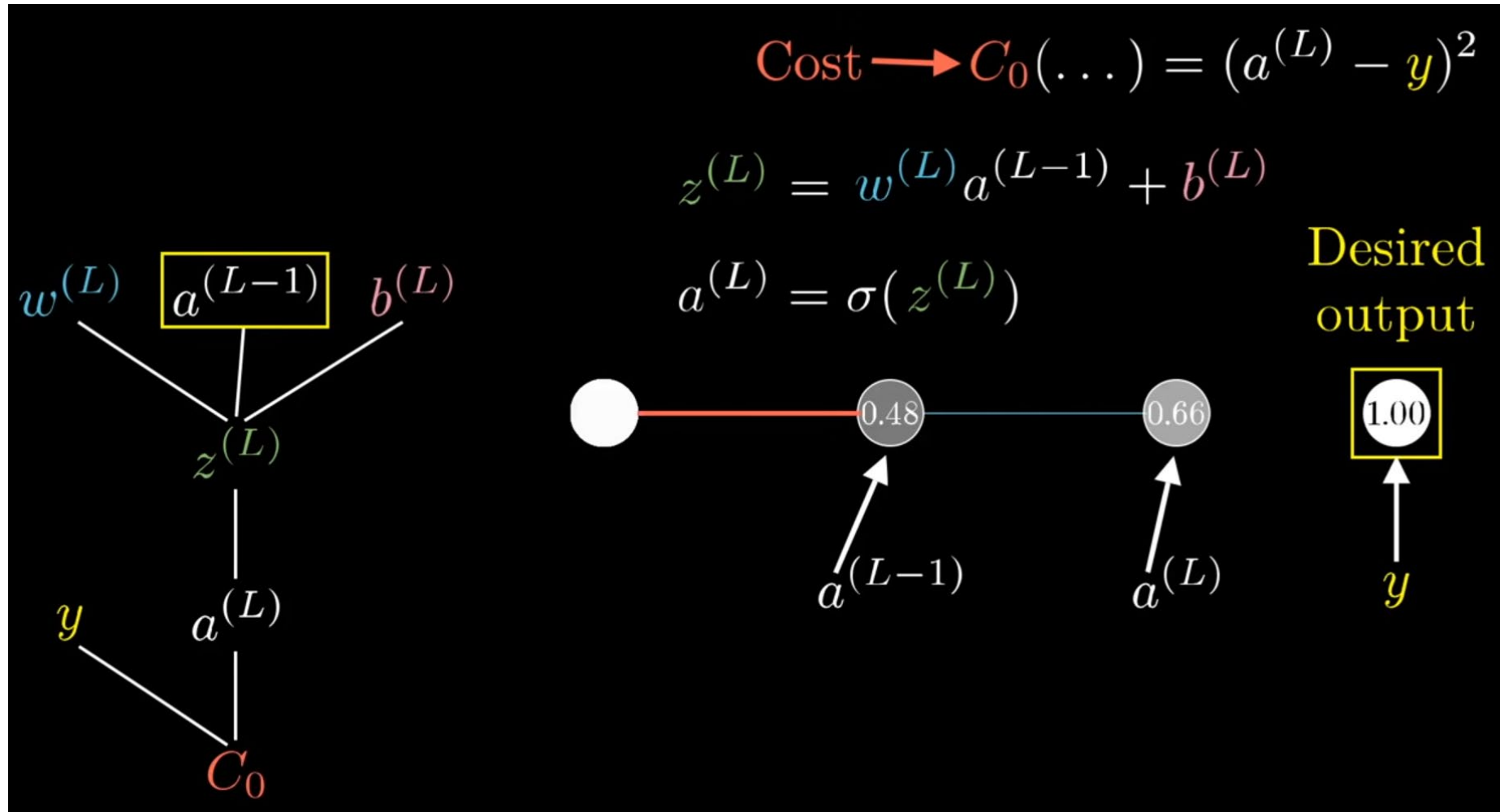


Calcul de l'erreur

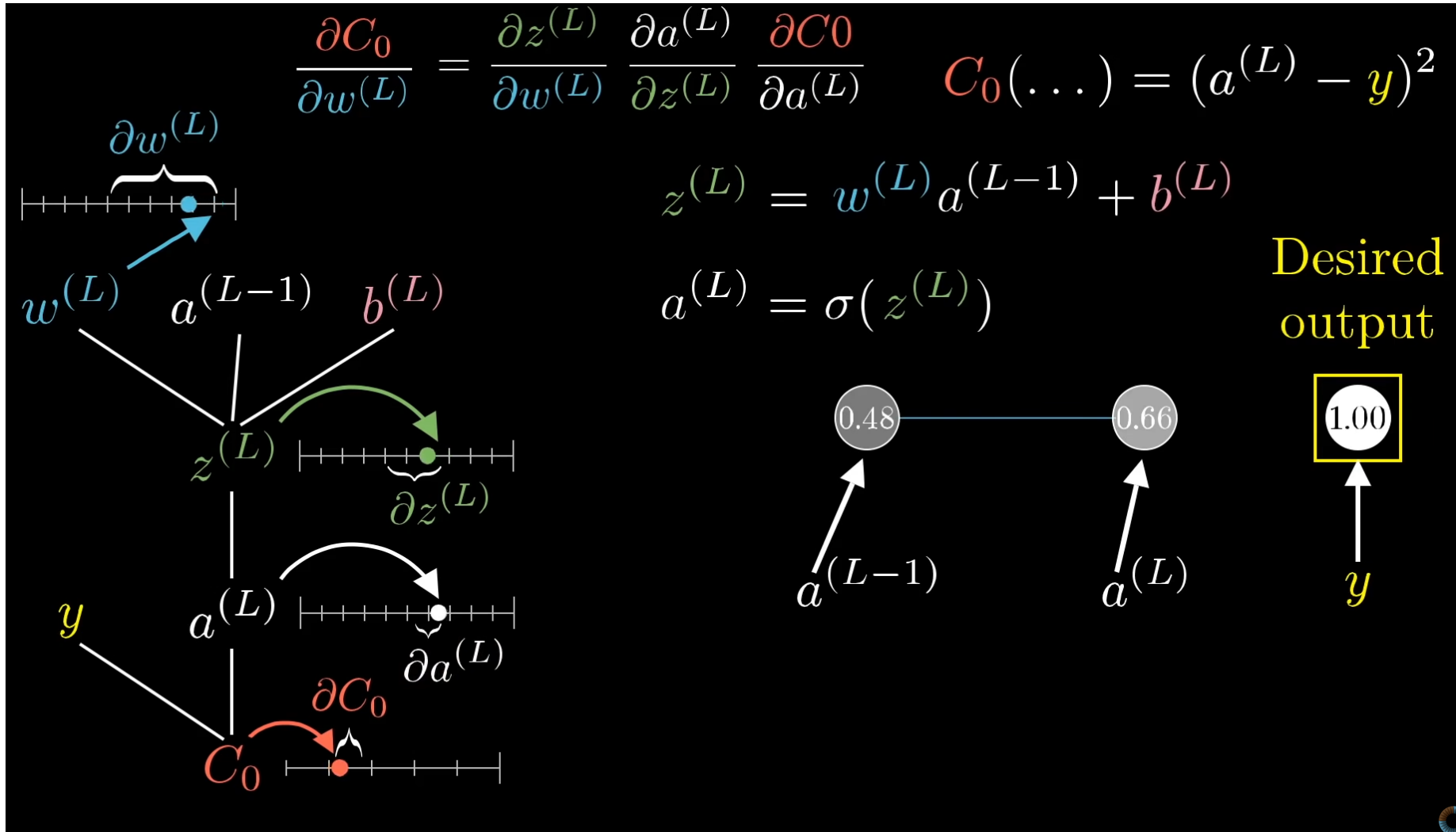
- Erreur de chaque neurone de la couche précédente par rapport à la réponse attendue pour chaque neurone de la couche de sortie



Rétropropagation du gradient

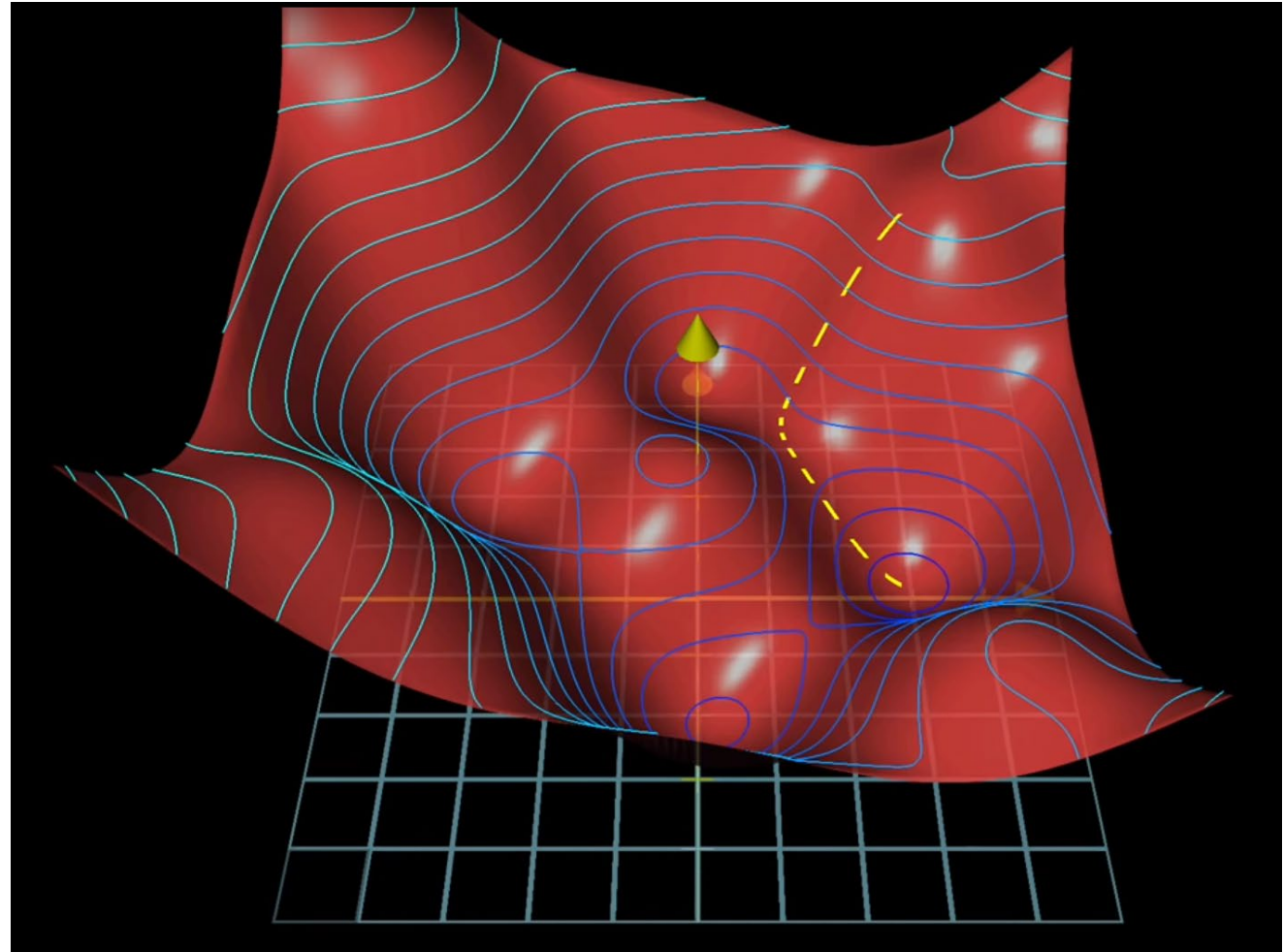


Rétropropagation du gradient

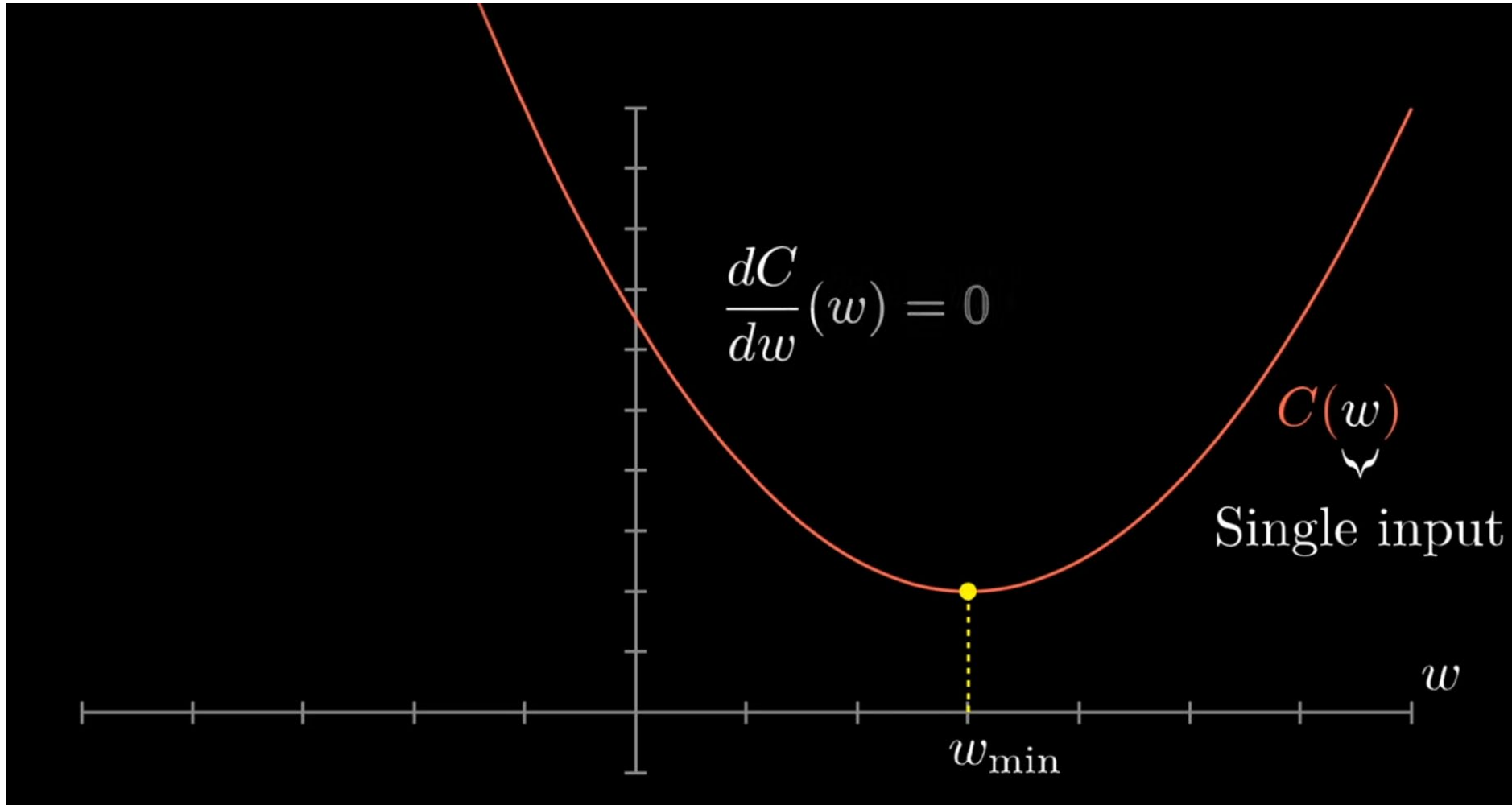


Rétropropagation du gradient

$$\nabla C = \begin{bmatrix} \frac{\partial C}{\partial w^{(1)}} \\ \frac{\partial C}{\partial b^{(1)}} \\ \vdots \\ \frac{\partial C}{\partial w^{(L)}} \\ \frac{\partial C}{\partial b^{(L)}} \end{bmatrix}$$



Descente du gradient



Rétropropagation du gradient

$$-\nabla C(\vec{\mathbf{W}}) = \begin{bmatrix} 0.18 \\ 0.45 \\ -0.51 \\ \vdots \\ 0.40 \\ -0.32 \\ 0.82 \end{bmatrix}$$



$$\vec{\mathbf{W}} = \begin{bmatrix} 2.43 \\ -1.12 \\ 1.47 \\ \vdots \\ -0.76 \\ 3.71 \\ 1.21 \end{bmatrix}$$

Rétropropagation sur plusieurs observations

							Average over all training data ...
w_0	-0.08	+0.02	-0.02	+0.11	-0.05	-0.14	... → -0.08
w_1	-0.11	+0.11	+0.07	+0.02	+0.09	+0.05	... → +0.12
w_2	-0.07	-0.04	-0.01	+0.02	+0.13	-0.15	... → -0.06
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$
$w_{13,001}$	+0.13	+0.08	-0.06	-0.09	-0.02	+0.04	... → +0.04

Entrainement du PMC

- Algorithme de descente de gradient
 - Ajuste les poids pour minimiser la fonction de coût
= modifications itératives en soustrayant le gradient de la fonction de coût.
 - Nécessite de calculer la variation du coût à la sortie du réseau relativement à la variation des paramètres du réseau
 - Ajustement des poids pour chaque entrée -> très coûteux en temps de calcul...
- Epoch
 - Nombre d'itérations où le modèle voit tout le training set pour mettre à jour ses coefficients.
- Mini-batches
 - Entrainement par lots avec un sous-ensemble des inputs